



PYTHON VARIABLES

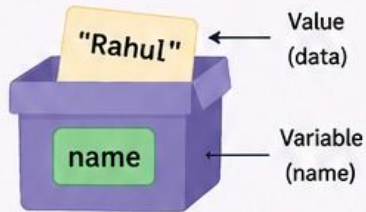
Store data. Reuse it. Make your code **smart!**



1. WHAT IS A VARIABLE?

A variable is a named container used to store data in memory.

It allows you to store a value and use it later in your program.



2. SYNTAX

variable_name = value

Example:

```
name = "Rahul"
age = 18
is_student = True
```

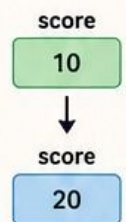
3. EXAMPLES

Variable	Value	Data Type	Example
name	"Rahul"	str (String)	name = "Rahul"
age	18	int (Integer)	age = 18
height	5.8	float (Float)	height = 5.8
is_student	True	bool (Boolean)	is_student = True

4. REASSIGNING VARIABLES

You can change the value of a variable.

```
score = 10
print(score) # 10
score = 20
print(score) # 20
```



5. MULTIPLE VARIABLES

a) Multiple assignment

```
a = 10
b = 20
c = 30
```

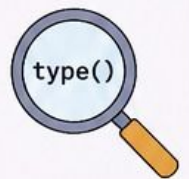
b) Assigning multiple values

```
x, y, z = 1, 2, 3
print(x, y, z) # 1 2 3
```

6. DATA TYPE OF A VARIABLE

Use **type()** function to check data type.

```
name = "Rahul"
age = 18
print(type(name)) # <class 'str'>
print(type(age)) # <class 'int'>
```



7. RULES FOR NAMING VARIABLES

- ✓ Can contain letters (a-z, A-Z), digits (0-9) and underscore (_).
- ✓ Must start with a letter or underscore (_).
- ✓ Case-sensitive (age, Age and AGE are different).
- ✗ Cannot start with a digit.
- ✗ Cannot use reserved keywords.
- ✗ No spaces allowed.

Valid Examples

- ✓ student_name
- ✓ _age
- ✓ age1
- ✓ total_marks

Invalid Examples

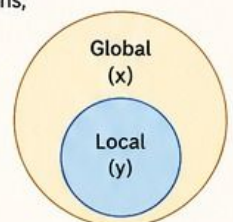
- ✗ 1name
- ✗ student name
- ✗ for
- ✗ @marks

8. SCOPE OF VARIABLES

a) **Local Variable:** Defined inside a function, accessible only inside that function.

b) **Global Variable:** Defined outside all functions, accessible throughout the program.

```
x = 100 # global variable
def my_func():
    y = 50 # local variable
    print(y)
my_func()
print(x) # 100
```



9. DYNAMIC TYPING

Python is dynamically typed. You don't need to declare the type of variable. It decides automatically.

```
a = 10 # int
a = "Hello" # str
a = 3.14 # float
a = False # bool
```

10. CONSTANTS IN PYTHON

Python doesn't have constants, but by convention we use UPPER_CASE names.

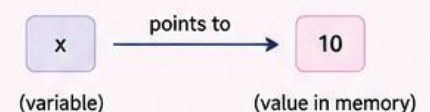
```
PI = 3.14159
GRAVITY = 9.8
NAME = "Earth"
```

(These are variables, but treated as constants by programmers.)



11. MEMORY IDEA (BASIC)

A variable holds a reference to the value in memory.



12. COMMON MISTAKES

- ✗ Forgetting quotes for strings: name = Rahul ✗
- ✗ Using spaces in variable names: student name ✗
- ✗ Starting with a digit: 1age ✗
- ✗ Using keywords: for = 10 ✗

QUICK RECAP

- 📁 Variables store data.
- 📁 Use meaningful names.
- 📁 Follow naming rules.
- 🔄 Reassign anytime.
- 📁 Python is case-sensitive.

EXTRA TIP

Use meaningful variable names so your code is easy to read and understand.

Example:

```
x = 5 # not good
total_students = 5 # better
```





LOCAL vs GLOBAL VARIABLES IN PYTHON

Variables define where data is stored.

The **scope** of a variable determines where it can be accessed in a program.



LOCAL VARIABLE

- Defined inside a function.
- Can be accessed only within that function.
- Created when the function is called.
- Destroyed when the function ends.



GLOBAL VARIABLE

- Defined outside any function.
- Can be accessed from anywhere in the program (inside or outside functions).
- Exists for the entire duration of the program.

1. EXAMPLE: LOCAL VARIABLE

```
def my_function():  
    x = 10    # Local variable  
    print("Inside function, x =", x)  
  
my_function()  
# print(x)   # ❌ Error: x is not defined
```

Explanation:

- x is created inside my_function().
- It exists only during the function call.
- Trying to access x outside the function will raise NameError.

Output:

Inside function, x = 10

If you uncomment print(x), you'll get:

NameError: name 'x' is not defined

2. EXAMPLE: GLOBAL VARIABLE

```
x = 10    # Global variable  
  
def my_function():  
    print("Inside function, x =", x)  
  
my_function()  
print("Outside function, x =", x)
```

Explanation:

- x is defined outside any function, so it is global.
- It can be accessed inside the function and outside as well.

Output:

Inside function, x = 10

Outside function, x = 10

3. MODIFYING GLOBAL VARIABLE INSIDE A FUNCTION

```
x = 10  
  
def update():  
    global x    # Tell Python we want  
                # to use the global x  
    x = 20    # Modifies global x  
    print("Inside function, x =", x)  
  
update()  
print("Outside function, x =", x)
```

Explanation:

- By default, if you assign a value to a variable inside a function, Python treats it as a local variable.
- The **global** keyword allows us to modify the global variable.

Output:

Inside function, x = 20

Outside function, x = 20

Without global keyword:

Uncommenting update() will not change the global x. It will create a new local x inside the function.

4. RULES SUMMARY

Rule	Local Variable	Global Variable
Where defined?	Inside a function	Outside any function
Scope	Only inside that function	Throughout the program
Lifetime	While the function runs	Entire program execution
Access	Cannot be accessed outside	Can be accessed anywhere
Modification	Doesn't affect global variable	Use global keyword to modify

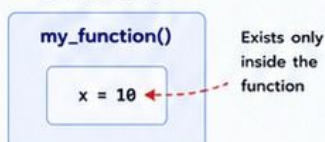


KEY TAKEAWAY

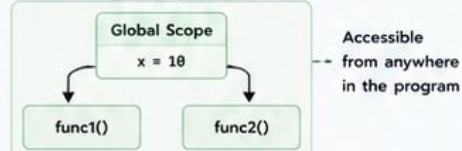
- Use local variables for temporary data inside functions.
- Use global variables for data that needs to be shared across functions.
- Avoid using too many global variables—they can make code hard to debug and maintain.
- Prefer passing data as function parameters and returning results.

5. QUICK VISUAL SUMMARY

Local Variable



Global Variable



COMMON ERROR

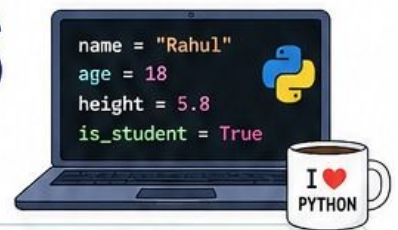
Trying to access a local variable outside its function causes NameError.

```
def f():  
    y = 5  
print(y)   # ❌ NameError
```




PYTHON DATA TYPES

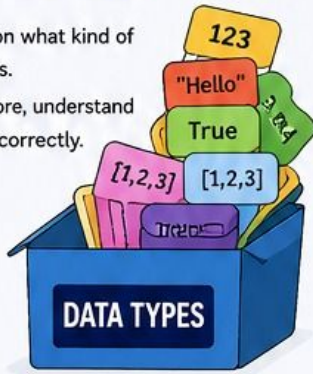
Every value in Python has a type.
Know the type, use it the right way!



1. WHAT ARE DATA TYPES?

Data types tell Python what kind of value a variable holds.

It helps Python to store, understand and work with data correctly.



MEMORY TRICK

Think of data types as different categories of items in a school bag. Every item belongs to a particular type.



2. BASIC (BUILT-IN) DATA TYPES

Type	Description	Example
int	Whole numbers	10, -5, 0
float	Decimal numbers	3.14, -0.5, 2.0
str	Text / String	"Hello", 'Python'
bool	True or False	True, False
list	Ordered collection (mutable)	[1, 2, 3]
tuple	Ordered collection (immutable)	(1, 2, 3)
dict	Key-Value pairs	{'name': 'Rahul'}
set	Unordered unique items	{1, 2, 3}

3. EXAMPLES WITH CODE

int	float	str	bool	list	tuple	dict	set
Whole number	Decimal number	Text / String	True or False	Ordered, Mutable	Ordered, Immutable	Key-Value Pairs	Unordered, Unique
<pre>age = 18 print(type(age))</pre>	<pre>height = 5.8 print(type(height))</pre>	<pre>name = "Rahul" print(type(name))</pre>	<pre>is_student = True print(type(is_student))</pre>	<pre>marks = [80, 90, 75] print(type(marks))</pre>	<pre>coords = (10, 20) print(type(coords))</pre>	<pre>student = {"name": "Rahul", "age": 18} print(type(student))</pre>	<pre>nums = {1, 2, 3} print(type(nums))</pre>
Output: <class 'int'>	Output: <class 'float'>	Output: <class 'str'>	Output: <class 'bool'>	Output: <class 'list'>	Output: <class 'tuple'>	Output: <class 'dict'>	Output: <class 'set'>

4. CHECK DATA TYPE USING type()

Use type() function to know the type of any value.

```
x = 10
print(type(x)) # <class 'int'>

y = "Hello"
print(type(y)) # <class 'str'>

z = [1, 2, 3]
print(type(z)) # <class 'list'>
```

5. MUTABLE vs IMMUTABLE

Mutable (can be changed)

list, dict, set

```
[1, 2, 3] → [1, 2, 3, 4]
{'a': 1} → {'a': 1, 'b': 2}
{1, 2} → {1, 2, 3}
```

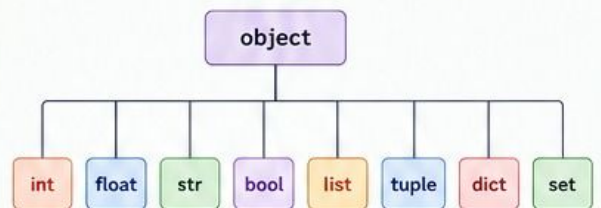
Immutable (cannot be changed)

int, float, str, tuple, bool

```
x = 10    x = 20 # new object created
s = "Hi"  s = "Hello" # new object created
t = (1, 2) # cannot change
```

6. TYPE HIERARCHY (SIMPLIFIED)

All data types in Python belong to the class 'object'.



(All other built-in types are derived from object)

7. TYPE CONVERSION (QUICK IDEA)

You can convert one data type into another using type casting.

```
x = "10"
print(int(x)) # '10' → 10

y = 5
print(float(y)) # 5 → 5.0

z = 3.7
print(str(z)) # 3.7 → '3.7'
```



8. COMMON OPERATIONS

- int / float: +, -, *, /, //, %, **
- str: + (concatenation), * (repeat)
- list: append(), remove(), len()
- dict: keys(), values(), items()
- set: add(), remove(), len()



9. COMMON MISTAKES

- ✗ Using numbers in quotes: age = "18" (This is string, not integer)
- ✗ Using single = instead of == in conditions
- ✗ Forgetting quotes for string: name = Rahul ✗
- ✗ Trying to change immutable types like int, str, tuple
- ✗ Mixing different data types without conversion



10. QUICK RECAP

- ✓ Every value in Python has a type.
- ✓ Knowing the data type helps you use the value correctly.
- ✓ Use type() to check the type.
- ✓ Some types are mutable (can change), some are immutable (cannot change).
- ✓ Choose the right data type for the right task.



11. PRACTICE TIME!

What is the data type of each of the following?

- 25
- 3.14
- "Python"
- True
- [10, 20, 30]
- (1, 2, 3)
- {'a': 1, 'b': 2}
- {1, 2, 3}



Practice daily,
become pro
in Python!



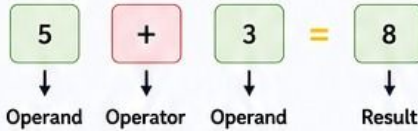
PYTHON OPERATORS

Operators are special symbols that perform operations on values and variables.



1. WHAT ARE OPERATORS?

Operators are used to perform operations on one, two or more values (operands) and return a result.



Memory Trick

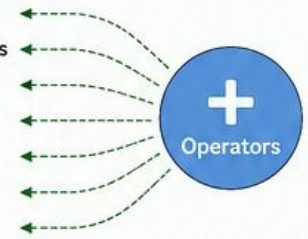


Operator = Action
Operand = On whom action is performed
Result = Output

2. TYPES OF OPERATORS IN PYTHON

Python has the following types of operators:

- ✓ Arithmetic Operators
- ✓ Comparison (Relational) Operators
- ✓ Assignment Operators
- ✓ Logical Operators
- ✓ Bitwise Operators
- ✓ Membership Operators
- ✓ Identity Operators

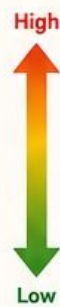


3. OPERATOR DETAILS WITH EXAMPLES

Type	Operator	Description	Example	Code	Result
1. Arithmetic (Performs mathematical operations)	+	Addition	5 + 3	print(5 + 3)	8
	-	Subtraction	5 - 3	print(5 - 3)	2
	*	Multiplication	5 * 3	print(5 * 3)	15
	/	Division (float result)	5 / 2	print(5 / 2)	2.5
	//	Floor Division (integer result)	5 // 2	print(5 // 2)	2
	%	Modulus (remainder)	5 % 2	print(5 % 2)	1
	**	Exponentiation (power)	5 ** 3	print(5 ** 3)	125
2. Comparison (Relational) (Compare two values)	==	Equal to	5 == 5	print(5 == 5)	True
	!=	Not equal to	5 != 3	print(5 != 3)	True
	>	Greater than	5 > 3	print(5 > 3)	True
	<	Less than	5 < 3	print(5 < 3)	False
	>=	Greater than or equal to	5 >= 5	print(5 >= 5)	True
	<=	Less than or equal to	5 <= 3	print(5 <= 3)	False
3. Assignment (Assign values)	=	Assign	x = 5	x = 5	x = 5
	+=	Add and assign	x += 3	x += 3	x = 8
	-=	Subtract and assign	x -= 2	x -= 2	x = 3
	*=	Multiply and assign	x *= 4	x *= 4	x = 12
	/=	Divide and assign	x /= 2	x /= 2	x = 6.0
	//=	Floor divide and assign	x //= 2	x //= 2	x = 2
	%=	Modulus and assign	x %= 3	x %= 3	x = 2
	**=	Exponent and assign	x **= 2	x **= 2	x = 4
4. Logical (Combine conditional statements)	and	True if both conditions are True	True and False	print(True and False)	False
	or	True if at least one is True	True or False	print(True or False)	True
	not	Reverse the result (True ↔ False)	not True	print(not True)	False
5. Bitwise (Works on binary representation)	&	Bitwise AND	5 & 3	print(5 & 3)	1
		Bitwise OR	5 3	print(5 3)	7
	^	Bitwise XOR	5 ^ 3	print(5 ^ 3)	6
	~	Bitwise NOT (inverts bits)	~5	print(~5)	-6
	<<	Left Shift	5 << 1	print(5 << 1)	10
6. Membership (Test if value is present in a sequence)	in	True if value exists	3 in [1,2,3]	print(3 in [1,2,3])	True
	not in	True if value does not exist	4 not in [1,2,3]	print(4 not in [1,2,3])	True
7. Identity (Compare memory location)	is	True if both are same object	x is y	print(x is y)	True / False
	is not	True if both are not same object	x is not y	print(x is not y)	True / False

4. OPERATOR PRECEDENCE (HIGH TO LOW)

1. () Parentheses
2. ** Exponentiation
3. +x -x ~x (Unary plus, minus, bitwise NOT)
4. * / // %
5. + -
6. << >>
7. &
8. ^
9. |
10. Comparison operators (==, !=, >, <, >=, <=)
11. not
12. and
13. or
14. = += -= *= /= //= %= **=



5. EXAMPLE: COMBINING OPERATORS

```
x = 10
y = 5
result = (x + y) * 2 > 20 and not (y == 5)
print(result) # False
```

Step by step:

- 1 (x + y) = 15
- 2 15 * 2 = 30
- 3 30 > 20 → True
- 4 y == 5 → True
- 5 not True → False
- 6 True and False → False

Use parentheses to control the order of operations!



6. COMMON MISTAKES

- ✗ Using = instead of == in conditions (if x = 5 ❌)
- ✗ Forgetting operator precedence
- ✗ Dividing two integers and expecting integer (use // for floor division)
- ✗ Confusing is with ==
- ✗ Using not with non-boolean values incorrectly



7. QUICK RECAP

- ✓ Operators help us perform actions on values.
- ✓ Python has 7 types of operators.
- ✓ Know the purpose and usage of each operator.
- ✓ Practice regularly to use them effectively!



8. PRACTICE TIME!

Predict the output of the following:

1. print(10 + 3 * 2)
2. print(10 // 3 + 2 * 3)
3. print(5 > 3 and 2 < 1 or 3 == 3)
4. print(2 ** 3 + 4 % 3)
5. print(5 is 5)
6. print(4 in [1,2,3,4])

Answers

1. 16
2. 6
3. True
4. 11
5. True
6. True





Python **is** vs **==**



Both are comparison operators, but they check different things.

1. == (Equality Operator)

Checks if the values of two objects are equal.

```
a = 10
b = 10
print(a == b) # True
```

- Compares the contents/values.
- Calls the `__eq__()` method internally.

VS

2. is (Identity Operator)

Checks if two variables point to the same object in memory.

```
a = 10
b = 10
print(a is b) # Maybe True (for small integers)
```

- Compares the identity (memory location).
- Calls the `__is__` logic (checks `id()`).

3. Key Difference

Aspect	== (Equality)	is (Identity)
What it checks	Value equality	Object identity (same memory location)
How it works	Compares contents using <code>__eq__()</code>	Compares memory addresses using <code>id()</code>
When to use	When you want to compare values	When you want to check if two variables refer to the same object
Example	<code>a == b</code> # Are the values same?	<code>a is b</code> # Are both variables same object?
Can be overridden?	Yes (<code>__eq__</code> can be customized)	No (identity is always about memory)

4. Examples

Example 1: Integers

```
a = 10
b = 10
print(a == b) # True
print(a is b) # True (due to interning)
```

Python reuses (interns) small integers (-5 to 256).

Example 2: Large Integers

```
a = 1000
b = 1000
print(a == b) # True
print(a is b) # False
```

Large integers are not interned, so different objects.

Example 3: Strings

```
a = "hello"
b = "hello"
print(a == b) # True
print(a is b) # True (may be True)
```

String interning may happen based on implementation.

Example 4: Lists

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True
print(a is b) # False
```

Lists with same contents are different objects.

5. id() Function

Returns the memory address (identity) of an object.

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(id(a)) # e.g., 140735208123456
print(id(b)) # same as a
print(id(c)) # different from a
```

- `a` and `b` reference the same object.
- `c` is a new object with same value.

6. When to Use

Use == when:

- ✓ You want to compare values.
- ✓ You don't care where the objects are stored.
- ✓ Default choice for most comparisons.

Use is when:

- ✓ You want to check if two variables refer to the exact same object.
- ✓ Useful with `None`.
- ✓ Useful in singletons or caching scenarios.

7. Special Case: None

```
a = None
b = None
print(a == b) # True
print(a is b) # True
```

i Always use `'is'` or `'is not'` to compare with `None`.
Don't use `==` or `!=` with `None`.

```
if x is None: # Good
if x == None: # Not recommended
```



Remember: `==` compares values, `is` compares identity (memory location).

For most cases, use `==`. Use `is` only when you really need to check identity.



PYTHON INPUT AND OUTPUT

Take input from the user. Show output to the user.

Interact with your program!



1. WHAT IS INPUT AND OUTPUT?

INPUT

Input means getting data from the user.

We use the `input()` function.



OUTPUT

Output means showing data to the user.

We use the `print()` function.



3. THE input() FUNCTION (INPUT)

The `input()` function is used to take input from the user. It always returns data as a string.

Example Code	What happens
<code>name = input("Enter your name: ")</code>	Asks user to enter name and stores it in 'name'
<code>age = input("Enter your age: ")</code>	Asks user to enter age and stores it in 'age'
<code>city = input("Enter your city: ")</code>	Asks user to enter city and stores it in 'city'



`input()` returns data as STRING.
If you enter a number, it will still be stored as text.

You type: 25 → Python stores: "25" (string)

5. COMPLETE EXAMPLES

Example 1: Greet the User

```
name = input("Enter your name: ")
print("Hello,", name + "!")
print("Welcome to Python!")
```

Sample Output

```
Enter your name: Rahul
Hello, Rahul!
Welcome to Python!
```



Example 2: Add Two Numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
total = a + b
print("Total is:", total)
```

Sample Output

```
Enter first number: 12
Enter second number: 18
Total is: 30
```



7. USEFUL TIPS

- ✓ Use clear prompts in `input()` so user knows what to enter.
- ✓ Always convert input to the correct data type.
- ✓ Use `print()` to show meaningful messages.
- ✓ You can print multiple values separated by commas.
- ✓ Use `end=""` in `print()` to print on the same line.
- ✓ Test your program with different inputs.



8. INPUT AND OUTPUT FLOW

How input and output work in a program.



1. Program asks for input (using `input()`)
2. User enters data
3. Python processes the data
4. Program displays output (using `print()`)

10. QUICK RECAP



input()
Takes input from the user.

Always returns STRING.



print()
Displays output on the screen.

Adds new line by default.



Type Conversion
Use `int()`, `float()` to convert input for calculations.

Always convert when needed.



Good Practice
Use clear prompts, handle input carefully, and test your code.

Practice makes perfect!

Remember:

Input takes data from the user.
Output gives results to the user.

Together they make your program interactive! 🎉



2. THE print() FUNCTION (OUTPUT)

The `print()` function is used to display text, numbers or values on the screen.

Example	Explanation
<code>print("Hello, World!")</code>	Prints a text (string)
<code>print(10)</code>	Prints a number
<code>name = "Rahul"</code> <code>print(name)</code>	Prints the value of a variable
<code>print(5 + 3)</code>	Prints the result of an expression
<code>print("Age:", 18)</code>	Prints multiple values
<code>print("I love", "Python")</code>	Prints multiple strings



By default, `print()` adds a new line after the output.
Use `end=""` to change it.

```
print("Hello", end=" ")
print("World")
```

→ Hello World

4. TAKING NUMERIC INPUT (TYPE CONVERSION)

Since `input()` returns string, we must convert it to `int`, `float` etc. to do calculations.

Conversion	Example Code	Explanation
Integer	<code>age = int(input("Enter age: "))</code>	Converts input to integer (whole number)
Float	<code>height = float(input("Enter height: "))</code>	Converts input to float (decimal number)
Example	<code>a = int(input("Enter a: "))</code> <code>b = int(input("Enter b: "))</code> <code>sum = a + b</code> <code>print("Sum is:", sum)</code>	Now we can do mathematical operations



Always convert input to the required data type before doing calculations.

6. COMMON MISTAKES

- ✗ Forgetting type conversion
`n = input("Enter number: ")`
`print(n + 5)` # Error! (string + int)
- ✗ Trying to convert invalid input
`age = int(input("Enter age: "))`
If user enters 'abc' -> ValueError
- ✗ Not using prompt message
`name = input()`
User may not know what to enter
- ✗ Expecting number but getting string
`a = input("Enter a: ")` # '10' (string)
`b = input("Enter b: ")` # '20' (string)
`print(a + b)` # Output: 1020 (not 30)



9. MINI PRACTICE

Try these programs:

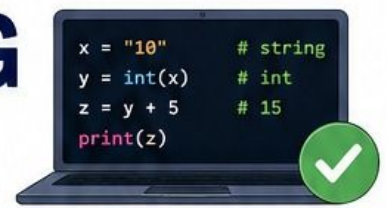
1. Take your name as input and print a welcome message.
2. Take two numbers as input and print their sum.
3. Take radius of a circle as input and print area.
4. Take temperature in Celsius and convert it to Fahrenheit.





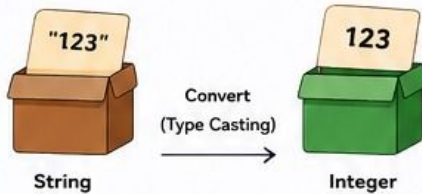
PYTHON TYPE CASTING

Convert one data type into another.
The right type for the right operation!



1. WHAT IS TYPE CASTING?

Type casting (or conversion) is the process of converting one data type into another. It helps us perform operations that require a specific data type.



Example: You cannot add a string and an integer directly. You must convert the string to integer first.

2. BUILT-IN TYPE CASTING FUNCTIONS

Function	Converts To	Example	Result
<code>int()</code>	Integer	<code>int("10")</code>	10
<code>float()</code>	Float	<code>float("10.5")</code>	10.5
<code>str()</code>	String	<code>str(123)</code>	"123"
<code>bool()</code>	Boolean	<code>bool(0)</code>	False
<code>list()</code>	List	<code>list("abc")</code>	['a', 'b', 'c']
<code>tuple()</code>	Tuple	<code>tuple([1,2])</code>	(1, 2)
<code>set()</code>	Set	<code>set([1,2,1])</code>	{1, 2}
<code>dict()</code>	Dictionary	<code>dict(name="Ravi")</code>	{'name': 'Ravi'}

3. BASIC CONVERSIONS WITH EXAMPLES

String to Integer

```
x = "25"
y = int(x)
print(y)
```

Output: 25

String to Float

```
x = "3.14"
y = float(x)
print(y)
```

Output: 3.14

Integer to String

```
x = 100
y = str(x)
print(y)
```

Output: "100"

Float to Integer

```
x = 7.89
y = int(x)
print(y)
```

Output: 7

4. BOOLEAN CONVERSIONS (bool())

In Python, the following values are considered **False**:

0, 0.0, "", [], (), {}, set(), , None

All other values are considered **True**.

Value	bool(value)	Result	Examples
0	False	✗	<code>print(bool(0))</code> <code>print(bool(1))</code> <code>print(bool(""))</code> <code>print(bool("Hi"))</code>
1	True	✓	<code>print(bool([]))</code> <code>print(bool([1]))</code> <code>print(bool(None))</code>
"" (empty string)	False	✗	Output: False True False True False True False
"Hello"	True	✓	
[] (empty list)	False	✗	
[1, 2]	True	✓	
None	False	✗	

5. LIST, TUPLE, SET, DICT CONVERSIONS

To List

```
x = "abc"
y = list(x)
print(y)
```

Output: ['a', 'b', 'c']

To Tuple

```
x = [1, 2, 3]
y = tuple(x)
print(y)
```

Output: (1, 2, 3)

To Set (removes duplicates)

```
x = [1, 2, 2, 3]
y = set(x)
print(y)
```

Output: {1, 2, 3}

To Dictionary (from key=value pairs)

```
x = dict(name="Ravi", age=20)
print(x)
```

Output: {'name': 'Ravi', 'age': 20}

6. IMPLICIT vs EXPLICIT CONVERSION

Implicit Conversion (Automatic)

Python automatically converts smaller type to larger type.

```
a = 10 # int
b = 2.5 # float
c = a + b # int is converted to float
print(c) # 12.5
```

Explicit Conversion (Manual)

We manually convert using built-in functions.

```
a = "20"
b = 5
c = int(a) + b # convert string to int
print(c) # 25
```

7. COMMON USE CASES

- ✓ Converting user input (string) to numbers for calculations.
- ✓ Converting numbers to strings for formatting or display.
- ✓ Converting data before storing or processing.
- ✓ Ensuring correct data type in functions and operations.



8. INVALID CONVERSIONS (WILL RAISE ERROR)

Not all conversions are possible. Some will raise an error.

- ✗ `int("abc")` # ValueError
- ✗ `float("hello")` # ValueError
- ✗ `list(123)` # TypeError
- ✗ `dict([1,2,3])` # ValueError



9. HOW TO HANDLE CONVERSION ERRORS

Use `try...except` to handle errors gracefully.

```
x = input("Enter a number: ")
try:
    y = int(x)
    print("You entered:", y)
except ValueError:
    print("Invalid input! Please enter a number.")
```



10. PRACTICE EXAMPLES

- Convert the following string to integer and add 5.

```
x = "15"
y = int(x) + 5
print(y)
```

Output: 20
- Convert the following float to integer.

```
x = 9.87
y = int(x)
print(y)
```

Output: 9
- Convert the number to string and print its type.

```
x = 123
y = str(x)
print(type(y))
```

Output: <class 'str'>
- Convert the string "python" to list.

```
x = "python"
y = list(x)
print(y)
```

Output: ['p', 'y', 't', 'h', 'o', 'n']

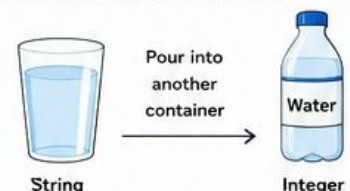
11. QUICK RECAP

- ✓ Type casting converts one type to another.
- ✓ Use `int()`, `float()`, `str()`, `bool()` and other functions.
- ✓ Not all conversions are valid.
- ✓ Be careful with user input.
- ✓ Convert before doing calculations.
- ✓ When in doubt, check the data type using `type()`.



12. MEMORY TRICK

Think of type casting like changing the container for the same content.



Content (value) same,
container (type) changed!





PYTHON TYPECASTING

Convert one data type into another.

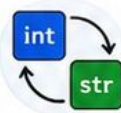
```
int(3.14)      # 3
float('2.5')   # 2.5
str(100)        # '100'
list('Hi')      # ['H', 'i']
```

1. WHAT IS TYPECASTING?

Typecasting (Type Conversion) is the process of converting a value from one data type to another data type.

Why?

- To perform operations between incompatible data types.
- To take input as one type and work with another.
- To ensure the correct type of data for functions.



2. TYPES OF TYPECASTING

Implicit Typecasting (Automatic)

Python automatically converts one data type to another (wider) type to avoid data loss.

Example:

```
a = 10      # int
b = 2.5     # float
c = a + b   # 12.5 (int → float)
```



Explicit Typecasting (Manual)

The programmer converts the data type manually using built-in functions.

Example:

```
a = '100'
b = int(a) # 100 (str → int)
```



3. BUILT-IN TYPECASTING FUNCTIONS

Function	Converts To	Example	Result
int()	Integer	int('25')	25
float()	Float	float('3.14')	3.14
str()	String	str(123)	'123'
bool()	Boolean	bool(0)	False
list()	List	list('ab')	['a', 'b']
tuple()	Tuple	tuple([1,2])	(1, 2)
set()	Set	set('abca')	{ 'a', 'b', 'c' }
dict()	Dictionary	dict(a=1, b=2)	{ 'a': 1, 'b': 2 }

4. EXAMPLES OF EXPLICIT TYPECASTING

To Integer (int())

```
print(int(10.9)) # 10
print(int('-20')) # -20
print(int('100')) # 100
```

To Float (float())

```
print(float(10)) # 10.0
print(float('3.14')) # 3.14
print(float('-2.5')) # -2.5
```

To String (str())

```
print(str(100)) # '100'
print(str(3.14)) # '3.14'
print(str(True)) # 'True'
```

To Boolean (bool())

```
print(bool(1)) # True
print(bool(0)) # False
print(bool('Hello')) # True
print(bool('')) # False
```

To List (list())

```
print(list('Hi')) # ['H', 'i']
print(list((1, 2))) # [1, 2]
print(list({3, 4})) # [3, 4] *
```

*Order may vary in set

To Tuple (tuple())

```
print(tuple([1, 2])) # (1, 2)
print(tuple('ab')) # ('a', 'b')
print(tuple({1, 2})) # (1, 2) *
```

*Order may vary in set

To Set (set())

```
print(set('banana')) # {'a', 'n', 'b'}
print(set([1, 2, 2, 3])) # {1, 2, 3}
print(set({4, 5, 5})) # {4, 5}
```

To Dict (dict())

```
print(dict(a=1, b=2)) # {'a': 1, 'b': 2}
print(dict([('x', 10), ('y', 20)]))
# {'x': 10, 'y': 20}
```

5. TYPECASTING WITH USER INPUT

```
name = input('Enter your name: ')
age = int(input('Enter your age: '))
height = float(input('Enter height in meters: '))
```

```
print(type(name)) # <class 'str'>
print(type(age)) # <class 'int'>
print(type(height)) # <class 'float'>
```



6. IMPORTANT NOTES

- Typecasting may result in data loss.
Example: int(9.99) → 9 (decimal part lost)
- Not all conversions are possible.
Example: int('hello') → ValueError
- Be careful while converting large numbers between int and float (precision issues).



7. COMMON TYPECASTING ERRORS

- ValueError: When the value is not appropriate for the target type.

```
int('abc') # ValueError
```

- TypeError: When the type is unsupported for conversion.

```
int([1, 2, 3]) # TypeError
```

8. TYPECASTING TABLE (QUICK REFERENCE)

From \ To	int	float	str	bool	list	tuple	set	dict
int	✓	float()	str()	bool()	list()	tuple()	set()	dict() ✗
float	int()	✓	str()	bool()	list() ✗	tuple() ✗	set() ✗	dict() ✗
str	int() *	float() *	✓	bool()	list()	tuple()	set()	dict() ✗
bool	int()	float()	str()	✓	list()	tuple()	set()	dict() ✗
list	int() ✗	float() ✗	str()	bool()	✓	tuple()	set()	dict() ✗
tuple	int() ✗	float() ✗	str()	bool()	list()	✓	set()	dict() ✗
set	int() ✗	float() ✗	str()	bool()	list()	tuple()	✓	dict() ✗
dict	int() ✗	float() ✗	str()	bool()	list()	tuple()	set()	✓

✓ = Possible

✗ = Not directly possible

* = Possible if the string represents a valid number

9. BEST PRACTICES

- ✓ Always know the data type you are working with (use type()).
- ✓ Convert explicitly when needed.
- ✓ Validate user input before typecasting.
- ✓ Handle exceptions using try-except.
- ✓ Avoid unnecessary type conversions.

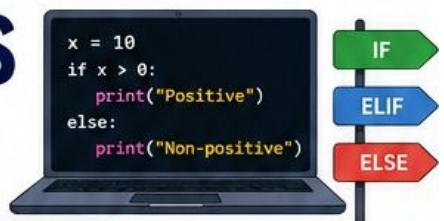


Understand **types**. Convert **wisely**. Code **effectively**! 😊



PYTHON CONDITIONALS (IF, ELSE, ELIF)

Make decisions in your program. Choose the right path!



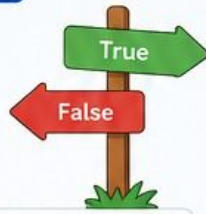
1. WHAT ARE CONDITIONALS?

Conditionals allow your program to make decisions.

Python checks a condition.

If it is True, it runs the block of code.

If it is False, it runs another block.



Memory Trick

Think of conditionals as traffic signals for your code.
Green light (True) → Go this way.
Red light (False) → Go another way.

2. COMPARISON OPERATORS (Used in Conditions)

Operator	Meaning	Example	Result (if x = 5, y = 10)
==	Equal to	x == y	False
!=	Not equal to	x != y	True
>	Greater than	x > y	False
<	Less than	x < y	True
>=	Greater than or equal to	x >= y	False
<=	Less than or equal to	x <= y	True

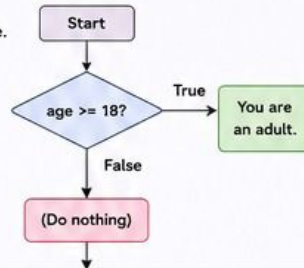
Example Values

x = 5
y = 10

3. IF STATEMENT

Executes a block of code if the condition is True.

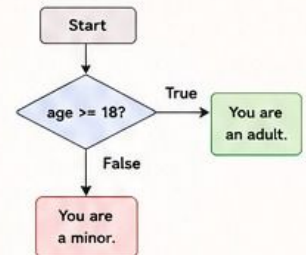
```
age = 18
if age >= 18:
    print("You are an adult.")
```



4. IF...ELSE STATEMENT

Executes one block if the condition is True, another block if it is False.

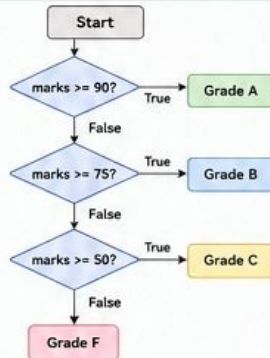
```
age = 16
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```



5. IF...ELIF...ELSE LADDER

Checks multiple conditions one by one.

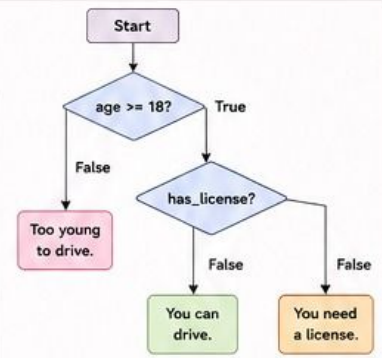
```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Grade F")
```



6. NESTED IF STATEMENT

An if statement inside another if statement.

```
age = 20
has_license = True
if age >= 18:
    if has_license:
        print("You can drive.")
    else:
        print("You need a license.")
else:
    print("You are too young to drive.")
```



7. LOGICAL OPERATORS (Combine Conditions)

Operator	Meaning	Example	Result (x = 5, y = 10)
and	True if both conditions are True	x < 10 and y > 5	True
or	True if at least one condition is True	x > 10 or y > 5	True
not	Reverses the condition	not (x > 10)	True

Example with and, or, not

```
x = 7
if x > 5 and x < 10:
    print("x is between 5 and 10")
if x < 5 or x == 7:
    print("x is less than 5 or equal to 7")
if not (x == 7):
    print("x is not 7")
else:
    print("x is 7")
```

```
1) x > 5 and x < 10 -> True
2) x < 5 or x == 7 -> True
3) not (x == 7) -> False

Output:
x is between 5 and 10
x is less than 5 or equal to 7
x is 7
```

8. TRUTH VALUES

All conditions evaluate to either True or False.

True	False
✓ Non-zero numbers (1, -5, 100)	✗ Zero (0)
✓ Non-empty strings "hello"	✗ Empty string ""
✓ Non-empty lists [1, 2]	✗ Empty list []
✓ The value True	✗ None
	✗ The value False

9. COMMON MISTAKES

- ✗ Using = instead of == in conditions
if x = 5: ✗ (This is assignment)
if x == 5: ✓ (This is comparison)
- ✗ Indentation errors
(Always indent the block under if, elif, else)
- ✗ Missing colon : after if, elif, else
- ✗ Using logical operators incorrectly



10. PRACTICE EXAMPLES

1 Check if a number is positive.

```
n = 7
if n > 0:
    _____
else:
    _____
```

2 Find the largest of two numbers.

```
a = 10
b = 15
if a > b:
    _____
else:
    _____
```

3 Check if a number is even or odd.

```
num = 8
if _____:
    print("Even")
else:
    print("Odd")
```

4 Grade a student based on marks.

```
marks = 72
if marks >= 90:
    _____
elif marks >= 75:
    _____
else:
    _____
```

11. QUICK RECAP

- ✓ Use if to run code when condition is True.
- ✓ Use if...else to choose between two options.
- ✓ Use if...elif...else for multiple options.
- ✓ Use logical operators to combine conditions.
- ✓ Always check conditions → True or False.



12. MINI CHALLENGE

Write a program that:

- Takes a number as input
- If the number is positive → print "Positive"
- If the number is negative → print "Negative"
- If the number is zero → print "Zero"
- (Use if...elif...else)



13. MEMORY TRICK

IF the condition is True, do this.

ELIF there is another option, check this.

ELSE do something else.

IF → ELIF → ELSE





Python if-else Shorthand

Write simple if-else logic in a single line.



Shorthand if-else is also called the "ternary operator" because it takes three parts: a condition, a value if True, and a value if False.

1. BASIC SYNTAX

```
value_if_true if condition else value_if_false
```

- If **condition** is **True** → returns **value_if_true**
- If **condition** is **False** → returns **value_if_false**

2. EXAMPLES

What You Want	Normal if-else	Shorthand	Result (if condition is True)	Result (if condition is False)
Check if a number is even or odd	<pre>if n % 2 == 0: result = "Even" else: result = "Odd"</pre>	<pre>result = "Even" if n % 2 == 0 else "Odd"</pre>	"Even"	"Odd"
Find the larger of two numbers	<pre>if a > b: larger = a else: larger = b</pre>	<pre>larger = a if a > b else b</pre>	a	b
Check if a person is adult or minor	<pre>if age >= 18: status = "Adult" else: status = "Minor"</pre>	<pre>status = "Adult" if age >= 18 else "Minor"</pre>	"Adult"	"Minor"
Return a default value if None	<pre>if name is not None: display = name else: display = "Guest"</pre>	<pre>display = name if name is not None else "Guest"</pre>	name	"Guest"

4. WHEN TO USE

Use Shorthand When:

- ✓ The logic is simple and short.
- ✓ It makes the code more concise and easy to read.
- ✓ You are assigning a value based on a condition.

Avoid Shorthand When:

- ✗ The logic is complex or long.
- ✗ You need multiple statements inside if or else.
- ✗ Readability would suffer.

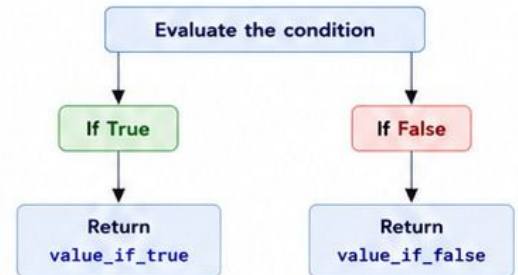
6. QUICK REFERENCE

Pattern	Example	Equivalent if-else
<code>value_if_true if condition else value_if_false</code>	<pre>x = 10 if n > 0 else -10</pre>	<pre>if n > 0: x = 10 else: x = -10</pre>
Chained (multiple conditions)	<pre>grade = "A" if s >= 90 else "B" if s >= 75 else "C" if s >= 50 else "D"</pre>	<pre>if s >= 90: grade = "A" elif s >= 75: grade = "B" elif s >= 50: grade = "C" else: grade = "D"</pre>
With None (default value)	<pre>name = user_input if user_input else "Guest"</pre>	<pre>if user_input: name = user_input else: name = "Guest"</pre>



```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status) # Adult
```

HOW IT WORKS



3. NESTED (CHAINED) SHORTHAND

You can chain multiple conditions, but use it only for simple cases (for readability).

```
score = 85
grade = "A" if score >= 90 else "B" if
score >= 75 else "C" if
score >= 50 else "D"
```

Equivalent to:

```
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "D"
```



Avoid overusing chaining.
If logic is complex, regular if-else is easier to read and maintain.

5. SHORTHAND WITH FUNCTIONS (COMMON PATTERN)

Often used to return one of two values.

```
def get_status(age):
    return "Adult" if age >= 18 else "Minor"

print(get_status(21)) # Adult
print(get_status(16)) # Minor
```

KEY TAKEAWAY

- ★ Shorthand if-else makes your code shorter and cleaner.
- ✓ Use it for simple value selection.
- ✓ Prefer readability over cleverness.
- ✓ When in doubt, use the normal if-else.





PYTHON MATCH-CASE

Pattern Matching Made Easy (Introduced in Python 3.10)



The match statement tests a value against multiple patterns.
It is more powerful and readable than multiple if-elif-else statements.

```
day = "Monday"
match day:
    case "Monday":
        print("Start of the week!")
    case "Friday":
        print("Almost weekend!")
    case _:
        print("Just another day.")
```

1. SYNTAX

```
match expression:
    case pattern_1:
        # code block
    case pattern_2:
        # code block
    ...
    case pattern_n:
        # code block
    case _: # default case (optional)
        # code block
```

★ KEY POINTS

- The expression is evaluated once.
- Patterns are checked from top to bottom.
- The first matching pattern's block runs.
- case _ acts as the default (like else).
- Indentation is significant.

2. BASIC EXAMPLE

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _:
        print("Invalid day")
```

Output
Wednesday

3. PATTERN TYPES WITH EXAMPLES

Pattern Type	Description	Example	Output
Literal Pattern	Matches exact values.	<pre>x = "apple" match x: case "apple": print("Apple")</pre>	Apple
Multiple Literals	Matches any one of the given literals.	<pre>x = "B" match x: case "A" "B" "C": print("Letter")</pre>	Letter
Sequence Pattern	Matches list or tuple structure.	<pre>point = (2, 3) match point: case (0, 0): print("Origin") case (x, y): print(f"Point({x}, {y})")</pre>	Point(2, 3)
Mapping Pattern	Matches dictionary structure.	<pre>person = {"name": "Alice", "age": 25} match person: case {"name": n, "age": a}: print(f"{n} is {a} years old")</pre>	Alice is 25 years old
Class Pattern	Matches object attributes.	<pre>class Point: def __init__(self, x, y): self.x = x; self.y = y p = Point(10, 20) match p: case Point(0, 0): print("Origin") case Point(x, y): print(f"Point({x}, {y})")</pre>	Point(10, 20)
Guard Pattern (If Condition)	Additional condition using if.	<pre>n = 7 match n: case x if x % 2 == 0: print("Even") case x if x % 2 != 0: print("Odd")</pre>	Odd
Wildcard Pattern	Matches anything (usually last case).	<pre>x = 100 match x: case 1: print("One") case _: print("Something else")</pre>	Something else

4. DEFAULT CASE

```
color = "yellow"
match color:
    case "red":
        print("Stop")
    case "yellow":
        print("Slow Down")
    case "green":
        print("Go")
    case _:
        print("Unknown color")
```

Output
Slow Down

5. NESTED PATTERNS

```
data = {"type": "user", "name": "Bob", "age": 22}
match data:
    case {"type": "user", "name": name, "age": age}:
        print(f"User {name} is {age} years old")
    case {"type": "admin", "name": name}:
        print(f"Admin {name}")
    case _:
        print("Unknown data")
```

Output
User Bob is 22 years old

6. MATCH-CASE vs IF-ELIF-ELSE

Aspect	match-case	if-elif-else
Readability	More clean and structured	Can be long and harder to read
Best for	Many options / complex patterns	Simple conditions
Expression Evaluated	Once	Multiple times
Pattern Matching	Yes (literals, sequences, dicts, objects)	No
Introduced in	Python 3.10	All versions

7. WHEN TO USE?

- ✓ When you have many possible conditions.
- ✓ When matching complex data structures (lists, dicts, objects).
- ✓ To make code cleaner, shorter and easier to read.



8. LIST / TUPLE PATTERNS (EXAMPLES)

```
numbers = [1, 2, 3]
match numbers:
    case []:
        print("Empty list")
    case [x]:
        print(f"Single item: {x}")
    case [x, y]:
        print(f"Two items: {x}, {y}")
    case [a, *rest]:
        print(f"First: {a}, Rest: {rest}")
    case [a, b, b, *rest, last]:
        print(f"a={a}, b={b}, last={last}, rest={rest}")
    case _:
        print("Other pattern")
```

Output
First: 1, Rest: [2, 3]

Sequence Patterns

[] → empty list
[x] → one item
[x, y] → fixed items
[a, *rest] → variable length tail
[a, b, *rest, last] → items with rest in the middle

! *rest captures zero or more items.

9. IMPORTANT NOTES

- ✓ case is not fall-through (like in some other languages). It stops at the first match.
- ✓ Patterns are checked in order.
- ✓ Variables in patterns are local to that case block.
- ✓ Use _ (underscore) for default case.
- ✓ Out of range index in sequence pattern raises IndexError.



10. COMMON MISTAKES

- ✗ Forgetting that case _ is needed for default behavior.
- ✗ Placing a general pattern before a specific one.
- ✗ Using sequence pattern on non-iterables.
- ✗ Expecting fall-through behavior.
- ✗ Not handling missing keys in mapping patterns (use guards or .get()).



11. PRACTICE EXAMPLES

- 1 Create a match-case to print the name of a month from number (1-12).
- 2 Match a list and print its length using pattern matching.
- 3 Classify a number as positive, negative or zero.
- 4 Match a dictionary and greet the user by name.
- 5 Reverse a list using match-case patterns.



12. QUICK RECAP

- ✓ match expression: tests a value against multiple patterns.
- ✓ Supports literals, sequences, mappings, class patterns, guards and more.
- ✓ First matching case block executes.
- ✓ Use _ for default case.
- ✓ Makes your code powerful, Pythonic and easy to maintain!



Use match-case to write **cleaner**, more **expressive** and **maintainable** Python code! 🚀



PYTHON LOOPS

Repeat actions, save time, write efficient code!



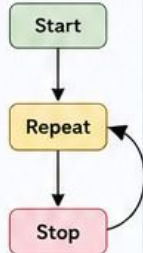
1. WHAT IS A LOOP?

A loop is used to repeat a block of code multiple times.

It helps to automate repetitive tasks and saves time and effort.

Memory Trick

Loop = Do something again and again, without rewriting!



2. TYPES OF LOOPS IN PYTHON

for Loop

Used to iterate over a sequence (list, tuple, string, range, etc.).

Best when you know how many times to repeat.



while Loop

Repeats as long as a condition is True.

Best when you don't know how many times to repeat.



3. FOR LOOP

Used to iterate over a sequence (list, tuple, string, range, etc.).

```
for variable in sequence:  
    # code to execute
```

Example 1: Print numbers 0 to 4

```
for i in range(5):  
    print(i)
```

Output
0
1
2
3
4

Example 2: Iterate over a list

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output
apple
banana
cherry

4. range() FUNCTION

range(start, stop, step)
Generates a sequence of numbers.

Example	Meaning	Output
range(5)	0 to 4	0 1 2 3 4
range(1, 6)	1 to 5	1 2 3 4 5
range(2, 10, 2)	2 to 8 (step 2)	2 4 6 8
range(10, 0, -2)	10 to 2 (step -2)	10 8 6 4 2

```
for i in range(1, 6):  
    print(i, end=" ")
```

Output
1 2 3 4 5

5. WHILE LOOP

Repeats as long as the condition is True.

```
while condition:  
    # code to execute  
    # update the condition
```

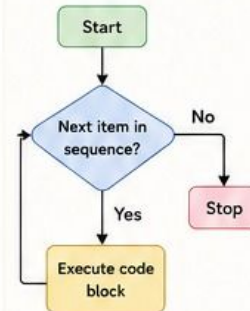
Example: Print numbers 1 to 5

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

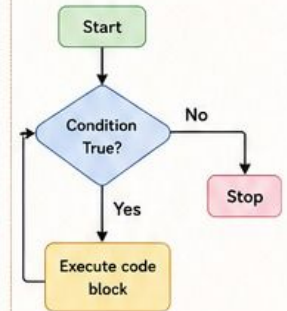
Output
1
2
3
4
5

6. LOOP FLOW DIAGRAMS

for Loop



while Loop



7. NESTED LOOPS

A loop inside another loop.

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(f"i={i}, j={j}")
```

Output
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3

Pattern Example

```
for i in range(1, 6):  
    print(" " * i)
```

Output
*
**

8. LOOP CONTROL STATEMENTS

Statement	Use	Example	Output
break	Terminates the loop completely.	<pre>for i in range(5): if i == 3: break print(i)</pre>	0 1 2
continue	Skips the current iteration and continues with next.	<pre>for i in range(5): if i == 2: continue print(i)</pre>	0 1 3 4
pass	Does nothing. Used as a placeholder.	<pre>for i in range(3): if i == 1: pass print(i)</pre>	0 1 2

9. INFINITE LOOP

If the condition never becomes False, the loop runs forever!

```
while True:  
    print("This runs forever!")  
    # Use break to stop
```



Be careful! Infinite loops can crash your program.

10. COMMON USE CASES

- ✓ Process each item in a list
- ✓ Repeat until user gives correct input
- ✓ Keep asking for data until valid
- ✓ Menu-driven programs
- ✓ Game loops
- ✓ Reading files line by line



11. COMMON MISTAKES

- ✗ Forgetting to update the condition in while loop (causes infinite loop)
- ✗ Using = instead of == in conditions
- ✗ Incorrect indentation
- ✗ Modifying list while looping over it
- ✗ Not using break in infinite loops



12. QUICK RECAP

- ✓ Use for loop when you know the number of iterations.
- ✓ Use while loop when you don't know the number of iterations.
- ✓ Use range() with for loop to generate numbers.
- ✓ Use break, continue, pass to control loops.
- ✓ Indentation is very important!
- ✓ Loops help make your code smart and efficient!



13. PRACTICE TIME!

Try these:

- Print numbers from 1 to 10 using for loop.
- Print even numbers from 2 to 20 using for loop.
- Print table of 5 (5 x 1 to 5 x 10).
- Take numbers as input until user enters 0. Print sum.
- Print the following pattern using nested loop.
*
* *
* * *
* * * *
* * * * *
- Count how many times the digit 7 appears in a given number.

Practice daily, become a Python Pro!





PYTHON enumerate()

In Python, `enumerate()` is a built-in function that lets you loop through a list (or any iterable) and keeps track of both the index (the count) and the item at the same time. Instead of just getting the items, it wraps them into a neat package of (index, item) pairs.

```
tasks = ["Write code", "Review PRs", "Fix bugs"]

for index, task in enumerate(tasks):
    print(f"Task {index}: {task}")

# Output:
# Task 0: Write code
# Task 1: Review PRs
# Task 2: Fix bugs
```



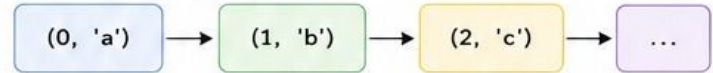
1. THE SYNTAX

```
enumerate(iterable, start=0)
```

- **iterable**: The collection of items you want to loop through (like a list, tuple, or string).
- **start** (optional): The number you want the counter to start from. By default, it is 0.

It returns an iterator that yields (index, item) pairs.

Example: ['a', 'b', 'c']



Each time you loop, you get the next pair.

2. HOW IT WORKS IN PRACTICE

Imagine you have a list of tasks for the day:

```
1 tasks = ["Write code", "Review PRs", "Fix bugs"]
2
3 for index, task in enumerate(tasks):
4     print(f"Task {index}: {task}")
```

Output:

```
Task 0: Write code
Task 1: Review PRs
Task 2: Fix bugs
```



By default, the index starts at 0.

3. STARTING FROM 1 (OR ANY OTHER NUMBER)

Human beings don't usually start counting at 0.

If you are building a menu, a quiz game, or a top-10 list, you can tell `enumerate()` to start counting from 1:

```
1 tasks = ["Write code", "Review PRs", "Fix bugs"]
2
3 for number, task in enumerate(tasks, start=1):
4     print(f"{number}. {task}")
```

Output:

```
1. Write code
2. Review PRs
3. Fix bugs
```



You can start from any number you like:
`start=5` → 5, 6, 7, ...

4. WHAT IS ACTUALLY HAPPENING UNDER THE HOOD?

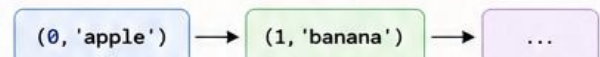
If you run `enumerate()` by itself outside of a loop, it returns an "enumerate object" (which is an iterator).

If you convert it directly into a list, you can see exactly what it's doing.

It creates a list of tuples:

```
1 fruits = ["apple", "banana"]
2 print(list(enumerate(fruits)))
3 # Output: [(0, 'apple'), (1, 'banana')]
```

```
fruits = ["apple", "banana"]
          |
          v enumerate(fruits)
```



When you use it in a for loop, Python automatically unpacks those tuples into two separate variables for you:

```
1 for index, fruit in enumerate(fruits):
2     print(index, fruit)
```



Think of it as:

The `enumerate()` function is a helper that keeps count for you while you enjoy the items!

QUICK SUMMARY

	Purpose	Loop through an iterable with an automatic counter (index).
	Syntax	<code>enumerate(iterable, start=0)</code>
	Default Start	0
	Custom Start	Use <code>start=1</code> (or any number) to change where counting begins.
	Returns	An iterator that yields (index, item) pairs.
	Best Use Cases	Menus, numbered lists, displaying rankings, processing items with their position.



Common Mistake

Using a manual counter:

```
i = 0
for item in items:
    print(i, item)
    i += 1
```



`enumerate()` is simpler, cleaner, and less error-prone!



PYTHON FUNCTIONS

Write once, use many times!

Functions make your code modular, reusable and easy to maintain.

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Python"))
# Hello, Python!
```



1. WHAT IS A FUNCTION?

A function is a block of code that performs a specific task. It runs only when it is called.

```
def function_name(parameters):
    # code block
    return value # (optional)
```

Keyword Name Parameters Return (value) Return



Memory Trick

Think of a function like a machine. You give input (parameters), it does the work, and gives output (return).



2. DEFINING AND CALLING FUNCTIONS

Defining a Function

```
def add(a, b):           # function name
    s = a + b            # parameters
    return s             # body
                        # return value
```

Calling a Function

```
result = add(5, 3)
print(result) # 8
```



A function is defined once, but can be called many times.

3. TYPES OF FUNCTIONS

(A) Built-in Functions

(Already available in Python)

print(), len(), type(), input(), max(), min()

Example:

```
print(len("Python")) # 6
```

(B) User-defined Functions

(Created by the user)

Example:

```
def square(n):
    return n * n
print(square(4)) # 16
```



4. PARAMETERS AND ARGUMENTS

- Parameters: variables in function definition.
- Arguments: values passed during function call.

Example:

```
def greet(name): # name -> parameter
    print(f"Hello, {name}!")

greet("Alice") # "Alice" -> argument
greet("Bob") # "Bob" -> argument
```

Parameter	Argument
Like a variable in the function definition.	The actual value passed to the function.



5. RETURN STATEMENT

- The return statement sends a value back.
- A function can have only one return.

Example:

```
def cube(x):
    return x ** 3

y = cube(3)
print(y) # 27
```



If there is no return, the function returns None.

6. DEFAULT PARAMETERS

If an argument is not provided, default value is used.

Example:

```
def greet(name, msg = "Welcome"):
    print(f'{msg}, {name}!')

greet("Alice") # Welcome, Alice!
greet("Bob", "Hi") # Hi, Bob!
```



Default parameters make functions more flexible.

7. VARIABLE LENGTH ARGUMENTS

(A) *args (Non-keyword Arguments)
Accepts multiple positional arguments.

Example:

```
def add_all(*args):
    return sum(args)

print(add_all(1, 2, 3, 4)) # 10
print(add_all(5, 10)) # 15
```

(B) **kwargs (Keyword Arguments)

Accepts multiple keyword arguments.

Example:

```
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(f'{key} = {value}')

show_info(name="Rahul", age=18, city="Delhi")
# name = Rahul # age = 18
# city = Delhi # city = Delhi
```

8. LAMBDA FUNCTIONS (Anonymous Functions)

Small, one-line functions without a name.

Syntax:

lambda arguments: expression

Example:

```
square = lambda x: x ** 2
print(square(5)) # 25
```

```
add = lambda a, b: a + b
print(add(3, 4)) # 7
```



Useful for short and simple functions.

9. RECURSIVE FUNCTIONS

A function that calls itself.

Example: Factorial of n

```
def fact(n):
    if n == 0:
        return 1
    else:
        return n * fact(n - 1)

print(fact(5)) # 120
```



```
fact(5) → 5 * fact(4) → 5 * 4 * fact(3)
```



```
... → * 1 = 120
```

10. SCOPE OF VARIABLES

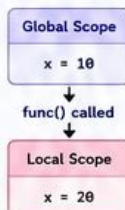
- Local variables: Inside function (only accessible there)
- Global variables: Outside function (accessible everywhere)

```
x = 10 # Global variable

def func():
    x = 20 # Local variable
    print("Inside:", x)

func()

print("Outside:", x) # 10
```



11. BUILT-IN FUNCTION EXAMPLES

Function	Use	Example	Output
len()	Length of item	len("Hi")	2
type()	Data type	type(5)	<class 'int'>
max()	Maximum value	max(4, 9, 2)	9
min()	Minimum value	min(4, 9, 2)	2
sum()	Sum of values	sum([1, 2, 3])	6
sorted()	Sorted list	sorted([3, 1, 2])	[1, 2, 3]
abs()	Absolute value	abs(-5)	5

12. PRACTICE EXERCISES

- Write a function to find the square of a number.
- Write a function to check if a number is even.
- Write a function to find the sum of first n natural numbers.
- Write a function that takes *args and returns their product.
- Write a function that takes **kwargs and prints all key-value pairs.



13. QUICK RECAP

Use def to define a function.

```
def()
{ }
```

Pass values as arguments.

→ func(5) →

Use return to get output.

return

Functions can have default values.

```
def(a = 10)
```

Use *args and **kwargs for flexibility.

```
*args
**kwargs
```

Lambda is for short one-line functions.

```
lambda x: x+1
```

Functions are reusable and save time!



Functions are the building blocks of clean and powerful Python programs!





Python Lambda Function

A lambda function is a small, anonymous (nameless) function defined using the `lambda` keyword.

Syntax

`lambda arguments: expression`

↑
Input

↑
Output

Key Points

- ✓ Lambda functions can have any number of arguments, but only one expression.
- ✓ The expression is evaluated and returned automatically.
- ✓ Lambda functions are anonymous (no name).
- ✓ Useful for short, simple operations.
- ✓ Often used as arguments to higher-order functions (like `map()`, `filter()`, `sorted()`, etc.).

Lambda vs def Function

def function	lambda function
Has a name	Anonymous (no name)
Can have multiple statements	Can have only one expression
Uses return keyword	Returns automatically
More readable for complex logic	Useful for short, simple logic

Basic Examples

1. No argument

```
f = lambda: "Hello"
print(f()) # Output: Hello
```

2. One argument

```
f = lambda x: x * 2
print(f(5)) # Output: 10
```

3. Multiple arguments

```
f = lambda x, y: x + y
print(f(3, 4)) # Output: 7
```

More Examples

1. Square of a number

```
square = lambda x: x ** 2
print(square(4)) # 16
```

4. Check even or odd

```
is_even = lambda x: x % 2 == 0
print(is_even(10)) # True
print(is_even(7)) # False
```

2. Maximum of two numbers

```
maximum = lambda a, b: a if a > b else b
print(maximum(10, 20)) # 20
```

5. String length

```
length = lambda s: len(s)
print(length("Python")) # 6
```

3. Concatenate two strings

```
concat = lambda a, b: a + " " + b
print(concat("Hello", "World"))
# Hello World
```

6. List sorting with lambda

```
nums = [(1, 3), (4, 1), (2, 2)]
nums.sort(key=lambda x: x[1])
print(nums) # [(4, 1), (2, 2), (1, 3)]
```

Using Lambda with Built-in Functions

1. `map()` – Apply function to all items

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x ** 2, nums))
print(squared) # [1, 4, 9, 16]
```

2. `filter()` – Filter items based on condition

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6]
```

3. `sorted()` – Sort using custom key

```
words = ['apple', 'banana', 'cherry', 'date']
sorted_words = sorted(words, key=lambda x: len(x))
print(sorted_words) # ['date', 'apple', 'cherry', 'banana']
```

When to Use Lambda?

- ✓ When you need a small function for a short time.
- ✓ When you are passing a function as an argument.
- ✓ When the logic is simple and readability is not compromised.

Limitations

- ❌ Can have only one expression.
- ❌ Not suitable for complex logic.
- ❌ Harder to debug (no name).
- ❌ Readability can decrease if overused.

Equivalent def vs lambda

```
lambda x: x * 2
is equivalent to:
def f(x):
    return x * 2
```

```
lambda a, b: a + b
is equivalent to:
def f(a, b):
    return a + b
```

```
lambda x: x if x > 0 else -x
is equivalent to:
def f(x):
    return x if x > 0 else -x
```

Summary



Lambda functions are small, anonymous functions defined using `lambda`. They can have any number of arguments but only one expression. They are mainly used with higher-order functions like `map()`, `filter()`, and `sorted()`.



PYTHON STRING

Strings are immutable sequence of characters.
They are enclosed in single (' '), double (" ") or triple ("'' ") quotes.

```
text = "Hello, Python!"  
print(text.upper()) # HELLO, PYTHON!  
print(text[0])      # H  
print(len(text))    # 14
```



1. CREATING STRINGS

- Single Quotes
`'Hello'`
- Double Quotes
`"Hello"`
- Triple Quotes (Multi-line)
`'''Hello Python'''`
- From Characters (join)
`''.join(['H','i']) # 'Hi'`
- From ASCII (chr)
`chr(65) # 'A'`



TIP

Strings are immutable,
so you cannot change a string,
you can only create a new one.

2. COMMON STRING METHODS

Method	Description	Example	Output
<code>lower()</code>	Convert to lowercase	<code>'PyThOn'.lower()</code>	<code>'python'</code>
<code>upper()</code>	Convert to uppercase	<code>'python'.upper()</code>	<code>'PYTHON'</code>
<code>title()</code>	First letter of each word capitalized	<code>'hello world'.title()</code>	<code>'Hello World'</code>
<code>capitalize()</code>	First character capitalized	<code>'python'.capitalize()</code>	<code>'Python'</code>
<code>swapcase()</code>	Swap case	<code>'PyThOn'.swapcase()</code>	<code>'pYtHoN'</code>
<code>strip()</code>	Remove leading & trailing whitespace	<code>' hi '.strip()</code>	<code>'hi'</code>
<code>lstrip()</code>	Remove leading whitespace	<code>' hi '.lstrip()</code>	<code>'hi'</code>
<code>rstrip()</code>	Remove trailing whitespace	<code>' hi '.rstrip()</code>	<code>'hi'</code>
<code>replace(old, new)</code>	Replace substring	<code>'hello'.replace('l','x')</code>	<code>'hexxo'</code>
<code>find(sub)</code>	Return index of first occurrence	<code>'hello'.find('l')</code>	<code>2</code>
<code>count(sub)</code>	Count occurrences	<code>'banana'.count('a')</code>	<code>3</code>
<code>startswith(sub)</code>	Check if starts with	<code>'hello'.startswith('he')</code>	<code>True</code>
<code>endswith(sub)</code>	Check if ends with	<code>'hello'.endswith('lo')</code>	<code>True</code>
<code>split(sep)</code>	Split into list	<code>'a,b,c'.split(',')</code>	<code>['a','b','c']</code>
<code>join(iterable)</code>	Join iterable by separator	<code>'-'.join(['a','b','c'])</code>	<code>'a-b-c'</code>
<code>len()</code>	Length of string	<code>len('Python')</code>	<code>6</code>
<code>center(width, fill)</code>	Center string	<code>'hi'.center(6, '*')</code>	<code>'**hi**'</code>
<code>ljust(width, fill)</code>	Left justify string	<code>'hi'.ljust(6, '.')</code>	<code>'hi.....'</code>
<code>rjust(width, fill)</code>	Right justify string	<code>'hi'.rjust(6, '.')</code>	<code>'.....hi'</code>
<code>isalnum()</code>	Alphanumeric?	<code>'abc123'.isalnum()</code>	<code>True</code>
<code>isalpha()</code>	Alphabetic?	<code>'abc'.isalpha()</code>	<code>True</code>
<code>isdigit()</code>	Digit?	<code>'123'.isdigit()</code>	<code>True</code>
<code>isspace()</code>	Whitespace?	<code>' '.isspace()</code>	<code>True</code>

3. STRING INDEXING & SLICING

Indexing

index starts from 0

```
text = 'Hello, Python!'

H e l l o , P y t h o n !
0 1 2 3 4 5 6 7 8 9 10 11 12 13
-14 -12 -10 -8 -6 -4 -2 -1
```

Slicing

Syntax	Example	Output
<code>s[start:stop]</code>	<code>text[0:5]</code>	<code>'Hello'</code>
<code>s[start:]</code>	<code>text[7:]</code>	<code>'Python!'</code>
<code>s[:stop]</code>	<code>text[:5]</code>	<code>'Hello'</code>
<code>s[start:stop:step]</code>	<code>text[:2]</code>	<code>'Hl o ol!'</code>
<code>s[::-1]</code>	<code>text[::-1]</code>	<code>'!nohtyP ,olleH'</code>

4. STRING FORMATTING

f-strings (Python 3.6+)

```
name = 'Alice'  
age = 25  
print(f"My name is {name} and I am {age}.")  
# My name is Alice and I am 25.
```

str.format()

```
print("My name is {} and I am {}".format(name, age))  
# My name is Alice and I am 25.
```

% Formatting (Old Style)

```
print("My name is %s and I am %d." % (name, age))  
# My name is Alice and I am 25.
```

5. ESCAPE SEQUENCES

Escape	Meaning
<code>\n</code>	New Line
<code>\t</code>	Tab
<code>\\</code>	Backslash
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote

6. STRING OPERATIONS

Operation	Example	Result
Concatenation (+)	<code>'Hello' + ' ' + 'World'</code>	<code>'Hello World'</code>
Repetition (*)	<code>'Hi' * 3</code>	<code>'HiHiHi'</code>
Membership (in)	<code>'Py' in 'Python'</code>	<code>True</code>
Not in	<code>'Java' not in 'Python'</code>	<code>True</code>

7. USEFUL BUILT-IN FUNCTIONS

Function	Example	Output
<code>len(s)</code>	<code>len('Python')</code>	<code>6</code>
<code>ord(c)</code>	<code>ord('A')</code>	<code>65</code>
<code>chr(i)</code>	<code>chr(65)</code>	<code>'A'</code>
<code>min(s)</code>	<code>min('Python')</code>	<code>'P'</code>
<code>max(s)</code>	<code>max('Python')</code>	<code>'y'</code>
<code>sorted(s)</code>	<code>sorted('bca')</code>	<code>['a','b','c']</code>
<code>reversed(s)</code>	<code>reversed('abc')</code>	<code><reversed object></code>

8. EXAMPLES (WITH OUTPUT)

```
1. text = "Hello, Python!"  
   print(text.upper()) # HELLO, PYTHON!  
   print(text.lower()) # hello, python!  
  
2. text = " Python "  
   print(text.strip()) # 'Python'  
  
3. text = "banana"  
   print(text.count('a')) # 3  
   print(text.find('na')) # 2  
  
4. text = "a,b,c"  
   print(text.split(',')) # ['a', 'b', 'c']  
  
5. text = "Hello"  
   print(text.center(10, '*'))  
   # '**Hello**'  
  
6. a = "Hi"  
   b = "There"  
   print(a + ' ' + b) # Hi There  
   print(a * 3) # HiHiHi  
  
7. text = "Python"  
   print(text[1:4]) # yth  
   print(text[::-1]) # nohtyP  
  
8. text = "123abc"  
   print(text.isalnum()) # True  
  
9. name = "Alice"  
   age = 25  
   print(f"Name: {name}, Age: {age}")  
   # Name: Alice, Age: 25  
  
10. for ch in "Py":  
     print(ch) # P  
              # y
```

9. MULTI-LINE STRINGS

Using Triple Quotes

```
text = """  
This is a multi-line string.  
It can span across  
multiple lines.  
"""  
  
print(text)  
  
# This is a multi-line string.  
# It can span across  
# multiple lines.
```

10. RAW STRINGS

```
raw = r"C:\Users\Alice\new_folder"  
print(raw)  
# C:\Users\Alice\new_folder
```

Use raw strings (r"...") to avoid escaping backslashes.

11. STRING COMPARISON

```
a = "apple"  
b = "banana"  
print(a == b) # False  
print(a != b) # True  
print(a < b) # True  
print(a > b) # False
```

12. IMPORTANT NOTES

- Strings are ordered.
- Strings are immutable (cannot be changed). Any operation returns a new string.
- Index starts from 0.
- Supports slicing.
- Strings in Python are Unicode.
- Use single, double or triple quotes.
- Many methods return new strings without modifying the original.



13. COMMON MISTAKES

- Trying to change a character:
`s = "hello"
s[0] = "H" # TypeError`
- Forgetting that methods return new strings:
`s = "hello"
s.upper() # Doesn't change s`
- Using + in loops for large strings (slow)
Better: use `join()`.



14. QUICK RECAP

- Strings are immutable sequences of characters.
- Created using `'', ""` or `''' '''`.
- Use indexing and slicing to access parts.
- Many built-in methods make string manipulation easy.
- Strings are powerful and widely used in Python programs.





PYTHON STRINGS METHODS



Strings are ordered, immutable sequence of characters.

```
text = "Hello, Python!"
print(text.upper())
# HELLO, PYTHON!
```

STRING METHODS (Most Used)

No.	Method	Description	Example	Output
1	upper()	Converts string to uppercase	text.upper()	'HELLO, PYTHON!'
2	lower()	Converts string to lowercase	text.lower()	'hello, python!'
3	title()	Converts first letter of each word to uppercase	text.title()	'Hello, Python!'
4	capitalize()	Capitalizes the first character	text.capitalize()	'Hello, python!'
5	swapcase()	Swaps case of each character	text.swapcase()	'hELLO, pYTHON!'
6	strip()	Removes whitespace from both sides	text.strip()	'Hello, Python!'
7	lstrip()	Removes whitespace from left side	text.lstrip()	'Hello, Python!'
8	rstrip()	Removes whitespace from right side	text.rstrip()	' Hello, Python!'
9	replace(old,new)	Replaces old substring with new substring	text.replace("Python", "World")	'Hello, World!'
10	find(sub)	Returns index of first occurrence of substring	text.find("Python")	7
11	count(sub)	Returns count of occurrences of substring	text.count("l")	2
12	startswith(sub)	Checks if string starts with substring	text.startswith("Hello")	True
13	endswith(sub)	Checks if string ends with substring	text.endswith("!")	True
14	split(sep)	Splits string into a list (by separator)	text.split(", ")	['Hello', 'Python!']
15	join(iterable)	Joins elements of iterable into a string	"-".join(['a', 'b', 'c'])	'a-b-c'
16	len()	Returns length of string	len(text)	14
17	center(width, fill)	Centers string in given width with fill character	text.center(20, '*')	'***Hello, Python!***'
18	ljust(width, fill)	Left aligns string in given width with fill	text.ljust(20, '.')	'Hello, Python!.....'
19	rjust(width, fill)	Right aligns string in given width with fill	text.rjust(20, '.')	'.....Hello, Python!'
20	isalpha()	Checks if all characters are alphabets	"Hello".isalpha()	True
21	isdigit()	Checks if all characters are digits	"12345".isdigit()	True
22	isalnum()	Checks if all characters are alphanumeric	"Hello123".isalnum()	True
23	isspace()	Checks if all characters are whitespace	" ".isspace()	True
24	islower()	Checks if all characters are lowercase	"hello".islower()	True
25	isupper()	Checks if all characters are uppercase	"HELLO".isupper()	True
26	istitle()	Checks if string is in title case	"Hello World".istitle()	True
27	isprintable()	Checks if all characters are printable	"Hello!".isprintable()	True

EXAMPLES (With Output)

```
1 text = "Hello, Python!"
2 print(text.upper()) # HELLO, PYTHON!
3 print(text.lower()) # hello, python!
4 print(text.title()) # Hello, Python!
5 print(text.capitalize()) # Hello, python!
6 print(text.swapcase()) # hELLO, pYTHON!
7 print(" Hello, Python! ".strip())
  # 'Hello, Python!'
8 print(text.replace("Python", "World"))
  # Hello, World!
9 print(text.find("Python")) # 7
10 print(text.count("l")) # 2
11 print(text.startswith("Hello")) # True
12 print(text.endswith("!")) # True
13 print(text.split(", ")) # ['Hello', 'Python!']
14 a = "-".join(['a', 'b', 'c'])
  print(a) # a-b-c
15 print(len(text)) # 14
16 print(text.center(20, '*')) # ***Hello, Python!***
17 print(text.ljust(20, '.')) # Hello, Python!.....
18 print(text.rjust(20, '.')) # .....Hello, Python!
19 print("Hello".isalpha()) # True
20 print("12345".isdigit()) # True
21 print("Hello123".isalnum()) # True
22 print(" ".isspace()) # True
23 print("hello".islower()) # True
24 print("HELLO".isupper()) # True
25 print("Hello World".istitle()) # True
26 print("Line1\nLine2".isprintable()) # False
```



TIP

Strings are immutable. You cannot change a string, you can only create a new one.



STRING SLICING (Not a Method)

```
text = "Hello, Python!"
```

Slicing	Meaning	Output
text[0:5]	First 5 characters	'Hello'
text[7:]	From index 7 to end	'Python!'
text[:5]	From start to index 5	'Hello'
text[-7:]	Last 7 characters	'Python!'
text[::-1]	Reverse the string	'!nohtyP ,oLleH'
text[2:10:2]	Step of 2 from 2 to 9	'loPto'

STRING FORMATTING

```
name = "Alice"; age = 25
```

```
• f"My name is {name} and age is {age}." # f-string
• "My name is {} and age is {}".format(name, age) # format()
• "My name is %s and age is %d." % (name, age) # % operator
```

ESCAPE SEQUENCES

\n	New Line	\'	Single Quote
\t	Tab	\\	Backslash
\"	Double Quote		

CHECK METHODS

- ✓ isalpha() -> Only alphabets
- ✓ isdigit() -> Only digits
- ✓ isalnum() -> Alphabets + digits
- ✓ isspace() -> Only spaces
- ✓ islower() -> All lowercase
- ✓ isupper() -> All uppercase
- ✓ istitle() -> Title case
- ✓ isprintable() -> All printable chars



Strings are immutable, ordered and versatile. Use these methods to process and manipulate text efficiently!





PYTHON STRING FORMATTING

String formatting allows you to insert variables or values into a string in a structured way.

```
name = "Alice"
age = 30
print(f"My name is {name}
and I am {age} years old.")
```



1. OLD STYLE FORMATTING (%)

Specifier	Meaning	Example	Output
%s	String (or any object with string representation)	"%s" % "Hello"	"Hello"
%d	Integers	"%d" % 123	"123"
%f	Floating point numbers	"%f" % 3.14159	"3.141590"
%x	Integers in hexadecimal (lowercase)	"%x" % 255	"ff"
%X	Integers in hexadecimal (uppercase)	"%X" % 255	"FF"
%%	A literal '%' character	"100%% sure"	"100% sure"

EXAMPLE

```
name = "Alice"
age = 30
print("My name is %s and I am %d years old." % (name, age))
# Output: My name is Alice and I am 30 years old.
```

2. NEW STYLE FORMATTING (str.format())

Format	Description
{}	Placeholder for any type of value
{0}, {1}, ...	Positional arguments
{name}	Named placeholders
{:.2f}	Formatting for floating point (2 decimal places)
{0:10}	Format with width (10 characters wide)
{0:<10}	Left-align within the specified width
{0:>10}	Right-align within the specified width
{0:^10}	Center-align within the specified width
{0:0>10}	Pad with zeros on the left to fill width
{0:0<10}	Pad with zeros on the right to fill width
{0:0^10}	Pad with zeros on both sides to fill width
{0:, .2f}	Number with commas and 2 decimal places
{0:.2%}	Number as percentage with 2 decimal places
{0:10.2f}	Number with width 10 and 2 decimal places
{0:>10.2f}	Number with width 10, 2 decimals, right-align

3. BASIC EXAMPLES (NEW STYLE)

```
# 1: Using {} placeholders
print("My name is {} and I am {} years old.".format(name, age))

# 2: Using positional arguments
print("My name is {0} and I am {1} years old.".format(name, age))

# 3: Using named placeholders
print("My name is {name} and I am {age} years old."
      .format(name=name, age=age))
```

4. FLOATING POINT FORMATTING

```
pi = 3.14159
print("Pi is approximately {:.2f}.".format(pi))
# Output: Pi is approximately 3.14.
```

5. WIDTH & ALIGNMENT

```
name = "Alice"
age = 30

# Default (same as {0:10})
print("Name: {0:10} Age: {1:5}".format(name, age)) # Name: Alice Age: 30

# Left align
print("Name: {0:<10} Age: {1:5}".format(name, age)) # Name: Alice Age: 30

# Right align
print("Name: {0:>10} Age: {1:5}".format(name, age)) # Name: Alice Age: 30

# Center align
print("Name: {0:^10} Age: {1:5}".format(name, age)) # Name: Alice Age: 30

# Pad with zeros on the left
print("Name: {0:0>10} Age: {1:5}".format(name, age)) # Name: 00000Alice Age: 30

# Pad with zeros on the right
print("Name: {0:0<10} Age: {1:5}".format(name, age)) # Name: Alice00000 Age: 30

# Pad with zeros on both sides
print("Name: {0:0^10} Age: {1:5}".format(name, age)) # Name: 00Alice00 Age: 30
```

6. COMMAS & PERCENTAGE

```
number = 1234567.89
print("Formatted number: {0:,.2f}".format(number))
# Output: Formatted number: 1,234,567.89

print("Percentage: {0:.2%}".format(0.1234))
# Output: Percentage: 12.34%
```

7. WIDTH & DECIMAL PLACES

```
pi = 3.14159
print("Formatted number: {0:10.2f}".format(pi))
# Output: Formatted number: 3.14

print("Formatted number: {0:>10.2f}".format(pi))
# Output: Formatted number: 3.14
```

8. MORE EXAMPLES

```
# Multiple types
print("String: {}, Integer: {}, Float: {:.2f}".format("Hi", 42, 3.14159))
# Output: String: Hi, Integer: 42, Float: 3.14

# Hexadecimal
print("Hex (lower): {0:x}, Hex (UPPER): {0:X}".format(255, 255))
# Output: Hex (lower): ff, Hex (UPPER): FF

# Percentage with width
print("Success rate: {0:>10.2%}".format(0.9876))
# Output: Success rate: 98.76%

# Number with width and commas
print("Total amount: {0:>15,.2f}".format(9876543.21))
# Output: Total amount: 9,876,543.21

# Mixed named and positional
print("Hello {0}, your balance is {balance:.2f} USD.".format("Alice", balance=2500))
# Output: Hello Alice, your balance is 2500.00 USD.
```

9. QUICK RECAP

- ★ % formatting is the old style (C-style).
- ★ str.format() is the new and powerful way.
- ★ F-strings (f"...{var}...") are the modern and most readable way.
- ★ Use alignment and width for clean output.
- ★ Use formatting options for numbers, percentages, and more.
- ★ Readable formatting makes your code and outputs professional!



TIP: F-strings (Python 3.6+) are the easiest and most readable way!

```
name = "Alice"; age = 30
print(f"My name is {name} and I am {age} years old.")
```





PYTHON F-STRINGS

f-strings (formatted string literals) make it easy to embed expressions inside string literals using curly braces {}.

```
name = "Alice"
age = 25
msg = f"Hello, {name}! You are {age} years old."
print(msg) # Hello, Alice! You are 25 years old.
```



1. BASIC SYNTAX

Prefix the string with f or F and place expressions inside {}.

```
name = "Alice"
age = 25
print(f"Name: {name}, Age: {age}")

# Output:
# Name: Alice, Age: 25
```

2. SIMPLE EXAMPLES

- Single variable

```
x = 10
print(f"Value: {x}") # Value: 10
```
- Multiple variables

```
a, b = 5, 7
print(f"Sum: {a + b}") # Sum: 12
```
- Expressions inside {}

```
pi = 3.1416
r = 2
print(f"Area: {pi * r ** 2}")
# Area: 12.5664
```
- Calling functions

```
import math
x = 16
print(f"Square root: {math.sqrt(x)}")
# Square root: 4.0
```

3. FORMATTING OPTIONS

Use format specifiers after a colon inside {}.

Specifier	Meaning	Example	Output
d	Integer	f"{42:d}"	42
f	Fixed-point	f"{3.14:f}"	3.140000
.2f	2 decimal places	f"{3.14159:.2f}"	3.14
.3f	3 decimal places	f"{3.14159:.3f}"	3.142
%	Percentage	f"{0.25:%}"	25.000000%
,	Comma separator	f"{1234567:,}"	1,234,567
_	Underscore sep.	f"{1234567:_}"	1_234_567
x	Hex (lowercase)	f"{255:x}"	ff
X	Hex (UPPERCASE)	f"{255:X}"	FF
>10	Right align (width 10)	f"{42:>10}"	42
<10	Left align (width 10)	f"{42:<10}"	42
^10	Center align (width 10)	f"{42:^10}"	42

12. PRACTICAL EXAMPLES

- Price Calculation

```
price = 99.99
tax = 0.18
total = price + price * tax
print(f"Total price: ${total:.2f}")
# Total price: $117.99
```
- Table-like Output

```
headers = ["Name", "Score"]
row = ["Alice", 95]
print(f'{headers[0]:<10} | {headers[1]:>5}')
print(f'{row[0]:<10} | {row[1]:>5}')
print(f'{row[0]:<10} | {row[1]:>5}')
# Name | Score
# -----+-----
# Alice | 95
```
- Number Formatting

```
num = 1234567.8912
print(f"With commas: {num:,}") # 1,234,567.89
print(f"With underscore: {num:_}") # 1_234_567.89
```
- Percentage

```
part = 32
whole = 80
print(f"Percentage: {part/whole:.1%}") # 40.0%
```

4. COMMON USE CASES

Use Case	Example	Output
Variables	<pre>name = "Bob" print(f"Hello, {name}!")</pre>	Hello, Bob!
Multiple variables	<pre>x, y = 3, 4 print(f"x={x}, y={y}")</pre>	x=3, y=4
Expressions	<pre>a, b = 10, 20 print(f"a+b = {a + b}")</pre>	a+b = 30
Condition in expression	<pre>n = 7 print(f'Even' if n%2==0 else 'Odd')</pre>	Odd
Loop with f-string	<pre>for i in range(3): print(f"Item {i}")</pre>	Item 0 Item 1 Item 2
Dictionary values	<pre>person = {"name": "Eve", "age": 30} print(f"{person['name']} is {person['age']} years old.")</pre>	Eve is 30 years old.

5. ALIGNMENT EXAMPLES

Alignment	Example	Output (width=10)
Left <	f"{42:<10}"	42
Right >	f"{42:>10}"	42
Center ^	f"{42:^10}"	42
Left (text)	f"{ 'hi':<10}"	hi
Right (text)	f"{ 'hi':>10}"	hi
Center (text)	f"{ 'hi':^10}"	hi

6. NESTED EXPRESSIONS

You can use expressions inside expressions.

```
x = 5
print(f"x squared is {x**2}") # x squared is 25
print(f"{'Yes' if x > 0 else 'No'}") # Yes
```

7. DATE & TIME EXAMPLE

```
from datetime import datetime
today = datetime(2024, 5, 20, 15, 30)
print(f"Date: {today:%Y-%m-%d}") # Date: 2024-05-20
print(f"Time: {today:%H:%M:%S}") # Time: 15:30:00
```

8. CONVERSION FLAGS

Use !r, !s, !a to convert values.

Flag	Meaning	Example	Output
!s	str()	f"{42!s}"	'42'
!r	repr()	f"{42!r}"	'42'
!a	ascii()	f"{42!a}"	'42'

9. MULTI-LINE F-STRINGS

Use triple quotes for multi-line strings.

```
name = "Alice"
age = 25
text = f"""
Hello {name},
You are {age} years old.
Have a great day!
"""
print(text)

# Output:
# Hello Alice,
# You are 25 years old.
# Have a great day!
```



10. RAW F-STRINGS

Combine r and f to avoid escape sequences while using expressions.

```
path = r"C:\new\folder"
file = "data.txt"
print(fr"Path: {path}\{file}")
# Output: Path: C:\new\folder\data.txt
```



TIP: Use fr"..." or rf"..."

11. DEBUGGING WITH = (PYTHON 3.8+)

Add = after expression to print its value and representation.

```
x = 42
y = x * 2
print(f"x = {x}") # x = 42
print(f"y = {y}") # y = 84
print(f"x + y = {x + y}") # x + y = 126
```



13. QUICK RECAP

- ✓ Prefix string with f or F.
- ✓ Embed expressions in {}.
- ✓ Use : for formatting.
- ✓ Use !r, !s, !a for conversions.
- ✓ Support alignment, width, precision, and more.
- ✓ Raw f-strings: fr"..."
- ✓ Debug with = (Python 3.8+).



f-strings are fast, readable, and powerful. Use them to write clean and maintainable Python code!

```
# Always prefer f-strings for string formatting!
name = "Python"
print(f"I love {name}!")
```





PYTHON LISTS

Store multiple items in a single variable.
Lists are ordered, changeable, and allow duplicates.



1. WHAT IS A LIST?

A list is an ordered collection of items.
It is changeable (mutable) and allows duplicate values.

```
my_list = [10, 20, 30, "hello", True]
           ↑   ↑   ↑   ↑   ↑
           0   1   2   3   4
```

Memory Trick

Think of a list as a shopping cart.
You can add, remove or change items anytime.



2. CREATING LISTS

You can create a list by placing items inside square brackets [].

Example	Description
[]	Empty list
[1, 2, 3]	List of integers
["a", "b", "c"]	List of strings
[1, "a", 3.5, True]	List with different data types
[[1, 2], [3, 4]]	Nested list (list inside a list)

Example Code

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob", "Charlie"]
mix = [10, "Python", 3.14, False]
nested = [[1,2], [3,4], [5,6]]
```

3. ACCESSING ITEMS

List items are indexed starting from 0.

Example:

```
fruits = ["apple", "banana", "cherry", "mango"]
         ↓   ↓   ↓   ↓
         0   1   2   3
```

Operation	Example	Result
First item	fruits[0]	'apple'
Second item	fruits[1]	'banana'
Last item	fruits[3]	'mango'
Negative index	fruits[-1]	'mango'
Second last	fruits[-2]	'cherry'
Slice (0 to 2)	fruits[0:3]	['apple', 'banana', 'cherry']
Slice (1 to end)	fruits[1:]	['banana', 'cherry', 'mango']

★ Syntax: list[start:stop:step]

4. MODIFYING LISTS

Lists are changeable. You can update, add or remove items.

Operation	Example	Result
Change item	nums = [10, 20, 30] nums[1] = 99	[10, 99, 30]
Add item (append)	nums.append(40)	[10, 99, 30, 40]
Insert item	nums.insert(1, 15)	[10, 15, 99, 30, 40]
Extend list	nums.extend([50, 60])	[10, 15, 99, 30, 40, 50, 60]
Remove item by value	nums.remove(99)	[10, 15, 30, 40, 50, 60]
Remove at index	nums.pop(0)	[15, 30, 40, 50, 60]
Clear list	nums.clear()	[]

✓ append() adds one item, extend() adds multiple items.

5. COMMON LIST OPERATIONS

Operation	Example	Result
Length	len(nums)	6
Min value	min(nums)	10
Max value	max(nums)	60
Sum (numbers)	sum(nums)	225
Check item	20 in nums	True
Count item	nums.count(30)	1
Index of item	nums.index(40)	3

Example Code

```
nums = [10, 20, 30, 40, 50, 60]
print(len(nums)) # 6
print(sum(nums)) # 210
print(30 in nums) # True
```

6. LIST SLICING

Get a part (slice) of the list.

Slice	Meaning	Example	Result
a[start:stop]	start to stop-1	a[1:4]	[20, 30, 40]
a[:stop]	from beginning	a[:3]	[10, 20, 30]
a[start:]	to end	a[2:]	[30, 40, 50, 60]
a[start:stop:step]	with step	a[::2]	[10, 30, 50]
a[::-1]	reverse list	a[::-1]	[60, 50, 40, 30, 20, 10]

Example:

```
a = [10, 20, 30, 40, 50, 60]
print(a[1:5]) # [20, 30, 40, 50]
print(a[::2]) # [10, 30, 50]
print(a[::-1]) # [60, 50, 40, 30, 20, 10]
```



7. NESTED LISTS

A list can contain other lists.

Example:

```
matrix = [[1,2,3], [4,5,6], [7,8,9]]
```

[1, 2, 3]	→ row 0
[4, 5, 6]	→ row 1
[7, 8, 9]	→ row 2
↓ ↓ ↓	
col 0 col 1 col 2	

Access Items

```
print(matrix[0][1]) # 2
print(matrix[2][2]) # 9
```

8. LIST METHODS (QUICK REFERENCE)

Method	Use	Example
append(x)	Add item at end	lst.append(10)
insert(i, x)	Insert at index i	lst.insert(1, 5)
extend(lst2)	Add items from another list	lst.extend([1,2])
remove(x)	Remove first occurrence of x	lst.remove(5)
pop(i)	Remove & return item at i	lst.pop(2)
clear()	Remove all items	lst.clear()
index(x)	Return index of first x	lst.index(5)
count(x)	Count occurrences of x	lst.count(5)
sort()	Sort list (ascending)	lst.sort()
reverse()	Reverse list	lst.reverse()

★ Most methods change the original list.

9. IMPORTANT NOTES

- ✓ Lists are ordered.
- ✓ Lists are changeable.
- ✓ Lists allow duplicate values.
- ✓ List index starts from 0.
- ✓ Lists can store different data types.
- ✓ Use len() to find the number of items.
- ✓ Use in to check if an item exists.

💡 Out of range index will give IndexError.



10. COMMON MISTAKES

- ✗ Accessing index that doesn't exist.
a = [1,2,3]
print(a[3]) # IndexError
- ✗ Using remove() for index (use pop() instead).
a.remove(1) # removes value 1
a.pop(1) # removes item at index 1
- ✗ Forgetting that lists are mutable (changes affect original list).
x = [1,2]
y = x
y.append(3) # x is now [1,2,3] too!
- ✗ Confusing append() and extend().
a = [1,2]
a.append([3,4]) # [[3,4]]
a.extend([3,4]) # [1,2,3,4]



11. PRACTICE EXAMPLES

- 1 Create a list of 5 your favorite movies and print it.
- 2 Add a new movie to the end of the list.
- 3 Remove the 2nd movie from the list.
- 4 Print the first and last movie.
- 5 Check if "Inception" is in the list.
- 6 Reverse the list and print it.
- 7 Find the number of movies in the list.



12. QUICK RECAP

- ✓ Lists store multiple items.
- ✓ They are ordered, changeable, allow duplicates.
- ✓ Use [] to create a list.
- ✓ Use index to access items.
- ✓ Use methods to modify lists.
- ✓ Slices help get parts of the list.
- ✓ Lists make your code powerful and flexible!



BONUS: LIST VS TUPLE

List

- ✓ Created with []
- ✓ Changeable (mutable)
- ✓ Allows duplicates
- ✓ Example: [1,2,3]



VS

Tuple

- ✓ Created with ()
- ✓ Unchangeable (immutable)
- ✓ Allows duplicates
- ✓ Example: (1,2,3)



When to Use?



Use List → when data can change.
Use Tuple → when data should not change (useful for fixed data and as dictionary keys).





LIST METHODS IN PYTHON

Python has a set of built-in methods that you can use on lists.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")
print(fruits)
# Output: ['apple', 'banana', 'cherry', 'orange']
```



#	METHOD	DESCRIPTION	SYNTAX	EXAMPLE	OUTPUT
1	append() 	Adds an element at the end of the list	<code>list.append(element)</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.append("orange") print(thislist)</pre>	<code>['apple', 'banana', 'cherry', 'orange']</code>
2	clear() 	Removes all the elements from the list	<code>list.clear()</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.clear() print(thislist)</pre>	<code>[]</code>
3	copy() 	Returns a copy of the list	<code>list.copy()</code>	<pre>thislist = ["apple", "banana", "cherry"] newlist = thislist.copy() print(newlist)</pre>	<code>['apple', 'banana', 'cherry']</code>
4	count() 	Returns the number of elements with the specified value	<code>list.count(value)</code>	<pre>thislist = ["apple", "banana", "cherry", "apple"] x = thislist.count("apple") print(x)</pre>	<code>2</code>
5	extend() 	Add the elements of a list (or any iterable), to the end of the current list	<code>list.extend(iterable)</code>	<pre>thislist = ["apple", "banana", "cherry"] tropical = ["mango", "pineapple", "papaya"] thislist.extend(tropical) print(thislist)</pre>	<code>['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']</code>
6	index() 	Returns the index of the first element with the specified value	<code>list.index(value)</code>	<pre>thislist = ["apple", "banana", "cherry"] x = thislist.index("banana") print(x)</pre>	<code>1</code>
7	insert() 	Adds an element at the specified position	<code>list.insert(index, element)</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.insert(1, "orange") print(thislist)</pre>	<code>['apple', 'orange', 'banana', 'cherry']</code>
8	pop() 	Removes the element at the specified position (default is the last item)	<code>list.pop(index)</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.pop(1) print(thislist)</pre>	<code>['apple', 'cherry']</code>
9	remove() 	Removes the first item with the specified value	<code>list.remove(value)</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.remove("banana") print(thislist)</pre>	<code>['apple', 'cherry']</code>
10	reverse() 	Reverses the order of the list	<code>list.reverse()</code>	<pre>thislist = ["apple", "banana", "cherry"] thislist.reverse() print(thislist)</pre>	<code>['cherry', 'banana', 'apple']</code>
11	sort() 	Sorts the list in ascending order (works on numbers and strings)	<code>list.sort()</code> # or <code>list.sort(reverse=True)</code> # for descending order	<pre>thislist = ["banana", "apple", "cherry"] thislist.sort() print(thislist)</pre>	<code>['apple', 'banana', 'cherry']</code>

12. LIST LENGTH (BUILT-IN FUNCTION)

The `len()` function returns the number of items in a list.

```
thislist = ["apple", "banana", "cherry"]
print(f"Length of this list is: {len(thislist)}
and the content is given below:")
for x in thislist:
    print(x)
```

OUTPUT

```
Length of this list is: 3
and the content is
given below:
apple
banana
cherry
```

QUICK NOTES

- ✓ Lists are ordered, mutable (changeable) collections.
- ✓ Many methods modify the original list and return `None`.
- ✓ Some methods like `copy()`, `index()`, `count()`, `sorted()` return new values.
- ✓ Use `help(list)` in Python to see all methods.



SUMMARY OF METHODS

- | | | | | | |
|---|-----------------|--|----|------------------|---|
| 1 | append() | Add element at the end | 7 | insert() | Insert element at a specific position |
| 2 | clear() | Remove all elements | 8 | pop() | Remove element at a specific position |
| 3 | copy() | Return a copy of the list | 9 | remove() | Remove first item with a specific value |
| 4 | count() | Count occurrences of a value | 10 | reverse() | Reverse the list in place |
| 5 | extend() | Add elements of iterable to the end | 11 | sort() | Sort the list in place (ascending by default) |
| 6 | index() | Get index of first occurrence of a value | | | |



Master these **list methods** and write **Python** code like a pro!





PYTHON TUPLES

Store multiple items in an ordered, immutable collection.

Tuples are fast, safe, and cannot be changed!

```
point = (10, 20)
colors = ("red", "green", "blue")
mixed = (1, "Python", 3.14, True)
print(point[0]) # 10
print(len(colors)) # 3
```



1. WHAT IS A TUPLE?

A tuple is an ordered collection of items enclosed in parentheses ().

It is immutable (cannot be changed after creation) and allows duplicate values.

Example:

```
t = (10, 20, 30, "hello", True)
      ↑   ↑   ↑   ↑   ↑
      0   1   2   3   4
```



Memory Trick

Think of a tuple as a packed box sealed with tape.

You can use the items, but cannot change what's inside.



2. CREATING TUPLES

Way	Example	Description
Empty tuple	t = ()	Creates an empty tuple
With values	t = (1, 2, 3)	Tuple with integers
Mixed data types	t = (1, "hi", 3.5, True)	Different data types
With single item	t = (5,)	Comma is important!
Without parentheses	t = 1, 2, 3	Valid tuple (optional !)
From another iterable	t = tuple([1, 2, 3])	Convert list to tuple

Example Code

```
empty = ()
nums = (1, 2, 3, 4)
fruits = ("apple", "banana", "cherry")
single = (42,) # note the comma
converted = tuple([10, 20, 30])
```

3. ACCESSING TUPLE ITEMS

Tuple items are indexed starting from 0.

Example:

```
t = ("a", "b", "c", "d")
      ↓   ↓   ↓   ↓
      0   1   2   3
```

Operation	Example	Result
First item	t[0]	'a'
Second item	t[1]	'b'
Last item	t[3]	'd'
Negative index -1	t[-1]	'd'
Negative index -2	t[-2]	'c'

Example Code

```
t = (10, 20, 30, 40)
print(t[0]) # 10
print(t[-1]) # 40
print(t[1]) # 20
```

4. TUPLE SLICING

Get a part (slice) of the tuple.

Slice	Meaning	Example (t = (10, 20, 30, 40, 50))	Result
t[start:stop]	start to stop-1	t[1:4]	(20, 30, 40)
t[:stop]	from beginning	t[:3]	(10, 20, 30)
t[start:]	to end	t[2:]	(30, 40, 50)
t[start:stop:step]	with step	t[::2]	(10, 30, 50)
t[::-1]	reverse tuple	t[::-1]	(50, 40, 30, 20, 10)

Example Code

```
t = (10, 20, 30, 40, 50)
print(t[1:4]) # (20, 30, 40)
print(t[:3]) # (10, 20, 30)
print(t[2:]) # (30, 40, 50)
print(t[::2]) # (10, 30, 50)
print(t[::-1]) # (50, 40, 30, 20, 10)
```



5. TUPLE OPERATIONS

Operation	Example	Result
Concatenation	(1, 2) + (3, 4)	(1, 2, 3, 4)
Repetition	(1, 2) * 3	(1, 2, 1, 2, 1, 2)
Membership (in)	2 in (1, 2, 3)	True
Length	len((1, 2, 3, 4))	4
Max value	max((1, 5, 3))	5
Min value	min((1, 5, 3))	1
Count	(1, 2, 2, 3).count(2)	2
Index	(1, 2, 3).index(2)	1

Example Code

```
t = (1, 2)
b = (3, 4)
print(a + b) # (1, 2, 3, 4)
print(a * 2) # (1, 2, 1, 2)
print(2 in a) # True
print(len(b)) # 2
```

6. IMMUTABILITY (IMPORTANT!)

Once a tuple is created, its items cannot be changed, added or removed.

Example:

```
t = (10, 20, 30)
t[0] = 100 # ❌ Error!
```

TypeError: 'tuple' object does not support item assignment

Why Immutability?

- ✓ Data is safe and cannot be accidentally modified.
- ✓ Tuples are faster than lists.
- ✓ Tuples can be used as dictionary keys.



7. TUPLE METHODS

Tuples have only two methods.

Method	Use	Example	Result
count(x)	Returns number of occurrences of x	(1, 2, 2, 3).count(2)	2
index(x)	Returns index of first occurrence of x	(1, 2, 3).index(2)	1

Example Code

```
t = (1, 2, 2, 3, 2)
print(t.count(2)) # 3
print(t.index(3)) # 3
```



8. TUPLE PACKING & UNPACKING

Packing: Combine values into a tuple.

Unpacking: Extract values from a tuple.

Packing

```
point = (10, 20)
print(point) # (10, 20)
```

Unpacking

```
x, y = (10, 20)
print(x, y) # 10 20
```

Multiple Unpacking

```
a, b, c = (1, 2, 3)
print(a, b, c) # 1 2 3
```



10. TUPLES VS LISTS

Feature	Tuple	List
Syntax	(1, 2, 3)	[1, 2, 3]
Mutable?	No (Immutable)	Yes (Mutable)
Performance	Faster	Slower
Memory	Less	More
Methods	Few	Many
Use Case	Fixed data	Dynamic data

Use Tuple when data should not change.
Use List when data needs to change.



11. COMMON MISTAKES

- ❌ Forgetting comma for single item tuple
t = (5) # ❌ Not a tuple
t = (5,) # ✅ Correct tuple
- ❌ Trying to change tuple items
t = (1, 2, 3)
t[1] = 10 # ❌ Error
- ❌ Using list methods on tuple
t = (1, 2, 3)
t.append(4) # ❌ AttributeError
- ❌ Confusing tuple with parentheses
x = (1 + 2) # ❌ This is 3
y = (1, +2) # ✅ This is a tuple (1, 2)



12. PRACTICE EXERCISES

- 1 Create a tuple with your 3 favorite colors.
- 2 Access and print the second color.
- 3 Print the last color using negative indexing.
- 4 Slice the tuple to get the first two colors.
- 5 Create a tuple of numbers (1 to 5) and reverse it.
- 6 Check if a number exists in the tuple.
- 7 Find the length of a tuple.
- 8 Count how many times an item appears.
- 9 Unpack a tuple into three variables.
- 10 Create a nested tuple and access an inner item.



13. QUICK RECAP



Tuple is an ordered collection of items.



It is immutable (cannot be changed).



Use parentheses () to create a tuple.



Access items using index (0, 1, 2, ...).



Use slicing to get parts of a tuple.



Supports useful operations and methods.



Safe, fast and memory efficient!



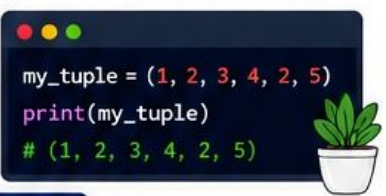
Great for fixed data and keys in dictionaries.



TUPLES METHODS IN PYTHON

Tuples are ordered, immutable collections.

They support various built-in methods and related functions.



#	METHOD / FUNCTION	DESCRIPTION	SYNTAX	EXAMPLE	OUTPUT
1	count(value) 	Returns the number of times a specified value appears in the tuple.	<code>tuple.count(value)</code>	<code>t = (1, 2, 3, 4, 2, 5)</code> <code>t.count(2)</code>	2
2	index(value) 	Returns the index of the first occurrence of a specified value. Raises <code>ValueError</code> if not found.	<code>tuple.index(value)</code>	<code>t = (1, 2, 3, 4, 2, 5)</code> <code>t.index(3)</code>	2
3	len() 	Returns the number of items in a tuple.	<code>len(tuple)</code>	<code>t = (1, 2, 3, 4, 2, 5)</code> <code>len(t)</code>	6
4	max() / min() 	Returns the largest (<code>max()</code>) or smallest (<code>min()</code>) item in the tuple.	<code>max(tuple)</code> <code>min(tuple)</code>	<code>t = (1, 2, 3, 4, 2, 5)</code> <code>max(t)</code> <code>min(t)</code>	5 1
5	sorted() 	Returns a new sorted list from the items in a tuple.	<code>sorted(tuple)</code>	<code>t = (3, 1, 4, 2, 2, 5)</code> <code>sorted(t)</code>	[1, 2, 2, 3, 4, 5]
6	sum() 	Returns the sum of all items in a tuple (numeric only).	<code>sum(tuple)</code>	<code>t = (1, 2, 3, 4, 5)</code> <code>sum(t)</code>	15
7	any() / all() 	<code>any()</code> returns True if any item is true. <code>all()</code> returns True if all items are true.	<code>any(tuple)</code> <code>all(tuple)</code>	<code>t = (True, False, True)</code> <code>any(t)</code> <code>all(t)</code>	True False
8	zip(*iterables) 	Takes iterables, aggregates them in a tuple, and returns an iterator.	<code>zip(iter1, iter2, ...)</code>	<code>a = (1, 2, 3)</code> <code>b = ('a', 'b', 'c')</code> <code>list(zip(a, b))</code>	[(1, 'a'), (2, 'b'), (3, 'c')]
9	enumerate() 	Adds a counter (index) to an iterable and returns an enumerate object.	<code>enumerate(iterable, start=0)</code>	<code>t = (10, 20, 30)</code> <code>list(enumerate(t))</code>	[(0, 10), (1, 20), (2, 30)]
10	tuple() 	Converts an iterable (like list, string, set, etc.) into a tuple.	<code>tuple(iterable)</code>	<code>lst = [1, 2, 3]</code> <code>tuple(lst)</code>	(1, 2, 3)
11	reversed() 	Returns a reversed iterator of the tuple.	<code>reversed(tuple)</code>	<code>t = (1, 2, 3, 4, 5)</code> <code>tuple(reversed(t))</code>	(5, 4, 3, 2, 1)
12	hash() 	Returns the hash value of a tuple. Tuple must contain only hashable items.	<code>hash(tuple)</code>	<code>t = (1, 2, 3)</code> <code>hash(t)</code>	(unique hash value)
13	__add__() 	Concatenates two tuples using the <code>+</code> operator.	<code>tuple1 + tuple2</code>	<code>a = (1, 2, 3)</code> <code>b = (4, 5, 6)</code> <code>a + b</code>	(1, 2, 3, 4, 5, 6)
14	__mul__() 	Repeats a tuple a specified number of times using the <code>*</code> operator.	<code>tuple * n</code>	<code>a = (1, 2, 3)</code> <code>a * 2</code>	(1, 2, 3, 1, 2, 3)

★ KEY POINTS

- Tuples are immutable (cannot be changed).
- Methods that try to modify return a new object instead.
- Many "methods" are actually built-in functions that work with tuples.
- All elements in a tuple must be hashable for the tuple itself to be hashable.

TUPLE EXAMPLE

```
t = (1, 2, 3, 4, 2, 5)

Index:  0  1  2  3  4  5
        ↓ ↓ ↓ ↓ ↓ ↓
Value:  1  2  3  4  2  5
```

WHEN TO USE?

- ✓ Use tuples when you have a fixed collection of items that should not be changed.
- ✓ Useful for returning multiple values from a function.

💡 NOTE

- `index()` raises `ValueError` if value is not found.
- `sum()` works only for numeric tuples.
- `zip()` and `enumerate()` return iterators in Python 3.

EXAMPLES (QUICK REFERENCE)

```
1 t = (1, 2, 3, 4, 2, 5)
2 t.count(2)           # 2
3 t.index(3)           # 2
4 len(t)               # 6
5 max(t), min(t)       # (5, 1)
6 sorted(t)            # [1, 2, 2, 3, 4, 5]
7 sum((1, 2, 3, 4, 5)) # 15
8 any((True, False, True)) # True
9 all((True, False, True)) # False

10 list(zip((1,2,3), ('a', 'b', 'c')))# [(1, 'a'), (2, 'b'), (3, 'c')]
11 list(enumerate(t))           # [(0, 1), (1, 2), (2, 3), (3, 4), (4, 2), (5, 5)]
12 tuple([1,2,3])              # (1, 2, 3)
13 tuple(reversed(t))          # (5, 2, 4, 3, 2, 1)
14 hash((1,2,3))               # Unique hash value
15 (1,2,3) + (4,5,6)           # (1, 2, 3, 4, 5, 6)
16 (1,2,3) * 2                 # (1, 2, 3, 1, 2, 3)
```

★ Tuples are **simple**, **fast** and **reliable**. Use the right method for the right job! 🚀



DOCSTRINGS IN PYTHON

Docstrings are used to document Python modules, classes, functions and methods. It is a string literal that appears right after the definition of a module, function, class or method.

```
def add(a, b):  
    """Return the sum of two numbers."""  
    return a + b  
  
print(add.__doc__) # Return the sum of two numbers
```



1. WHAT IS A DOCSTRING?

- Docstrings are string literals used to explain the purpose of a module, function, class or method.
- Written inside triple quotes: `"""..."""` (or `'...''`).
- The first statement in the body becomes the docstring.
- Accessible using the `__doc__` attribute or the `help()` function.



TIP

Good docstrings make your code easier to understand and maintain.

2. WHERE TO USE DOCSTRINGS?

Place	Description
Module	At the top of a file (module level)
Class	At the beginning of a class definition
Function	At the beginning of a function definition
Method	At the beginning of a method definition

Example: Module Docstring

```
"""  
This is a module docstring.  
It describes what this module does.  
"""  
  
def example():  
    pass
```

3. ACCESSING DOCSTRINGS

Using `__doc__` attribute

```
def greet(name):  
    """Greet the person passed as parameter."""  
    print(f"Hello, {name}!")  
  
print(greet.__doc__) # Greet the person passed as parameter
```

Using `help()` function

```
def greet(name):  
    """Greet the person passed as parameter."""  
    print(f"Hello, {name}!")  
  
help(greet)
```



4. EXAMPLES

Function Docstring

```
def add(a, b):  
    """Return the sum of two numbers.  
  
    Parameters:  
        a (int): First number  
        b (int): Second number  
  
    Returns:  
        int: Sum of a and b  
    """  
    return a + b  
  
print(add.__doc__)
```

Method Docstring

```
class Calculator:  
    """A simple calculator class."""  
  
    def add(self, a, b):  
        """Add two numbers and return the result."""  
        return a + b  
  
print(Calculator.add.__doc__)
```

Class Docstring

```
class Person:  
    """Represents a person.  
  
    Attributes:  
        name (str): The name of the person  
        age (int): The age of the person  
    """  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
print(Person.__doc__)
```

Module Docstring

```
"""  
This module provides basic math operations.  
"""  
  
def multiply(a, b):  
    """Multiply two numbers and return the result."""  
    return a * b  
  
import math_operations  
print(math_operations.__doc__)
```

5. DOCSTRING CONVENTIONS (PEP 257)

- Use triple double quotes `"""` for all docstrings.
- The opening quotes should be on the same line as the definition.
- Use a one-line summary (first line) describing the object.
- Leave a blank line after the summary.
- Provide more detailed description if necessary.
- Use sections like Parameters, Returns, Raises, Examples etc.
- Indent docstring content to the same level as the code block.

Common Sections

Section	Description
Parameters	Describe the parameters (name, type, purpose)
Returns	Describe the return value (type, meaning)
Raises	Describe exceptions that may be raised
Examples	Provide example usage
Notes	Any additional notes

6. MULTI-LINE vs SINGLE-LINE DOCSTRINGS

Single-line Docstring

```
def square(x): """Return the square of x."""  
    return x * x
```

- ✓ Use single-line for short and simple docstrings.

Multi-line Docstring

```
def square(x):  
    """Return the square of a number.  
  
    Parameters:  
        x (int or float): The number to square  
  
    Returns:  
        int or float: The square of x  
    """  
    return x * x
```

- ✓ Use multi-line for detailed explanations.

7. DOCSTRINGS AND INTROSPECTION

List all functions in a module with their docstrings

```
import math  
  
for name in dir(math):  
    obj = getattr(math, name)  
    if callable(obj) and obj.__doc__:  
        print(f"{name}: {obj.__doc__}")
```

Get help for a function

```
def divide(a, b):  
    """Divide a by b and return the result."""  
    return a / b  
  
help(divide)
```

Check if an object has a docstring

```
def greet(name):  
    """Greet someone by name."""  
    pass  
print(greet.__doc__ is not None) # True
```

Get the first line (summary)

```
def greet(name):  
    """Greet someone by name.  
    This function prints a greeting message.  
    """  
    pass  
summary = greet.__doc__.splitlines()[0]  
print(summary) # Greet someone by name.
```

8. GOOD DOCSTRING EXAMPLE (GOOGLE STYLE)

```
def calculate_average(numbers):  
    """Calculate the average of a list of numbers.  
  
    Args:  
        numbers (list[int] | list[float]): A list of numbers.  
  
    Returns:  
        float: The average of the numbers.  
  
    Raises:  
        ValueError: If the list is empty.  
  
    Example:  
    >>> calculate_average([1, 2, 3])  
    2.0  
    """  
    if not numbers:  
        raise ValueError("List is empty")  
    return sum(numbers) / len(numbers)
```

8b. GOOD DOCSTRING EXAMPLE (NUMPY STYLE)

```
def calculate_average(numbers):  
    """  
    Calculate the average of a list of numbers.  
  
    Parameters  
    -----  
    numbers : list of int or float  
        A list of numbers.  
  
    Returns  
    -----  
        The average of the numbers.  
  
    Raises  
    -----  
    ValueError  
        If the list is empty.  
  
    Examples  
    -----  
    >>> calculate_average([1, 2, 3])  
    2.0  
    """  
    if not numbers:  
        raise ValueError("List is empty")  
    return sum(numbers) / len(numbers)
```

9. IMPORTANT NOTES

- ✓ The docstring is NOT a comment. It is an executable string. It can be accessed at runtime.
- ✓ Docstrings improve code readability and usability.
- ✓ IDE's and documentation tools use docstrings to display information.
- ✓ Follow a consistent style (PEP 257, Google, NumPy etc.).
- ✓ Keep docstrings up to date with your code.



TIP

Write docstrings for every module, class, function and method. Future you (and others) will thank you!



10. COMMON MISTAKES

- ✗ Forgetting to write docstrings.
- ✗ Placing docstring after other statements.
- ✗ Writing unclear or outdated docstrings.
- ✗ Not following a consistent style.



11. QUICK RECAP

- ✓ Use triple quotes `"""..."""`.
- ✓ First statement becomes the docstring.
- ✓ Access using `__doc__` or `help()`.
- ✓ Follow a good style and structure.
- ✓ Good docstrings = Better code!



12. WHEN TO USE DOCSTRINGS?

- ★ When creating modules, classes, functions or methods.
- ★ When your code will be used by others (or future you).
- ★ When writing libraries or APIs.
- ★ Always!





PYTHON SETS

Unordered. Unique. Powerful!

```
s = {1, 2, 3}
s.add(4)
s.remove(2)
print(s) # {1, 3, 4}
```



1. WHAT IS A SET?

- ✓ A set is a collection of unique items.
- ✓ Unordered (no index, no position).
- ✓ Mutable (can add/remove items).
- ✓ Defined using curly braces {} or set() function.



2. CREATING SETS

Using curly braces {}

```
s = {10, 20, 30}
colors = {"red", "green", "blue"}
```

Using set() constructor

```
s = set([1, 2, 3])
empty_set = set()
```



Note: {} creates an empty dict, not a set. Use set() for an empty set.

3. SET CHARACTERISTICS

- ◆ Items must be immutable (int, float, str, tuple, etc.).
- ◆ No duplicate values.
- ◆ Unordered: items have no index.
- ◆ Faster membership testing.

Example

```
s = {1, 2, 2, 3, 3, 4}
print(3)
# Output (order may vary)
# {1, 2, 3, 4}
```



4. BASIC OPERATIONS

Operation	Example	Result
Add	s.add(5)	Adds 5
Remove	s.remove(5)	Removes 5 (Error if not found)
Discard	s.discard(5)	Removes 5 (No error if not found)
Pop	s.pop()	Removes & returns an item
Clear	s.clear()	Removes all items
Delete	del s	Deletes the set

5. SET OPERATIONS (MATHEMATICAL)

Union ($\cup$)	s1 s2	Items in s1 or s2 (or both)
Intersection ($\cap$)	s1 & s2	Items in both s1 and s2
Difference ($-$)	s1 - s2	Items in s1 but not in s2
Symmetric Difference ($\oplus$)	s1 ^ s2	Items in either s1 or s2, but not both
Subset ($\subseteq$)	s1 <= s2	True if s1 is subset of s2
Superset ($\supseteq$)	s1 >= s2	True if s1 is superset of s2

6. EXAMPLES OF SET OPERATIONS

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print("a | b =", a | b) # {1, 2, 3, 4, 5, 6}
print("a & b =", a & b) # {3, 4}
print("a - b =", a - b) # {1, 2}
print("b - a =", b - a) # {5, 6}
print("a ^ b =", a ^ b) # {1, 2, 5, 6}
print("a <= b ?", a <= b) # False
print("a >= {1,2} ?", a >= {1, 2}) # True
```

7. MEMBERSHIP TESTING

Check if an item exists in a set.

```
s = {10, 20, 30}
print(20 in s) # True
print(40 not in s) # True
```



8. LOOPING THROUGH A SET

Loop through all items in a set.

```
s = {"apple", "banana", "cherry"}
for item in s:
    print(item)
```



9. SET COMPREHENSION

Create sets using a compact syntax.

```
squares = {x*x for x in range(1, 6)}
print(squares) # {1, 4, 9, 16, 25}
```

Expression for Item in Iterable

10. FROZEN SETS (IMMUTABLE SETS)

Frozen sets are immutable (cannot be changed) and can be used as dictionary keys.

```
fs = frozenset([1, 2, 3])
# fs.add(4) ❌ Error
# fs.remove(2) ❌ Error
print(fs) # frozenset({1, 2, 3})
```



11. SET METHODS

Method	Description
copy()	Returns a shallow copy of the set
update(iterable)	Adds all items from iterable
intersection_update(set)	Keeps only items common with set
difference_update(set)	Removes items present in set
symmetric_difference_update(set)	Keeps items in either set, but not both

12. QUICK RECAP

- ✓ Sets store unique items (no duplicates).
- ✓ Unordered and mutable (except frozenset).
- ✓ Use sets for fast membership testing.
- ✓ Powerful set operations make code concise.
- ✓ Useful in removing duplicates, comparisons, and data analysis.



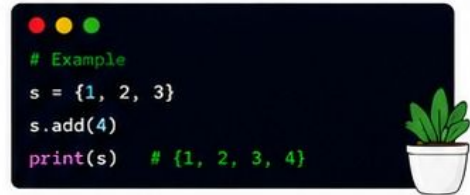
Sets help you store **unique** data and perform **powerful** operations!





PYTHON SET METHODS

Sets are unordered collections of unique elements.
Python provides a rich set of built-in methods to perform various operations on sets.



1. WHAT IS A SET?

- A set is an unordered collection of unique and immutable elements.
- Duplicates are not allowed.
- Elements must be hashable (immutable types).
- Defined using curly braces {} or the set() constructor.

```
s = {1, 2, 3, 2}
print(s)      # {1, 2, 3}
print(type(s)) # <class 'set'>
```

2. CREATING SETS

- Using curly braces {}

```
s = {1, 2, 3}
```

- Using set() constructor

```
s = set([1, 2, 3])
```

- Empty set

```
s = set() # Not: s = {} (this is an empty dict)
```

3. BASIC SET METHODS

Method	Description	Example	Result
add(x)	Adds an element x to the set	s = {1, 2}; s.add(3)	{1, 2, 3}
remove(x)	Removes element x (raises KeyError if not found)	s = {1, 2, 3}; s.remove(2)	{1, 3}
discard(x)	Removes element x if present (no error if not found)	s = {1, 2}; s.discard(3)	{1, 2}
pop()	Removes and returns an arbitrary element	s = {1, 2, 3}; s.pop()	Returns 1 or 2 or 3
clear()	Removes all elements	s = {1, 2, 3}; s.clear()	set()
copy()	Returns a shallow copy of the set	s = {1, 2}; t = s.copy()	t = {1, 2}

4. SET OPERATIONS (MATHEMATICAL OPERATIONS)

Operation	Method	Description	Example	Result
Union	s.union(t) or s t	Elements in s or or both	s = {1, 2, 3} t = {3, 4, 5} s t	{1, 2, 3, 4, 5}
Intersection	s.intersection(t) or s & t	Elements common to both s and t	s & t	{3}
Difference	s.difference(t) or s - t	Elements in s but not in t	s - t	{1, 2}
Symmetric Difference	s.symmetric_difference(t) or s ^ t	Elements in s or t but not in both	s ^ t	{1, 2, 4, 5}
Subset	s.issubset(t)	True if all elements of s are in t	{1, 2}.issubset({1, 2, 3})	True
Superest	s.issuperset(t)	True if all elements of t are in s	{1, 2, 3}.issuperset({1, 2})	True
Disjoint	s.isdisjoint(t)	True if s and t have no common elements	{1, 2}.isdisjoint({3, 4})	True

5. UPDATE METHODS

Method	Description	Example
update(t)	Adds all elements from t to the set	s = {1, 2} s.update({2, 3}) # {1, 2, 3}
intersection_update(t)	Updates set with intersection with t	s = {1, 2, 3} s.intersection_update({2, 3, 4}) # {2, 3}
difference_update(t)	Updates set with difference with t	s = {1, 2, 3} s.difference_update({2, 3}) # {1}
symmetric_difference_update(t)	Updates set with symmetric difference with t	s = {1, 2, 3} s.symmetric_difference_update({3, 4}) # {1, 2, 4}

6. FROZEN SET

- frozenset is an immutable version of set.
- Elements cannot be added or removed.
- Can be used as dictionary keys.

```
fs = frozenset([1, 2, 3])
print(fs)      # frozenset({1, 2, 3})
```

7. ITERATING OVER A SET

You can loop through set elements.

```
s = {'a', 'b', 'c'}
for item in s:
    print(item)
# Output order is not guaranteed
```

8. USEFUL BUILT-IN FUNCTIONS WITH SETS

Function	Description	Example	Result
len(s)	Number of elements	len({1, 2, 3})	3
min(s)	Smallest element	min({3, 1, 2})	1
max(s)	Largest element	max({3, 1, 2})	3
sum(s)	Sum of elements	sum({1, 2, 3})	6
any(s)	True if any element is True	any({0, False, 1})	True
all(s)	True if all elements are True	all({1, True})	True

9. EXAMPLES

Example 1: Basic Operations

```
s = {1, 2, 3}
s.add(4)
s.discard(2)
print(s)      # {1, 3, 4}
```

Example 2: Set Operations

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)   # {1, 2, 3, 4, 5}
print(a & b)   # {3}
print(a - b)   # {1, 2}
print(a ^ b)   # {1, 2, 4, 5}
```

Example 3: Update Methods

```
s = {1, 2}
s.update({2, 3})      # {1, 2, 3}
s.difference_update({2}) # {1, 3}
```

Example 4: Subset & Superset

```
s = {1, 2, 3}
t = {1, 2}
print(t.issubset(s))  # True
print(s.issuperset(t)) # True
```

10. QUICK SUMMARY

- ✓ Sets store unique, unordered elements.
- ✓ Use add(), remove(), discard(), pop(), clear(), copy().
- ✓ Use union, intersection, difference, symmetric difference for set operations.
- ✓ Use update methods to modify sets in-place.
- ✓ Use issubset(), issuperset(), isdisjoint() to test relationships.
- ✓ Use frozenset for immutable sets.



11. IMPORTANT NOTES

- ★ Sets are unordered: you cannot access elements by index.
- ★ Set elements must be immutable (e.g., int, str, tuple).
- ★ Set operations return new sets (except update methods).
- ★ pop() removes and returns an arbitrary element.
- ★ {} creates a dictionary, not a set. Use set() for an empty set.

Sets are great for membership testing and removing duplicates!



PYTHON DICTIONARY

A dictionary in Python is an unordered, mutable collection of key-value pairs. Keys must be unique and immutable.



1. CREATING DICTIONARIES

- Using curly braces {}

```
d = {"name": "Alice", "age": 25, "city": "New York"}
```

- Using dict() constructor

```
d = dict(name="Alice", age=25, city="New York")
```

- From sequence of key-value pairs

```
d = dict([("name", "Alice"), ("age", 25), ("city", "New York")])
```

- Empty dictionary

```
d = {} # or d = dict()
```

2. ACCESSING ITEMS

```
d = {"name": "Alice", "age": 25, "city": "New York"}
```

- Access by key

```
d["name"] # 'Alice'
```

- Using get() method (safe access)

```
d.get("age") # 25
d.get("country") # None (default)
```

- Access with default value

```
d.get("country", "Unknown") # 'Unknown'
```

3. ADDING / MODIFYING ITEMS

```
d = {"name": "Alice", "age": 25}
```

- Add new key-value pair

```
d["city"] = "New York"
```

- Modify existing value

```
d["age"] = 26
```

- Add multiple items using update()

```
d.update({"country": "USA", "city": "New York"})
```

4. DICTIONARY METHODS

Method	Description	Example	Output
keys()	Returns a view object of all keys	d.keys()	dict_keys([...])
values()	Returns a view object of all values	d.values()	dict_values([...])
items()	Returns a view object of key-value pairs	d.items()	dict_items([...])
get(key[, default])	Returns value for key or default if key not found	d.get("age", 0)	25
setdefault(key, default)	Returns value if key exists, else inserts key with default	d.setdefault("age", 0)	25
update(other)	Updates dictionary with key-value pairs from other	d.update({"age": 26})	Updated dict
pop(key[, default])	Removes key and returns its value	d.pop("age")	25
popitem()	Removes and returns the last inserted item (key, value)	d.popitem()	(key, value)
clear()	Removes all items	d.clear()	{}
copy()	Returns a shallow copy	d.copy()	New dict
fromkeys(seq[, value])	Creates new dict with keys from seq and value	dict.fromkeys(["a", "b"], 0)	{'a': 0, 'b': 0}

5. REMOVING ITEMS

```
d = {"name": "Alice", "age": 25, "city": "New York"}
```

- del statement

```
del d["age"] # Removes 'age'
```

- pop() method

```
d.pop("age") # Removes and returns value
```

- popitem() method

```
d.popitem() # Removes and returns last inserted item
```

- clear() method

```
d.clear() # Removes all items
```

6. DICTIONARY OPERATIONS

Operation	Example	Output
Length	len(d)	3
Membership (key)	"name" in d	True
Membership (value)	"Alice" in d.values()	True
Iterate keys	for k in d:	keys
Iterate values	for v in d.values():	values
Iterate items	for k, v in d.items():	key-value pairs

7. DICTIONARY COMPREHENSION

```
# Create a dictionary with squares of numbers
squares = {x: x*x for x in range(1, 6)}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

8. EXAMPLES

Example 1: Basic Usage

```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Alice
print(person.get("age")) # 25
person["city"] = "New York"
print(person)
# {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Example 2: Using Methods

```
person = {"name": "Alice", "age": 25}
person.update({"city": "New York"})
print(person.keys()) # dict_keys(['name', 'age', 'city'])
print(person.values()) # dict_values(['Alice', 25, 'New York'])
print(person.items()) # dict_items([('name', 'Alice'), ('age', 25), ('city', 'New York')])
```

Example 3: Looping Through Dictionary

```
person = {"name": "Alice", "age": 25, "city": "NY"}
for key in person:
    print(f"{key}: {person[key]}")
# name: Alice # age: 25 # city: NY
```

Example 4: Dictionary Comprehension

```
students = ["Ann", "Bob", "Charlie"]
scores = {student: i*10 for i, student in enumerate(students, 1)}
print(scores) # {'Ann': 10, 'Bob': 20, 'Charlie': 30}
```

9. IMPORTANT NOTES

- Keys must be unique and immutable (strings, numbers, tuples).
- Values can be of any mutable or immutable type.
- Dictionaries are unordered (in Python < 3.7). From Python 3.7+, insertion order is preserved.
- Dictionaries are mutable (can be changed after creation).

Immutable Types
(Valid as Keys)

- str (string)
- int (integer)
- float
- tuple (with immutable items)

10. NESTED DICTIONARY

```
students = {
    "Alice": {"age": 20, "grade": "A"},
    "Bob": {"age": 21, "grade": "B"}
}
print(students["Alice"]["grade"]) # A
```

11. MERGING DICTIONARIES (Python 3.9+)

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 20, "c": 3}
merged = d1 | d2 # {'a': 1, 'b': 20, 'c': 3}
merged.update = d1 | d2 # same as above
d1 |= d2 # Update d1 in-place
```

12. QUICK SUMMARY

- Create: {}, dict()
- Access: d[key], d.get(key)
- Add/Modify: d[key] = value, d.update()
- Remove: del d[key], d.pop(key), d.popitem(), d.clear()
- View: d.keys(), d.values(), d.items()
- Check: key in d, value in d.values()





PYTHON DICTIONARIES

Store data in key-value pairs.
Fast, flexible, and easy to use!

```
student = {  
    "name": "Rahul",  
    "age": 18,  
    "course": "Python"  
}  
print(student["name"]) # Rahul
```

DICTIONARY

{:}

1. WHAT IS A DICTIONARY?

A dictionary is an unordered collection of items stored in key-value pairs. Each key is unique and maps to a value. It is mutable (changeable).

Example:

```
student = {"name": "Rahul", "age": 18,  
           "course": "Python", "marks": 95}
```

Memory Trick

Think of a dictionary as a real dictionary. You look up a word (key) and get its meaning (value).



2. CREATING DICTIONARIES

Way	Example	Description
Empty dictionary	<code>d = {}</code>	Creates an empty dictionary
With data	<code>d = {"a": 1, "b": 2}</code>	Create with key-value pairs
Using dict()	<code>d = dict(name="Ravi", age=20)</code>	Using dict() constructor
From list of tuples	<code>d = dict([("a", 1), ("b", 2)])</code>	Convert list of tuples to dictionary

Example Code

```
student = {"name": "Rahul", "age": 18, "marks": 95}  
print(student)  
# {'name': 'Rahul', 'age': 18, 'marks': 95}
```

3. ACCESSING ITEMS

Access values using keys.

Method	Example	Result
Using key	<code>student["name"]</code>	'Rahul'
Using get()	<code>student.get("age")</code>	18
Using get() with default	<code>student.get("grade", "A")</code>	'A'
Check key	<code>"name" in student</code>	True
Check key	<code>"grade" in student</code>	False

Example Code

```
print(student["course"]) # Python  
print(student.get("marks")) # 95  
print(student.get("grade", "Not found")) # Not found
```

4. MODIFYING ITEMS

You can change, add or remove key-value pairs.

Operation	Example	Result
Update value	<code>student["age"] = 19</code>	Updates age
Add new item	<code>student["grade"] = "A"</code>	Adds new key-value
Update or add	<code>student["age"] = 20</code>	Updates if key exists, else adds new key

Example Code

```
student["age"] = 19  
student["grade"] = "A"  
student["name"] = "Rahul Sharma"  
print(student)  
# {'name': 'Rahul Sharma', 'age': 19,  
#   'course': 'Python', 'marks': 95, 'grade': 'A'}
```

5. REMOVING ITEMS

Method	Example	Description
del	<code>del student["marks"]</code>	Removes item by key
pop()	<code>student.pop("age")</code>	Removes item & returns value
popitem()	<code>student.popitem()</code>	Removes & returns last inserted item
clear()	<code>student.clear()</code>	Removes all items

Example Code

```
age = student.pop("age") # 19  
del student["course"]  
last_item = student.popitem() # ('grade', 'A')  
student.clear() # {}
```

6. DICTIONARY METHODS

Method	Description	Example
keys()	Returns all keys	<code>student.keys()</code>
values()	Returns all values	<code>student.values()</code>
items()	Returns key-value pairs	<code>student.items()</code>
copy()	Returns a shallow copy	<code>student.copy()</code>
update()	Updates with another dict	<code>student.update(d2)</code>
setdefault()	Returns value of key. If key not found, inserts key with default value	<code>student.setdefault("city", "Delhi")</code>

Example Code

```
print(student.keys()) # dict_keys(['name', 'age', 'grade'])  
print(student.values()) # dict_values(['Rahul Sharma', 19, 'A'])  
print(student.items()) # dict_items([('name', 'Rahul Sharma'), ...])
```

7. NESTED DICTIONARIES

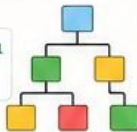
A dictionary can contain another dictionary.

Example:

```
students = {  
    1: {"name": "Rahul", "age": 18},  
    2: {"name": "Priya", "age": 17}  
}
```

Access:

```
print(students[1]["name"]) # Rahul  
print(students[2]["age"]) # 17
```



8. LOOPING IN DICTIONARIES

Loop through keys

```
for key in student:  
    print(key)  
# name age grade
```

Loop through values

```
for value in student.values():  
    print(value)  
# Rahul Sharma 19 A
```

Loop through key-value pairs

```
for key, value in student.items():  
    print(key, "->", value)  
# name -> Rahul Sharma, age -> 19, grade -> A
```

9. DICTIONARY COMPREHENSION (Python 3+)

Create dictionaries in a single line of code.

Example:

```
squares = {x: x*x for x in range(1, 6)}  
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

How it works?

```
for x in range(1, 6)  
    -> keys: 1, 2, 3, 4, 5  
x: x*x -> value = x*x  
Creates key-value pairs
```

10. COMMON MISTAKES

- ✗ Using list index instead of key.
`student[0]` # ✗ Error
- ✗ Duplicate keys (only last one is kept).
`d = {"a": 1, "a": 2}` # {'a': 2}
- ✗ Mutating dictionary while looping.
- ✗ Trying to access a key that doesn't exist.
`student["city"]` # ✗ KeyError
- ✗ Forgetting colons in key-value pairs.
`d = {"a": 1}` # ✗ SyntaxError

11. PRACTICE EXERCISES

- Create a dictionary with your name, age and city. Print it.
- Add a new key "email" and value to the dictionary.
- Update your age to a new value.
- Remove the city from the dictionary.
- Print all keys and all values using methods.
- Loop through the dictionary and print key and value.
- Create a dictionary where keys are numbers 1 to 5 and values are their cubes.



12. QUICK RECAP

- ✓ Dictionary stores data in key-value pairs.
- ✓ Keys must be unique and immutable.
- ✓ Values can be of any data type.
- ✓ Use keys to access values.
- ✓ Dictionaries are mutable and unordered.
- ✓ Many built-in methods make it powerful.
- ✓ Use dictionaries when you have related data with meaningful keys!



13. REAL-WORLD USE CASES



Student Records
Store student details like name, age, marks, grade.



Phone Book
Map names (keys) to phone numbers (values).



Settings
Store user preferences like theme, language, notifications.



Count Frequency
Count how many times each item occurs in data.



APIs & JSON
Data from APIs is often in dictionary (JSON) format.

When to Use Dictionary?

- ✓ When you have data with unique keys.
- ✓ When fast lookup is important.
- ✓ When data is related and meaningful.



★ Keys are like addresses, values are like the data stored at those addresses!





PYTHON DICTIONARY METHODS

Dictionaries are mutable collections of key-value pairs.

Python provides several built-in methods to work with dictionaries efficiently.

```
my_dict = {"name": "Tanish",
           "age": 33,
           "profession": "Software Engineer"}
print(my_dict)
# {'name': 'Tanish', 'age': 33,
#   'profession': 'Software Engineer'}
```



1. dict() - Constructor

Build dictionaries directly from sequences of key-value pairs.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
my_dict_with_constructor = dict(name="Alice",
                                age=23, profession="Software Analyst")
print(my_dict)
print(my_dict_with_constructor)
```

Output:

```
{'name': 'Tanish', 'age': 33,
 'profession': 'Software Engineer'}
{'name': 'Alice', 'age': 23,
 'profession': 'Software Analyst'}
```

2. keys()

Returns a view object of all the keys.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
list_of_keys = my_dict.keys()
print(list_of_keys)
print(type(list_of_keys))
```

Output:

```
dict_keys(['name', 'age', 'profession'])
<class 'dict_keys'>
```

💡 dict_keys is a view object. Changes in the dictionary will reflect in the view.

3. values()

Returns a view object of all the values.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
list_of_values = my_dict.values()
print(list_of_values)
print(type(list_of_values))
```

Output:

```
dict_values(['Tanish', 33, 'Software Engineer'])
<class 'dict_values'>
```

💡 dict_values is a view object.

4. items()

Returns a view object of key-value pairs.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
list_of_items = my_dict.items()
for key, value in list_of_items:
    print(f"{key}:{value}")
```

Output:

```
name:Tanish
age:33
profession:Software Engineer
```

💡 items() returns key-value pairs as tuples.

5. update()

Updates dictionary with key-value pairs from another dictionary or iterable.

```
my_dict = {"name": "Tanish", "age": 33}
new_data = {"profession": "Software Engineer",
            "age": 34}
my_dict.update(new_data)
print(my_dict)
```

Output:

```
{'name': 'Tanish', 'age': 34,
 'profession': 'Software Engineer'}
```

💡 If the same key exists, its value is overwritten.

6. clear()

Removes all items from the dictionary.

```
my_dict = {"name": "Tanish", "age": 33}
my_dict.clear()
print(my_dict)
```

Output:

```
{}
```

💡 The dictionary becomes empty.

7. pop(key)

Removes the specified key and returns its value.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
value = my_dict.pop("profession")
print(value)
print(my_dict)
```

Output:

```
Software Engineer
{'name': 'Tanish', 'age': 33}
```

💡 Raises KeyError if the key is not found.

8. popitem()

Removes and returns the last inserted key-value pair as a tuple.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Software Engineer"}
last_item = my_dict.popitem()
print(last_item)
print(my_dict)
```

Output:

```
('profession', 'Software Engineer')
{'name': 'Tanish', 'age': 33}
```

💡 Dictionary order is preserved (in Python 3.7+).

9. del

Deletes a key or the entire dictionary.

```
my_dict = {"name": "Tanish", "age": 33}
del my_dict["age"]
print(my_dict)
del my_dict
# print(my_dict) # NameError
```

Output:

```
{'name': 'Tanish'}
```

💡 del can delete a key or the whole dictionary.

10. get(key)

Returns the value for the specified key.
Returns None if key is not found (or a default value).

```
my_dict = {"name": "Tanish", "age": 33}
print(my_dict.get("name"))
print(my_dict.get("profession"))
print(my_dict.get("profession", "Not Found"))
```

Output:

```
Tanish
None
Not Found
```

💡 Safe way to access values without KeyError.

11. setdefault(key, default)

Returns the value of the key if it exists.
If not, inserts the key with the default value.

```
my_dict = {"name": "Tanish", "age": 33}
my_dict.setdefault("profession", "Python Engineer")
print(my_dict)
my_dict.setdefault("part_time", "NO")
print(my_dict)
```

Output:

```
{'name': 'Tanish', 'age': 33,
 'profession': 'Python Engineer'}
{'name': 'Tanish', 'age': 33,
 'profession': 'Python Engineer', 'part_time': 'NO'}
```

💡 If key exists, default value is ignored.

12. copy()

Returns a shallow copy of the dictionary.

```
my_dict = {"name": "Tanish", "age": 33,
           "profession": "Python Engineer"}
cloned = my_dict.copy()
cloned.popitem()
print("Cloned:", cloned)
print("Original:", my_dict)
```

Output:

```
Cloned: {'name': 'Tanish', 'age': 33}
Original: {'name': 'Tanish', 'age': 33,
 'profession': 'Python Engineer'}
```

💡 Modifying the copy does not affect the original.

QUICK SUMMARY TABLE

Method	Description	Example	Result
dict()	Constructor to create a dictionary	dict(name="Alice", age=23)	{'name': 'Alice', 'age': 23}
keys()	Returns all keys	d.keys()	dict_keys([...])
values()	Returns all values	d.values()	dict_values([...])
items()	Returns key-value pairs	d.items()	dict_items([(...), ...])
update()	Updates dictionary with new key-value pairs	d.update(data)	Updated dictionary
clear()	Removes all items	d.clear()	{}
pop(key)	Removes key and returns its value	d.pop('name')	'Tanish'
popitem()	Removes and returns last key-value pair	d.popitem()	('key', 'value')
del	Deletes a key or entire dictionary	del d['key'] / del d	Key removed / dict deleted
get(key)	Returns value for key (safe)	d.get('name', 'NA')	Value or default
setdefault(key, default)	Returns value if key exists, else sets default	d.setdefault('k:k', 0)	Value or inserts default
copy()	Returns shallow copy	d.copy()	New dictionary copy



TIPS

- ✓ Dictionary keys must be unique and immutable (strings, numbers, tuples).
- ✓ Values can be of any mutable or immutable type.
- ✓ Use get() or setdefault() to avoid KeyError.
- ✓ Dictionaries maintain insertion order (Python 3.7+).
- ✓ Use copy() when you want to modify a dictionary without affecting the original.



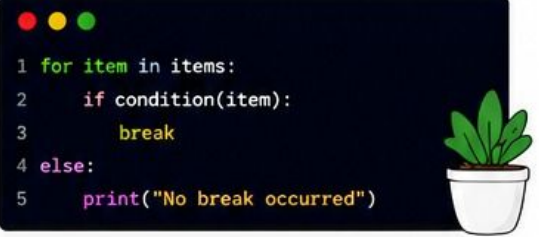


Python Loop ... else

Python allows an `else` statement to be attached to `for` and `while` loops.

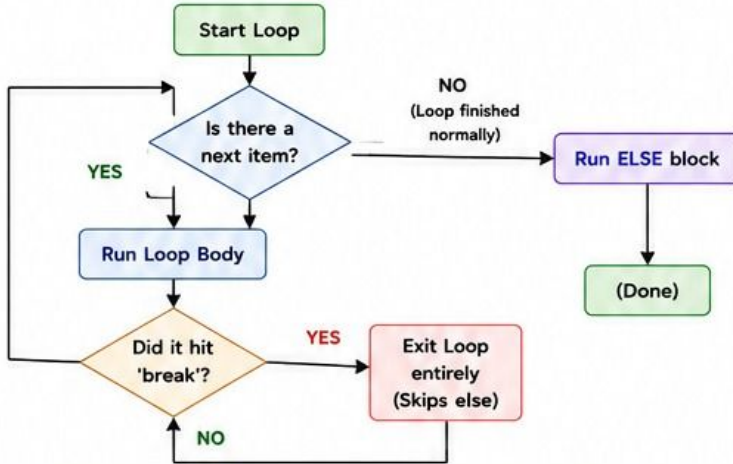


The `else` block executes only if the loop completes normally, without hitting a `break` statement.
Think of it as an "if the loop finished all its work" block.



HOW IT WORKS VISUALLY

Here is the basic logic flow of a loop with an `else` statement:



Key Points

- ✓ If the loop runs to completion (condition becomes False or items are exhausted), the `else` block runs.
- ✓ If a `break` statement is executed, the `else` block is skipped.
- ✓ `continue` does not affect `else`.
- ✓ The `else` belongs to the loop, not the `if`.

1. The `for...else` Loop

In a `for` loop, the `else` block runs after the loop has iterated through the entire sequence. If the loop is terminated early by a `break`, the `else` block is completely skipped.

Example: Searching a list

```

1 numbers = [2, 4, 6, 8, 10]
2 search_for = 7
3
4 for num in numbers:
5     if num == search_for:
6         print("Found it!")
7         break # This exits the loop and SKIPS the else block
8 else:
9     # This runs ONLY if the loop finishes without hitting the break
10    print("Item not found in the list.")
  
```

Output:
Item not found in the list.

💡 If `search_for` was 6, the loop would print "Found it!", hit the `break`, and the `else` block would never run.

2. The `while...else` Loop

In a `while` loop, the `else` block runs when the loop's condition becomes False. Just like the `for` loop, if you exit the `while` loop using `break`, the `else` block is ignored.

```

1 count = 0
2 while count < 3:
3     print(f"Count is {count}")
4     count += 1
5 else:
6     print("Loop finished successfully because count is no longer < 3")
  
```

Output:
Count is 0
Count is 1
Count is 2
Loop finished successfully because count is no longer < 3

Why did Python design it this way? (The "Flag" Alternative)

Before you knew about `loop...else`, you probably wrote code using a "flag" variable (a boolean tracker) to see if something happened inside a loop.

Look at how much cleaner the `else` method is compared to the traditional flag method:

The Old Way (Using a Flag)

```

1 found = False
2 for item in items:
3     if condition(item):
4         found = True
5         break
6
7 if not found:
8     print("Not found")
  
```



Needs extra variable, more lines, and extra checks.

The Pythonic Way (Using else)

```

1 for item in items:
2     if condition(item):
3         break
4 else:
5     print("Not found")
  
```



Cleaner, shorter, and more Pythonic!



Mental Trick

If the name `else` confuses you (and it confuses many, including core Python developers who have admitted `nobreak` might have been a better name), try reading it as if no `break` in your head.

“Loop through these items... if no `break` occurred, do this next.”

More Examples

Example 1: Find a prime number

```

1 n = 29
2 for i in range(2, n):
3     if n % i == 0:
4         print(f"{n} is not prime")
5         break
6 else:
7     print(f"{n} is prime")
  
```

Output:
29 is prime

Example 2: Read file until a keyword is found

```

1 keyword = "END"
2 with open("data.txt") as f:
3     for line in f:
4         if keyword in line:
5             print("Keyword found!")
6             break
7 else:
8     print("Keyword not found in file.")
  
```

Output (if not found):
Keyword not found in file.



ERROR HANDLING IN PYTHON

Error handling in Python is managed using exceptions. An exception is an error that happens during the execution of a program (runtime), which disrupts the normal flow of instructions. Python uses the `try...except` block to catch and handle these exceptions so your program doesn't crash.

```
try:
    risky_operation()
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    print("Cleanup done.")
```



1. THE CORE STRUCTURE (try, except, else, finally)

A complete error-handling block can contain up to four parts:

- try:** • The block of code where you expect an error might happen.
- except:** • The block of code that runs if an error occurs in the try block.
- else:** • The block of code that runs only if no errors occurred in the try block.
- finally:** • The block that always runs, regardless of whether an error occurred or not (often used for cleanup, like closing files).

Full Example:

```
1 try:
2     # Try to perform an action
3     numerator = 10
4     denominator = int(input("Enter a denominator: "))
5     result = numerator / denominator
6
7 except ZeroDivisionError:
8     # Runs if user enters 0
9     print("Error: You cannot divide by zero!")
10
11 except ValueError:
12     # Runs if user enters text instead of a number
13     print("Error: Please enter a valid integer.")
14
15 else:
16     # Runs only if division was successful
17     print(f"Success! The result is {result}")
18
19 finally:
20     # Always runs
21     print("Execution complete. Cleaning up resources.")
```

Example Runs:

- Input 0 → Error: You cannot divide by zero!
- Input "abc" → Error: Please enter a valid integer.
- Input 2 → Success! The result is 5.0
Execution complete. Cleaning up resources.

3. RAISING EXCEPTIONS (raise)

Sometimes you want to intentionally trigger an error if certain conditions are met in your code. You do this using the `raise` keyword.

```
1 def set_age(age):
2     if age < 0:
3         raise ValueError("Age cannot be negative!")
4     print(f"Age set to {age}")
5
6 try:
7     set_age(-5)
8 except ValueError as e:
9     print(f"Validation failed: {e}")
```

Output:
Validation failed: Age cannot be negative!

4. CUSTOM EXCEPTIONS

If Python's built-in errors don't precisely describe your problem, you can create your own by creating a class that inherits from the built-in `Exception` class.

```
1 class BalanceInsufficientError(Exception):
2     """Exception raised when a withdrawal amount
3     exceeds the balance."""
4     pass
5
6 def withdraw_money(balance, amount):
7     if amount > balance:
8         raise BalanceInsufficientError(
9             f"You tried to withdraw ${amount} but only
10             have ${balance}."
11         )
12     return balance - amount
13
14 # Using the custom exception
15 try:
16     withdraw_money(50, 100)
17 except BalanceInsufficientError as e:
18     print(f"Transaction Denied: {e}")
```

Output:
Transaction Denied: You tried to withdraw \$100 but only have \$50.

2. CATCHING SPECIFIC EXCEPTIONS

It is a best practice in Python to catch specific exceptions rather than using a blanket, empty `except:` block. Catching everything can hide bugs you didn't anticipate.

Common Built-in Exceptions:

Exception Name	When It Occurs
<code>ZeroDivisionError</code>	Division or modulo by zero.
<code>ValueError</code>	A function receives an argument of the right type but inappropriate value (e.g., <code>int("apple")</code>).
<code>TypeError</code>	An operation is applied to an object of inappropriate type (e.g., <code>'len' + 5</code>).
<code>FileNotFoundError</code>	Trying to open a file that doesn't exist.
<code>IndexError</code>	Trying to access a sequence index that is out of range.
<code>KeyError</code>	Looking up a dictionary key that doesn't exist.

Accessing the Error Message

You can use the `as` keyword to inspect the exact error message generated by Python:

```
1 try:
2     my_list = [1, 2, 3]
3     print(my_list[5])
4 except IndexError as e:
5     print(f"Caught an error: {e}")
```

Output:
Caught an error: list index out of range

BEST PRACTICES SUMMARY



1. Be Specific

Avoid bare `except:` statements.
Target the exact error you expect.



2. Keep try Blocks Small

Only place lines of code inside the try block that are actually capable of throwing the error you want to handle.



3. Don't Suppress Errors Silently

Avoid using `except: pass` unless you have a highly deliberate reason to ignore the error completely.



4. Log Errors (When Appropriate)

In real-world applications, log the error (e.g., using the logging module) so you can debug issues later.



5. Use finally for Cleanup

Use `finally` to release resources (close files, database connections, etc.) regardless of whether an error occurred.

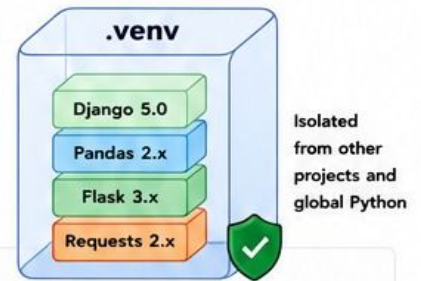


Pro Tip: Good error handling doesn't just prevent crashes—it makes your program more reliable, user-friendly, and easier to debug.



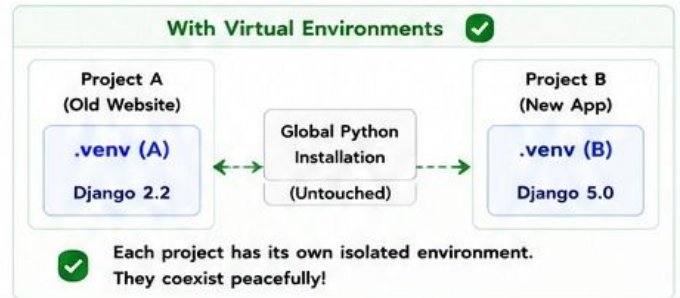
VIRTUAL ENVIRONMENT IN PYTHON

A Virtual Environment is an **isolated, self-contained** digital box for a specific Python project. You can install libraries and dependencies inside this box **without affecting** other projects or your system's global Python installation.



WHY DO YOU NEED IT? (THE DEPENDENCY NIGHTMARE)

Imagine two projects on the same machine that need different versions of the same library.



HOW TO USE VIRTUAL ENVIRONMENTS (THE 4-STEP PROCESS)

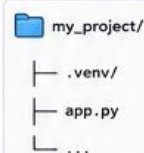
Python comes with a built-in tool called **venv** to manage this. Open your terminal or command prompt, navigate to your project folder, and follow these steps:

1 CREATE THE ENVIRONMENT

Run this command to create a new virtual environment. We usually name the folder `.venv` or `env`:

```
python -m venv .venv
```

This creates a folder named `.venv` in your project directory containing a copy of the Python executable and a clean playground for packages.



2 ACTIVATE THE ENVIRONMENT

Before you install packages, you must "step inside" the box by activating it. The command depends on your OS:

Windows (Command Prompt)	<code>.venv\Scripts\activate.bat</code>
Windows (PowerShell)	<code>.venv\Scripts\Activate.ps1</code>
macOS / Linux	<code>source .venv/bin/activate</code>



How do you know it worked?

Your terminal prompt will change to show the environment name in parentheses, like this: `(.venv) user@computer:~$`

3 INSTALL YOUR PACKAGES

Now, any package you install using **pip** will be locked inside this project folder only.

```
pip install requests
```



Global Python (Untouched)



4 DEACTIVATE (WHEN YOU'RE DONE)

When you are finished working on this project and want to return to your normal global terminal, simply type:

```
deactivate
```

```
(.venv) user@computer:~/my_project$ deactivate
```

```
user@computer:~$
```

SHARE YOUR ENVIRONMENT: requirements.txt

You should never upload your `.venv` folder to GitHub (it can be massive and doesn't transfer well to other computers). Instead, you create a text file listing all your project's dependencies.

To save your current list of packages:

```
pip freeze > requirements.txt
```

requirements.txt

```
requests==2.31.0
numpy==1.26.4
pandas==2.2.2
flask==3.0.3
...
```



Upload these files to GitHub

- ✓ app.py
- ✓ requirements.txt
- ✓ README.md
- ✓ ...



When someone else downloads your project, they can recreate your exact environment by running:

```
pip install -r requirements.txt
```

```
$ pip install -r requirements.txt
Collecting requests==2.31.0
Collecting numpy==1.26.4
Collecting pandas==2.2.2
Collecting flask==3.0.3
...
Successfully installed!
```

BEST PRACTICES



Keep your `.venv` folder out of version control. Add it to `.gitignore`



Create a new virtual environment for each project.



Always use `requirements.txt` to share dependencies.



Deactivate when you're done working on a project.



PYTHON MODULES

Organize. Reuse. Simplify!

```
import math
print(math.sqrt(16)) # 4.0
print(math.pi)      # 3.14159...
```



1. WHAT IS A MODULE?

A module is a Python file containing functions, classes, or variables. It helps in code organization and reusability.



Python has many built-in modules and you can also create your own!

2. IMPORTING MODULES

Use the import statement to use a module.

```
import math      # import entire module
import random    # another module
import datetime  # built-in module
```

Using a module:

```
print(math.sqrt(25))      # 5.0
print(random.randint(1, 10)) # random int
```



Import once, use many times!

3. IMPORTING SPECIFIC ITEMS

Import only what you need from a module.

```
from math import pi, pow
from random import choice
```

Using them directly:

```
print(pi)          # 3.14159...
print(pow(2, 3))   # 8
print(choice(["a", "b", "c"])) # random
```



Keeps your code clean and efficient.

4. ALIASING MODULES

Use an alias (short name) for a module.

```
import numpy as np
import pandas as pd

arr = np.array([1, 2, 3])
df = pd.DataFrame({"A": [1, 2, 3]})
```



Common aliases: np (NumPy), pd (Pandas), plt (Matplotlib).

5. THE dir() FUNCTION

See what a module contains using dir().

```
import math
print(dir(math))

# Output: list of all attributes and functions
# in the math module
```



Useful to explore modules!

6. SOME USEFUL BUILT-IN MODULES

Module	Use
math	Mathematical functions
random	Generate random numbers
datetime	Work with dates and times
os	Interact with operating system
sys	System-specific parameters
json	Work with JSON data
re	Regular expressions



7. CREATING YOUR OWN MODULE

- Create a file (mymodule.py)
- Write functions, classes, or variables
- Import and use it in other files

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159
```



8. USING YOUR MODULE

Import and use your custom module.

```
# main.py
import mymodule
print(mymodule.greet("Python")) # Hello, Python!
print(mymodule.PI)              # 3.14159
```

9. THE __name__ VARIABLE

Every Python file has a special variable `__name__`. It helps run code only when the file is executed directly.

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

if __name__ == "__main__":
    print("This runs only when executed directly.")
```



10. PACKAGE (COLLECTION OF MODULES)

A package is a folder containing modules. It must have an `__init__.py` file.

```
mypackage/
├── __init__.py
├── module1.py
└── module2.py
```

Import from package:

```
from mypackage import module1
module1.my_function()
```



11. RELOADING A MODULE

During development, reload a module to get the latest changes without restarting Python.

```
import importlib
import mymodule
importlib.reload(mymodule) # reloads module
```



Useful in interactive sessions.

12. QUICK RECAP

- ✓ Modules help organize and reuse code.
- ✓ Use `import` to access modules.
- ✓ Import only what you need.
- ✓ Create your own modules for better structure.
- ✓ Use `__name__ == "__main__"` for safe execution.
- ✓ Explore powerful built-in modules.



Modules make your code **modular**, **reusable**, and **maintainable**!





HOW import WORKS IN PYTHON

The import statement lets Python reuse code from other files (modules and packages).
Here's what actually happens behind the scenes.

```
import math
print(math.sqrt(16)) # 4.0
```

1 THE BIG PICTURE

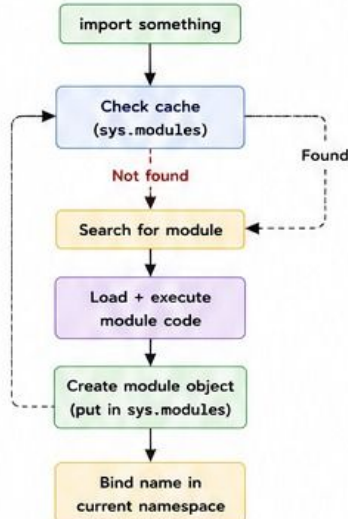
When you write an import statement, Python does roughly this:

- 1 Check if the module is already loaded.
- 2 If yes → return it from cache.
- 3 If not → find the module (search).
- 4 Load and execute the module's code.
- 5 Create a module object and bind it to a name in your program.



Key Point

A module's top-level code runs only once — the first time it is imported. After that, Python reuses the existing module object from memory.



2 WHERE DOES PYTHON SEARCH?

Python searches for modules in a list of locations called `sys.path` (the import search path). It looks in order:

- The directory of the script that's being run
- Directories listed in the `PYTHONPATH` environment variable
- Standard library locations
- Site-packages (where third-party packages are installed)

You can see the search path:

```
import sys
from pprint import pprint
pprint(sys.path)
```



The first matching module name found in this path is used.

3 COMMON WAYS TO IMPORT

1. Import a module

Import the whole module and use it with dot notation.

```
import math
print(math.pi)
print(math.sqrt(25))
```

Good when the module name adds clarity.

2. Import specific names

Import specific functions, classes, or variables.

```
from math import pi, sqrt
print(pi)
print(sqrt(25))
```

Good for using a few things from a module.

3. Import with alias

Give a shorter name (alias) to a module or name.

```
import numpy as np
arr = np.array([1, 2, 3])
```

Common for long module names.

4. Import all (not recommended)

Import everything from a module into the current namespace.

```
from math import *
print(pi)
print(sqrt(25))
```



Avoid this! Can cause name collisions and confusion.

4 EXAMPLE: WHAT HAPPENS DURING IMPORT

Suppose we have a file called `my_module.py`

```
1 print("Top level code in my_module is running")
2
3 value = 42
4
5 def hello():
6     print("Hello from my_module!")
```

`main.py`

```
1 print("Before import")
2 import my_module
3 print("After import")
4
5 print(my_module.value)
6 my_module.hello()
```

Output:

```
Before import
Top level code in my_module is running  <- runs during import
After import
42
Hello from my_module!
```

What happened?

- 1 Python didn't find `my_module` in cache.
- 2 It searched `sys.path` and found `my_module.py`.
- 3 It executed the top-level code in `my_module.py` (so the `print ran!`).
- 4 It created a module object, stored it in `sys.modules` under the name `'my_module'`.
- 5 It bound the name `my_module` in `main.py` to that module object.



If you import `my_module` again, step 1 finds it in `sys.modules`, so Python skips re-executing the code.

5 PACKAGES (DIRECTORIES WITH `__init__.py`)

A package is a directory containing Python modules, with an optional `__init__.py` file.

```
mypackage/
├── __init__.py
├── utils.py
└── core.py
```

```
import mypackage.utils  # import the module
from mypackage import utils  # same thing
from mypackage.utils import some_function
```



`__init__.py` can be empty, but it tells Python: "This directory is a package." It can also run initialization code for the package.

6 THE CACHE: `sys.modules`

Imported modules are stored in `sys.modules` (a dictionary).

```
import sys, math
print('math' in sys.modules)  # True
print(type(sys.modules))     # <class 'dict'>
```

This prevents Python from loading the same module multiple times and makes imports fast.

7 IMPORTANT TIPS

- ✓ Use absolute imports: `from package.module import name`
- ✓ Avoid circular imports (module A imports B and B imports A).
- ✓ Keep imports at the top of your file (PEP 8 recommendation).
- ✓ Use virtual environments to manage third-party packages.



import finds → loads → executes (once) → caches → binds.
After that, it reuses the cached module object.

import is not just copying code—
it's loading a module object and reusing it!





Python: `__name__ == "__main__"`

The `__name__ == "__main__"` idiom lets you run code only when a Python file is executed directly, not when it is imported as a module in another file.

```
# Run this file directly
python my_script.py  ✓ runs main code

# Import from another file
import my_script      ✗ main code does not run
```

1. THE IDEA

Every Python file has a special built-in variable named `__name__`. Its value depends on how the file is being used:

How the file is used	Value of <code>__name__</code>
Run directly (e.g., <code>python my_file.py</code>)	<code>"__main__"</code>
Imported by another file (e.g., <code>import my_file</code>)	<code>"my_file"</code> (the module's name)

i So, the condition `__name__ == "__main__"` is True only when the file is run directly.

2. HOW IT WORKS (EXAMPLE)

my_module.py

```
1 def greet():
2     print("Hello from greet()")
3
4 def add(a, b):
5     return a + b
6
7 if __name__ == "__main__":
8     print("This code runs only when the
9         file is executed directly.")
10    greet()
11    print("2 + 3 =", add(2, 3))
```

Run directly

\$ `python my_module.py`

This code runs only when the file is executed directly.
Hello from greet()
2 + 3 = 5

Importing from another file

```
# another_file.py
import my_module

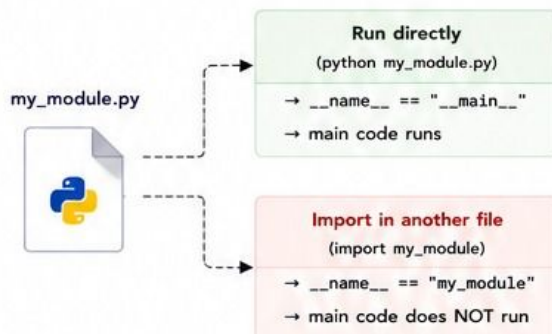
my_module.greet()
print(my_module.add(10, 5))
```

Output:
Hello from greet()
15

3. WHY IS IT USEFUL?

It allows a Python file to act both as:

1. A script (can be run directly)
2. A module (can be imported and reused)
...without the test or demo code running on import.



4. A PRACTICAL EXAMPLE

math_utils.py

```
1 def square(x):
2     return x * x
3
4 def cube(x):
5     return x * x * x
6
7 if __name__ == "__main__":
8     # Simple tests / demo
9     print("square(4) =", square(4))
10    print("cube(3) =", cube(3))
```

Run directly

\$ `python math_utils.py`

square(4) = 16
cube(3) = 27

Import and use in another file

```
# app.py
from math_utils import square
print("5 squared is", square(5))
```

Output:
5 squared is 25

💡 Bottom line: The test/demo code (lines 7–10) does not run when imported.

5. REAL-WORLD USE CASES



Command-line scripts with reusable functions

Put CLI code under
`if __name__ == "__main__":`
so functions can be imported elsewhere.



Web apps / Libraries

Keep `app.run(...)` or test code inside the main block so it doesn't start on import.



Testing / Demos

Run quick tests or demos only when the file is executed directly.



Modular design

Encourages clean separation between reusable code and execution code.

6. COMMON PATTERN (TEMPLATE)

```
1 """
2 my_module.py - description of the module
3 """
4
5 # Imports
6
7 # Functions / Classes
8
9 def main():
10     # Code that should run when executed directly
11     pass
12
13 if __name__ == "__main__":
14     main()
```

- ✓ Place the program's entry point in a `main()` function.
- ✓ Call it only if the file is run directly.
- ✓ Keeps your code clean, import-safe, and reusable.

7. KEY TAKEAWAY

```
if __name__ == "__main__":
```

- ✓ Runs only when the file is executed directly.
- ✓ Does not run when the file is imported.
- ✓ Essential for writing reusable, import-safe Python modules.
- ✓ A small line that makes your code much more powerful!



Think of it as: "Only do this when I am the main program, not when I am just a helper."



Python os Module

The **os** module provides a way to interact with the operating system.

```
import os
print(os.getcwd()) # current working directory
print(os.name)     # "nt" (Windows) or "posix" (Linux/Mac)
```

Why use os?

- ✓ Interact with files & directories
- ✓ Work with paths
- ✓ Run system commands
- ✓ Make cross-platform programs



Important Constants

os.name → 'nt' (Windows) / 'posix' (Unix, Linux, Mac)
os.sep → path separator ('\\' on Windows, '/' on Unix)
os.getcwd() → current working directory
os.environ → environment variables (dict)



Basic Usage

```
import os
print(os.getcwd())
print(os.listdir())
```



1. Commonly Used Functions

Function	What it does	Example
os.getcwd()	Get current working directory	<code>os.getcwd()</code>
os.chdir(path)	Change current directory	<code>os.chdir('my_folder')</code>
os.listdir(path)	List files and folders in a directory	<code>os.listdir('.')</code>
os.mkdir(path)	Create a new directory	<code>os.mkdir('new_dir')</code>
os.makedirs(path)	Create nested directories	<code>os.makedirs('a/b/c')</code>
os.rmdir(path)	Remove an empty directory	<code>os.rmdir('empty_dir')</code>
os.remove(path)	Remove a file	<code>os.remove('file.txt')</code>
os.rename(src, dst)	Rename or move a file/directory	<code>os.rename('old.txt', 'new.txt')</code>
os.path.exists(path)	Check if path exists	<code>os.path.exists('file.txt')</code>
os.path.isfile(path)	Check if path is file	<code>os.path.isfile('file.txt')</code>
os.path.isdir(path)	Check if path is dir	<code>os.path.isdir('my_dir')</code>
os.path.join(a,b,...)	Join paths	<code>os.path.join('a','b','c.txt')</code>
os.walk(path)	Walk through directory tree	<code>os.walk('my_dir')</code>
os.environ.get(key)	Get environment variable	<code>os.environ.get('PATH')</code>
os.system(cmd)	Run system command	<code>os.system('dir')</code>
os.popen(cmd)	Run command and get output	<code>os.popen('ipconfig').read()</code>

2. Examples

Get current directory	<pre>import os print(os.getcwd())</pre>	Output (Example) C:\Users\tanish\Projects
List files & folders	<pre>print(os.listdir('.'))</pre>	['file1.txt', 'data', 'main.py']
Create a directory	<pre>os.mkdir('demo')</pre>	(Directory 'demo' created)
Remove a file	<pre>os.remove('temp.txt')</pre>	(File 'temp.txt' deleted)
Rename a file	<pre>os.rename('old.txt', 'new.txt')</pre>	(File renamed)
Check if path exists	<pre>print(os.path.exists('main.py'))</pre>	True
Join paths (cross-platform)	<pre>print(os.path.join('data', 'report.csv'))</pre>	data\report.csv (Win) data/report.csv (Linux/Mac)
Walk directory tree	<pre>for root, dirs, files in os.walk('.'): print(root, dirs, files)</pre>	['data', 'images'] ['main.py'] ./data [] ['a.csv'] ./images ['2024'] []

3. Working with Environment Variables

```
import os
print(os.environ.get('USERNAME'))
print(os.environ['PATH'])
```

Tanish
C:\Windows\System32;C:\Python312;...

4. Run System Commands

os.system()

```
os.system('ping google.com')
os.system('cls') # clear screen
```

os.popen() (read output)

```
output = os.popen('ipconfig')
print(output.read())
```

5. Practical Mini Project: File Organizer

Organize files in a folder by extension.



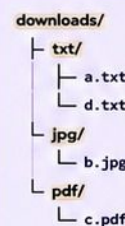
```
import os, shutil
folder = 'downloads'
files = os.listdir(folder)

for file in files:
    if os.path.isfile(os.path.join(folder, file)):
        ext = file.split('.')[-1]
        os.makedirs(os.path.join(folder, ext), exist_ok=True)
        shutil.move(os.path.join(folder, file),
                    os.path.join(folder, ext, file))

print("Files organized!")
```



After running:



6. Quick Reference

Task	Function
Get current dir	os.getcwd()
List files	os.listdir()
Create dir	os.mkdir(), os.makedirs()
Delete file/dir	os.remove(), os.rmdir()
Rename/Move	os.rename()
Check path	os.path.exists()
Join paths	os.path.join()
Environment vars	os.environ
Run commands	os.system(), os.popen()



PYTHON FILE HANDLING

Work with **Files** Easily! 🚀

```
with open("data.txt", "r") as f:
    content = f.read()
print(content) # Hello Python!
```



1. WHAT IS FILE HANDLING?

File handling allows your program to read from files or write data to files on your computer.



2. THE OPEN() FUNCTION

open() is used to open a file.

```
open(filename, mode)
```

Mode	Description	If File Exists	If File Doesn't Exist
'r'	Read (default)	Opens	Error
'w'	Write	Overwrites	Creates new
'a'	Append	Appends	Creates new
'x'	Create (exclusive)	Error	Creates new
'b'	Binary mode	(Use with other modes)	
't'	Text mode (default)	(Use with other modes)	
'+'	Read and Write	(Use with other modes)	

3. THE WITH STATEMENT (RECOMMENDED)

Automatically closes the file after its block is done.

```
with open("file.txt", "r") as f:
    data = f.read()
# File is automatically closed here
```



Why use it?

- Cleaner code
- No need to explicitly close the file
- Handles exceptions better

4. READING FROM A FILE

read() - reads the entire file.

```
with open("example.txt", "r") as f:
    content = f.read()
print(content)
```

readline() - reads one line.

```
with open("example.txt", "r") as f:
    line = f.readline()
print(line)
```

readlines() - reads all lines into a list.

```
with open("example.txt", "r") as f:
    lines = f.readlines()
print(lines) # ['line1\n', 'line2\n', ...]
```



5. WRITING TO A FILE

write() - writes a string to the file.

```
with open("output.txt", "w") as f:
    f.write("Hello Python!\n")
    f.write("Have a nice day!")
```

writelines() - writes a list of strings.

```
lines = ["Apple\n", "Banana\n", "Cherry\n"]
with open("fruits.txt", "w") as f:
    f.writelines(lines)
```



6. APPENDING TO A FILE

append mode ('a') adds data at the end.

```
with open("log.txt", "a") as f:
    f.write("New log entry\n")
```



7. CLOSING A FILE (WITHOUT WITH)

If you don't use with, close the file manually.

```
f = open("data.txt", "r")
content = f.read()
f.close() # Important!
```



8. EXAMPLE: READ AND WRITE

Read from one file and write to another.

```
with open("source.txt", "r") as fin,
    open("destination.txt", "w") as fout:
    for line in fin:
        fout.write(line.upper())
```



9. CHECKING FILE PROPERTIES

Use the os module.

```
import os
print(os.path.exists("file.txt")) # True / False
print(os.path.getsize("file.txt")) # Size in bytes
```



10. WORKING WITH CSV FILES

Read and write CSV files using csv module.

```
import csv
# Reading CSV
with open("data.csv", "r", newline="") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```



11. HANDLING EXCEPTIONS

Handle file errors using try...except.

```
try:
    with open("file.txt", "r") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found!")
except PermissionError:
    print("Permission denied!")
```



12. BINARY FILES

Use 'b' mode for non-text files (images, etc.)

```
# Copy an image
with open("image.png", "rb") as fin,
    open("copy.png", "wb") as fout:
    fout.write(fin.read())
```



13. QUICK RECAP

- ✓ Use open() to open files with the right mode.
- ✓ Use with statement for cleaner and safer code.
- ✓ Use read(), readline(), readlines() to read.
- ✓ Use write(), writelines() to write.
- ✓ Use 'a' to append data.
- ✓ Always close the file (or use with).
- ✓ Handle errors with try...except.



Files help your programs **remember**, **store**, and **share** data!



Python File Methods: **seek()**, **tell()** and **truncate()**

These methods give fine control over a file's cursor (pointer) and size.

1. seek() – Move the File Pointer

Moves the file pointer to a specific position.

```
file.seek(offset, whence=0)
```

Parameters:

- **offset** (int): The number of bytes to move.
- **whence** (int): Reference point for the offset.

whence	Meaning	Reference Point
0	Absolute	Beginning of the file (default)
1	Relative	Current position of the pointer
2	From end	End of the file

Notes:

- Positive **offset** moves forward.
- Negative **offset** moves backward (if allowed).
- Cannot seek to a negative position.

2. tell() – Get Current Position

Returns the current position of the file pointer.

```
position = file.tell()
```

Return Value:

An integer representing the current byte position from the beginning of the file.

Example:

```
with open('sample.txt', 'r') as f:
    print(f.tell())    # 0 (start)
    f.read(4)
    print(f.tell())    # 4
    f.seek(2)
    print(f.tell())    # 2
```

3. truncate() – Resize the File

Resizes the file to the specified size.

```
file.truncate(size=None)
```

Parameters:

- **size** (int, optional): New size of the file in bytes.
- If **size** is **None**, the file is truncated at the current position (same as **file.tell()**).

How it works:

- If the new size is smaller than the current size, extra data is removed.
- If the new size is larger, the file is extended (added space, filled with null bytes).

Notes:

- Must be opened in a mode that allows writing (e.g., 'r+', 'w', 'a+').

4. Visual Overview

Initial File Content

File: "Hello Python!"

Index:	0	1	2	3	4	5	7	8	9	10	11
Char :	H	e	l	l	o		y	y	o	n	!

seek(6)

Index:	0	1	2	3	4	5	7	8	9	10	11
Char :	H	e	l	l	o	P	y	h	o	n	!

↑

After truncate(5)

(File size becomes 5 bytes)

Index:	0	1	2	3	4
Char :	H	e	l	l	o

After truncate(10)

(File size becomes 10 bytes)

Index:	0	1	2	3	4	5	6	7	8	9
Char :	H	e	l	l	o	\0	\0	\0	\0	\0

Key Idea

- **seek()** moves the pointer.
- **tell()** tells you where the pointer is.
- **truncate()** changes the size of the file.

5. Examples (All in One)

```
with open('demo.txt', 'r+') as f:
    print('Initial content :', f.read())
    # Initial content : Hello Python!
    f.seek(6)          # Move to 'P'
    print('After seek(6), tell() =', f.tell())  # 6
    print('Read next 3 chars :', f.read(3))     # Pyt
    f.seek(0)          # Go to start
    f.truncate(5)       # Keep only first 5 bytes
    f.seek(0)
    print('After truncate(5):', f.read())        # Hello
    f.truncate(10)      # Extend to 10 bytes
    f.seek(0)
    print('After truncate(10):', f.read())
    # Output: Hello\x00\x00\x00\x00\x00
```

6. Important Notes

- ✓ All positions are in bytes, not characters.
- ✓ In text mode, seeking to arbitrary positions (except 0 or end) may behave unexpectedly on some systems. Safer in binary mode ('rb', 'wb', 'r+b').
- ✓ **truncate()** does not move the file pointer.
- ✓ To overwrite data in the middle, use **seek()** first, then **write()**.
- ✓ Always open in a writable mode for **truncate()**.

7. Common Use Cases

- 🎯 Skip headers and read specific parts.
- Resume reading/writing from a saved position.
- Update or overwrite data in a file.
- Cut off extra data (log rotation, temp files).

8. Summary

Method	Purpose
seek(offset, whence)	Move the file pointer.
tell()	Get the current pointer position.
truncate(size=None)	Resize the file to the given size (or current position).

✓ Remember: **seek()** moves, **tell()** tells, **truncate()** resizes.



PYTHON OOP BASICS

Build real-world applications with Objects!

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hello, {self.name}!"

p = Person("Alice")
print(p.greet()) # Hello, Alice!
```



1. WHAT IS OOP?

OOP (Object-Oriented Programming) is a programming paradigm based on the concept of "objects".

It helps in writing clean, reusable and maintainable code.



Key Benefits

- Modularity
- Reusability
- Scalability
- Easy Maintenance

2. CLASS AND OBJECT

A class is a blueprint for creating objects.
An object is an instance of a class.

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def info(self):
        return f"{self.brand} - {self.year}"
```

```
# Creating objects
car1 = Car("Toyota", 2020)
car2 = Car("Honda", 2022)

print(car1.info()) # Toyota - 2020
```



3. THE __INIT__() METHOD

The `__init__()` method is a special method (constructor) that is called automatically when an object is created.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
s = Student("Riya", 18)
print(s.name) # Riya
print(s.age) # 18
```



4. ATTRIBUTES AND METHODS

Attributes are variables that belong to a class.
Methods are functions that belong to a class.

Attribute (Data)	Method (Behavior)
name	<code>greet()</code>
age	<code>info()</code>
brand	<code>calculate()</code>
year	<code>display()</code>

- ✓ Attributes describe the object.
Methods define what the object can do.

5. SELF KEYWORD

`self` refers to the current object.
It is used to access attributes and methods inside a class.

```
class Book:
    def __init__(self, title):
        self.title = title

    def show(self):
        print(f"Book: {self.title}")
```

← self

```
b = Book("Python Basics")
b.show() # Book: Python Basics
```

6. ACCESS MODIFIERS (CONVENTIONS)

Python uses naming conventions for access modifiers.

Public	name	Accessible from anywhere
Protected	<code>_name</code>	Should be accessed within class and subclasses
Private	<code>__name</code>	Name mangled, meant to be private

- ★ Python doesn't enforce privacy strictly, but these conventions improve readability.

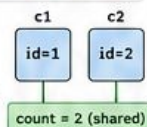
7. CLASS AND INSTANCE VARIABLES

Class variables are shared by all objects.
Instance variables are unique to each object.

```
class Counter:
    count = 0 # class variable

    def __init__(self):
        Counter.count += 1 # shared
        self.id = Counter.count # unique
```

```
c1 = Counter()
c2 = Counter()
print(c1.id, c2.id) # 1 2
```



8. INHERITANCE

Inheritance allows a class (child) to inherit attributes and methods from another class (parent).

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Bark")
```

```
d = Dog()
d.speak() # Bark
```



9. POLYMORPHISM

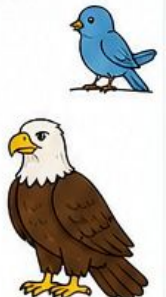
Polymorphism allows methods to do different things based on the object.

```
class Bird:
    def sound(self):
        print("Chirp")

class Eagle(Bird):
    def sound(self):
        print("Screech")
```

```
def make_sound(bird):
    bird.sound()

make_sound(Bird()) # Chirp
make_sound(Eagle()) # Screech
```



10. ENCAPSULATION

Encapsulation is bundling data (attributes) and methods together and restricting direct access to some components.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance # private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

```
acc = Account(1000)
acc.deposit(500)
print(acc.get_balance()) # 1500
```



11. ABSTRACTION

Abstraction hides complex implementation and shows only the essential features.
We achieve this using abstract classes.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14 * self.r * self.r

c = Circle(5)
print(c.area()) # 78.5
```



12. QUICK RECAP

- ✓ Class is a blueprint.
- ✓ Object is an instance of a class.
- ✓ Use `__init__()` to initialize objects.
- ✓ Use `self` to access attributes and methods.
- ✓ Inheritance promotes code reusability.
- ✓ Polymorphism allows flexibility.
- ✓ Encapsulation protects data.
- ✓ Abstraction simplifies complexity.
- ✓ OOP leads to clean, reusable, and scalable Python code!



Think in Objects. Code with Power. Build for the Future!





Public, Private and Protected in Python

Python doesn't have strict access modifiers like C++ or Java.
Instead, it uses **naming conventions** to indicate the level of access intended.

1. Public (No Underscore)

- ✓ Attributes and methods with no leading underscore are public.
- ✓ Accessible from anywhere: inside the class, outside the class, or in subclasses.
- ✓ This is the default level of access.

Example:

```
class MyClass:
    def __init__(self):
        self.name = "Alice" # public attribute

    def show(self):
        return self.name # public method
```

2. Private (Double Underscore __)

- ✓ Attributes and methods with double leading underscore are intended to be private.
- ✓ Not directly accessible from outside the class.
- ✓ Python performs name mangling to make them harder to access.
- ✓ Still accessible, but not recommended.

Example:

```
class MyClass:
    def __init__(self):
        self.__age = 25 # private attribute

    def __secret(self): # private method
        return "This is secret"
```

Name mangling: `_ClassName__attributeName`

3. Protected (Single Underscore _)

- ✓ Attributes and methods with single leading underscore are intended to be protected.
- ✓ Indicates "internal use" or "for subclass use".
- ✓ Accessible within the class and its subclasses.
- ✓ Still accessible from outside (no strict enforcement).

Example:

```
class MyClass:
    def __init__(self):
        self._salary = 50000 # protected attribute

    def _calculate(self): # protected method
        return self._salary * 0.1
```

4. Access Comparison

Access Level	Naming Convention	Accessible Inside Class	Accessible in Subclass	Accessible Outside Class	Purpose
Public	No underscore	✓	✓	✓	Everyone can access.
Protected	Single underscore _	✓	✓	✓ (not recommended)	For internal use / subclass.
Private	Double underscore __	✓	✓	✗ (direct access not allowed)	For internal use only.

5. Code Demonstration

```
class Employee:
    def __init__(self, name, salary):
        self.name = name # Public
        self._salary = salary # Protected
        self.__id = 101 # Private

    def show(self):
        print("Name:", self.name)
        print("Salary:", self._salary)
        print("ID:", self.__id)

    def _calculate_bonus(self):
        return self._salary * 0.1

    def __secret(self):
        return "Secret Method"

emp = Employee("John", 60000)
emp.show()

# Accessing public
print(emp.name) # ✓ Works

# Accessing protected
print(emp._salary) # ⚠ Works, but not recommended

# Accessing private (will raise AttributeError)
# print(emp.__id) # ✗ Error

# Accessing private via name mangling (not recommended)
print(emp._Employee__id) # ✓ Works

# Accessing protected method
print(emp._calculate_bonus()) # ⚠ Works, but not recommended

# Accessing private method via name mangling
print(emp._Employee__secret()) # ✓ Works
```

Output:

```
Name: John
Salary: 60000
ID: 101
John
60000
101

6000.0
Secret Method
```

6. Key Takeaways

- Python follows the principle: "We are all consenting adults here."
- Public members are for everyone.
- Protected members are for internal use and subclasses.
- Private members are for internal use only (name mangling).
- These are conventions, not strict access restrictions.
- Use them to write clean, maintainable and safe code.

7. Name Mangling (for Private)

When you use double underscores, Python internally changes the name:

`__id` in class `MyClass` becomes `_MyClass__id` internally

This makes it harder to access by accident.

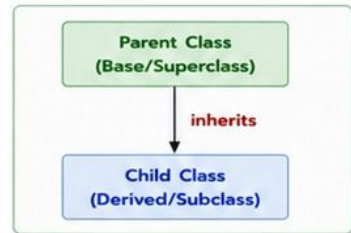
8. Summary

Modifier	Example	Convention	Access Level
Public	name	No underscore	Full access
Protected	<code>_name</code>	Single underscore	Class & Subclass (by convention)
Private	<code>__name</code>	Double underscore	Class only (name mangled)



Inheritance in Python

Inheritance is a way of creating a new class (child/subclass) from an existing class (parent/superclass). The child class inherits attributes and methods from the parent class and can also add new features or override existing ones.



1. Basic Syntax

```
class ParentClass:
    # parent class body
    pass

class ChildClass(ParentClass):
    # child class body
    pass
```

2. Example

Child class inherits attributes and methods of parent class.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):
    def bark(self):
        print(f"{self.name} barks")

d = Dog("Buddy")
d.speak() # Buddy makes a sound (inherited)
d.bark()  # Buddy barks (own method)
```

3. How it Works

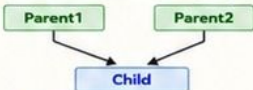
- ✓ Dog is a child class of Animal.
- ✓ Dog inherits `__init__()` and `speak()` from Animal.
- ✓ Dog can use these methods directly.
- ✓ Dog can also define its own methods (bark).
- ✓ This promotes code reusability and hierarchical classification.

4. Types of Inheritance

1. **Single Inheritance**
One child class inherits from one parent.



2. **Multiple Inheritance**
One child class inherits from multiple parents.



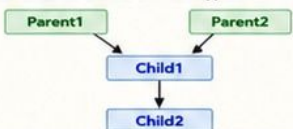
3. **Multilevel Inheritance**
A child class becomes a parent and is inherited by another child.



4. **Hierarchical Inheritance**
Multiple child classes inherit from the same parent.



5. **Hybrid Inheritance**
Combination of two or more types.



5. Method Overriding

Child class can override methods of parent class.

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Cat(Animal):
    def speak(self): # overriding
        print("Cat meows")

c = Cat()
c.speak() # Cat meows
```



Note:

Method overriding allows child class to provide specific implementation.

6. The super() Function

Used to call the parent class method from child class.

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # call parent constructor
        self.breed = breed

d = Dog("Buddy", "Labrador")
print(d.name) # Buddy
print(d.breed) # Labrador
```



Note:

`super()` is especially useful when overriding constructors or methods, to extend parent behavior.

7. isinstance() and issubclass()

```
isinstance(object, classinfo)
• Returns True if object is an instance of classinfo (or its subclass).

d = Dog("Buddy", "Labrador")
print(isinstance(d, Dog)) # True
print(isinstance(d, Animal)) # True

issubclass(subclass, classinfo)
• Returns True if subclass is subclass of classinfo.

print(issubclass(Dog, Animal)) # True
```

8. Key Points

- ✓ Inheritance supports code reusability.
- ✓ Child class inherits public and protected members of parent class.
- ✓ Private members are not directly inherited.
- ✓ Use `super()` to access parent class members.
- ✓ Method overriding enables polymorphism.

9. Summary

- Inheritance allows us to create a new class from an existing class.
- It helps in building a relationship between classes (is-a relationship).
- Python supports different types of inheritance.
- It improves code reusability, readability and maintainability.

★ Tip:

Think of inheritance as "borrowing" properties from a parent, and adding your own features!

Inheritance in Python



In Python, inheritance allows a new class (called a child or derived class) to inherit attributes and methods from an existing class (called a parent or base class). This is a foundational concept for keeping your code DRY (Don't Repeat Yourself).

1. Single Inheritance

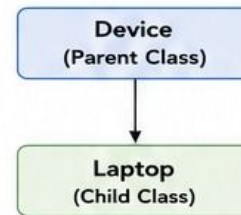
In Single Inheritance, a child class inherits from only one parent class. This is the simplest and most common form of inheritance.

```
# Parent Class
class Device:
    def __init__(self, brand):
        self.brand = brand

    def power_on(self):
        print(f'{self.brand} device is turning on...')

# Child Class inheriting from Device
class Laptop(Device):
    def boot_os(self):
        print(f'{self.brand} laptop is loading operating system.')

# Usage
my_laptop = Laptop("Dell")
my_laptop.power_on() # Inherited method
my_laptop.boot_os() # Child method
```



Output:

Dell device is turning on...
Dell laptop is loading operating system.

2. Multilevel Inheritance

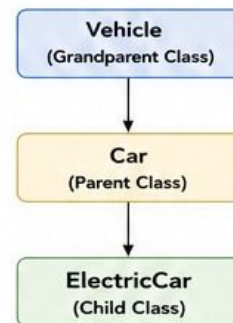
In Multilevel Inheritance, a class inherits from a child class, creating a chain of inheritance (like a family tree: Grandparent → Parent → Child). The class at the bottom gets access to everything above it.

```
# Grandparent Class
class Vehicle:
    def move(self):
        print("Moving forward...")

# Parent Class (inherits from Vehicle)
class Car(Vehicle):
    def play_music(self):
        print("Playing radio music...")

# Child Class (inherits from Car)
class ElectricCar(Car):
    def charge_battery(self):
        print("Battery charging...")

# Usage
tesla = ElectricCar()
tesla.move() # Inherited from Grandparent (Vehicle)
tesla.play_music() # Inherited from Parent (Car)
tesla.charge_battery() # Defined in Child (ElectricCar)
```



Output:

Moving forward...
Playing radio music...
Battery charging...

3. Multiple Inheritance

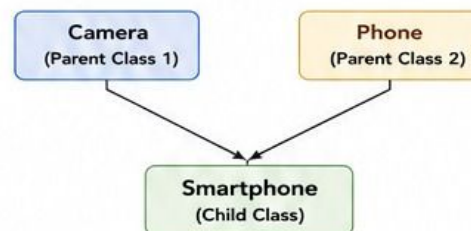
In Multiple Inheritance, a single child class can inherit directly from more than one parent class simultaneously. This allows the child class to combine characteristics from completely different families of classes.

```
# Parent Class 1
class Camera:
    def take_photo(self):
        print("Click! Photo taken.")

# Parent Class 2
class Phone:
    def make_call(self):
        print("Dialing and connecting call...")

# Child Class inheriting from BOTH Camera and Phone
class Smartphone(Camera, Phone):
    def browse_internet(self):
        print("Loading web browser...")

# Usage
iphone = Smartphone()
iphone.make_call() # Inherited from Phone
iphone.browse_internet() # Inherited from Camera
# Defined in Smartphone
```



Output:

Dialing and connecting call...
Click! Photo taken.
Loading web browser...

⚠ The Gotcha with Multiple Inheritance: The Diamond Problem

When you inherit from multiple classes, what happens if both parent classes have a method with the exact same name? Which one does Python run?

This is known as the Diamond Problem. Python solves this cleanly using an ordering system called MRO (Method Resolution Order). Python searches for methods from left to right through the parent classes you specified.

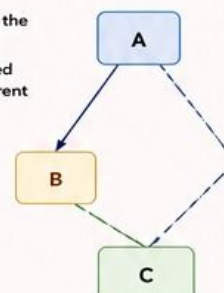
```
class A:
    def greet(self):
        print("Hello from A")

class B:
    def greet(self):
        print("Hello from B")

# Python will prioritize A over B because A is written first (left-to-right)
class C(A, B):
    pass

obj = C()
obj.greet() # Output: "Hello from A"

# You can inspect the exact lookup order using .__mro__
print(C.__mro__)
# Output: (<class 'C'>, <class 'A'>, <class 'B'>, <class 'object'>)
```



Method Resolution Order (MRO) for C

1. C
2. A (searched first)
3. B
4. object (top of hierarchy)

Output:

Hello from A
(<class 'C'>, <class 'A'>, <class 'B'>, <class 'object'>)



Static Methods in Python



A static method is a method that belongs to a class, but does not require access to the instance (**self**) or the class (**cls**).

1. What is a Static Method?

- ✓ Defined using the `@staticmethod` decorator.
- ✓ Does not take `self` (instance) or `cls` (class) as the first parameter.
- ✓ Belongs to the class and can be called using the class name.
- ✓ Used when a method is related to the class logically, but does not need access to class or instance data.

2. Syntax

```
class ClassName:
    @staticmethod
    def method_name(parameters):
        # method body
        # No self or cls parameter
        pass
```

i `@staticmethod` tells Python that this method doesn't need implicit first argument.

3. Example

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def is_even(n):
        return n % 2 == 0

# Calling static methods
print(MathUtils.add(5, 3))      # 8
print(MathUtils.is_even(10))   # True
```

Output:
8
True

4. Comparison: Instance, Class and Static Methods

Type of Method	Decorator	First Parameter	Access To	Called Using	Use Case
Instance Method	(none)	self	Instance data	object.method()	Works with instance-specific data
Class Method	@classmethod	cls	Class data (class variables)	class.method()	Works with class-level data
Static Method	@staticmethod	None	Nothing (no implicit access)	class.method()	Utility/helper function related to the class

5. Key Points

- ✓ Static methods improve code organization.
- ✓ They cannot access `self` (instance attributes) or `cls` (class attributes) directly.
- ✓ They are just like regular functions, but live inside a class namespace.
- ✓ Useful for helper/utility functions that logically belong to a class.

6. Example: When to Use Static Method

Suppose we have a class that works with dates. A method to validate a date does not need any class or instance data.

```
from datetime import datetime

class DateUtils:
    @staticmethod
    def is_valid_date(d, m, y):
        try:
            datetime(y, m, d)
            return True
        except ValueError:
            return False

print(DateUtils.is_valid_date(31, 2, 2024)) # False
print(DateUtils.is_valid_date(29, 2, 2024)) # True
```

7. Can Static Method Access Class or Instance Data?

No direct access:

```
class Example:
    class_var = 10
    def __init__(self):
        self.inst_var = 20

    @staticmethod
    def show():
        # print(self.inst_var) # XError
        # print(cls.class_var) # XError
        print("No access to class or instance data")
```

Example.show()
Output: No access to class or instance data

8. Static Method vs Regular Function

Aspect	Static Method	Regular Function
Belongs to	a class	Does not belong to any class
Called using	class name	Called by function name
Access to class/instance data	No	No
Purpose	Organizational (related to class)	General purpose function

9. Summary

- ★ Use `@staticmethod` for methods that don't need `self` or `cls`.
- ★ Call using `ClassName.method()`.
- ★ Improves code readability and structure.
- ★ Ideal for utility/helper functions inside a class.

★ Best Practice

Use static methods when the method logically belongs to the class, but does not need to access or modify the class or instance.

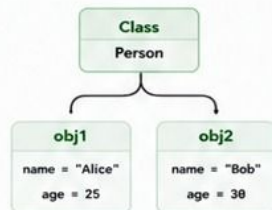


Instance Variable vs Class Variable in Python

In Python, variables defined inside a class can be either instance variables or class variables. They differ in where they are stored, how they are accessed, and how they behave.

1. Instance Variable

- ✓ Defined inside `__init__()` or any instance method.
- ✓ Each instance has its own copy.
- ✓ Belongs to the instance (object).
- ✓ Accessed using `self.variable_name`.
- ✓ Changes made to one instance do not affect others.



```
class Person:
    def __init__(self, name, age):
        self.name = name # instance variable
        self.age = age   # instance variable
```

```
p1 = Person("Alice", 25)
p2 = Person("Bob", 30)
```

```
print(p1.name) # Alice
print(p2.name) # Bob
```

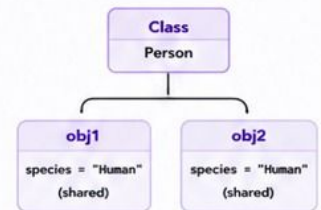
```
p1.age = 26
print(p1.age) # 26
print(p2.age) # 30 (unchanged)
```

Output

```
Alice
Bob
26
30 (unchanged)
```

2. Class Variable

- ✓ Defined inside the class, outside any method.
- ✓ All instances share the same copy.
- ✓ Belongs to the class.
- ✓ Accessed using `ClassName.variable_name` or `instance.variable_name`.
- ✓ Changes made via the class affect all instances.



```
class Person:
    species = "Human" # class variable
    def __init__(self, name):
        self.name = name # instance variable
```

```
p1 = Person("Alice")
p2 = Person("Bob")
```

```
print(p1.name) # Alice
print(p2.name) # Bob
print(p1.species) # Human
print(p2.species) # Human
```

```
Person.species = "Homo sapiens" # change via class
print(p1.species) # Homo sapiens
print(p2.species) # Homo sapiens
```

Output

```
Alice
Bob
Human
Human
Homo sapiens
Homo sapiens
```

3. Key Differences

Aspect	Instance Variable	Class Variable
Where Defined	Inside <code>__init__()</code> or instance method	Inside class, outside any method
Belongs To	Individual instance (object)	The class itself
Copies	Each instance has its own copy	All instances share the same copy
Access	<code>self.variable_name</code>	<code>ClassName.variable_name</code> or <code>instance.variable_name</code>
Memory	More memory (separate copy per instance)	Less memory (single copy for all instances)
Effect of Change	Affects only that instance	Affects all instances (when changed via class)
Use Case	Store unique data per object	Store common data shared by all objects

5. Important Notes

- ★ If you create an instance variable with the same name as a class variable, the instance variable shadows (hides) the class variable for that instance.
- ★ Use class variables for constants or data that is common to all instances.
- ★ Use instance variables for data that is unique to each instance.

```
class Example:
    value = 10 # class variable
    def __init__(self):
        self.value = 20 # instance variable

obj = Example()
print(obj.value) # 20 (instance variable)
print(Example.value) # 10 (class variable)
```

4. Example: Both Together

```
class Student:
    school = "ABC School" # class variable
    total_students = 0 # class variable

    def __init__(self, name):
        self.name = name # instance variable
        Student.total_students += 1
```

```
s1 = Student("Alice")
s2 = Student("Bob")
```

```
print(s1.name, s1.school) # Alice ABC School
print(s2.name, s2.school) # Bob ABC School
print(Student.total_students) # 2
```

```
Student.school = "XYZ School"
```

```
print(s1.school) # XYZ School
print(s2.school) # XYZ School
```

Output

```
Alice ABC School
Bob ABC School
2
XYZ School
XYZ School
```

6. Summary



Instance variables are unique to each object.
Changes affect only that object.



Class variables are shared among all objects.
Changes affect all objects.



Use the right variable type to write efficient and clean Python code.



Pro Tip

Prefer class variables for constants and counters, instance variables for state and behavior.



Class Methods in Python



A class method is a method that is bound to the class and not to the instance of the class. It takes **cls** as the first parameter, which refers to the class itself.

1. What is a Class Method?

- ✓ Defined using the `@classmethod` decorator.
- ✓ Takes `cls` (not `self`) as the first parameter.
- ✓ Bound to the class, not the instance.
- ✓ Can access or modify class variables.
- ✓ Useful for alternative constructors or methods related to the class itself.

2. Syntax

```
class ClassName:
    @classmethod
    def method_name(cls, parameters):
        # method body
        pass
```

i `@classmethod` tells Python that this method is bound to the class.

3. Example

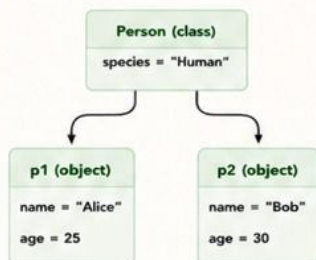
```
class Person:
    species = "Human" # class variable

    def __init__(self, name, age):
        self.name = name # instance variable
        self.age = age

    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species # modify class variable

    @classmethod
    def from_birth_year(cls, name, birth_year):
        import datetime
        age = datetime.datetime.now().year - birth_year
        return cls(name, age) # alternative constructor
```

4. How it Works



- `change_species()` is called on the class.
- It uses `cls` to modify the class variable.
- All instances reflect the change.
- `from_birth_year()` creates a new instance using class data.

5. Example: Using Class Methods

```
p1 = Person("Alice", 25)
p2 = Person("Bob", 30)

print(Person.species) # Human

Person.change_species("Homo sapiens")

print(Person.species) # Homo sapiens
print(p1.species)     # Homo sapiens
print(p2.species)     # Homo sapiens

p3 = Person.from_birth_year("Charlie", 2000)
print(p3.name, p3.age) # Charlie 24 (if current year is 2024)
```

Output:

```
Human
Homo sapiens
Homo sapiens
Homo sapiens
Homo sapiens
Charlie 24
```

6. Class Method vs Instance Method vs Static Method

Aspect	Instance Method	Class Method	Static Method
Decorator	None	@classmethod	@staticmethod
First Parameter	self	cls	None
Bound To	Instance	Class	Neither
Access Instance Variables	✓	✗	✗
Access Class Variables	✓	✓	✗
Use Case	Work with instance data	Work with class data (or alternative constructors)	Utility functions (not related to class or instance)

7. More Example

```
class MathUtils:
    pi = 3.14159

    @classmethod
    def set_pi(cls, value):
        cls.pi = value

    @classmethod
    def get_pi(cls):
        return cls.pi

MathUtils.set_pi(3.14)
print(MathUtils.get_pi()) # 3.14
```

Output:

```
3.14
```



Class methods are commonly used as alternative constructors or to modify class-level data.

8. Key Points

- ★ Class methods are defined with `@classmethod`.
- ★ They take `cls` as the first argument.
- ★ They can modify class state that applies across all instances.
- ★ They can be called on the class itself, not just on instances.
- ★ Often used for factory methods or operations related to the class.

9. Common Use Cases

- ✓ Alternative constructors (e.g., from string, from dict).
- ✓ Modifying class-level data.
- ✓ Factory methods.
- ✓ Operations that logically belong to the class.

10. Quick Example Summary

```
Person.change_species("Homo sapiens") # class method
p1 = Person("Alice", 25)              # instance
p3 = Person.from_birth_year("Charlie", 2000) # alt constructor
print(Person.species)                  # Homo sapiens
```




super() in Python



super() is a built-in function that returns a temporary object of the superclass, which allows us to access methods and attributes of the parent class.

1. What is super()?

- ✓ Provides a way to call methods from a parent (super) class.
- ✓ Helps in code reusability and avoids hard-coding parent class names.
- ✓ Especially useful in inheritance, multiple inheritance, and method overriding.
- ✓ Returns a proxy object that delegates method calls to the parent class.

2. Syntax

```
super(type, object)
```

- ▶ type → The class type.
- ▶ object → The instance of the class.

Commonly used as:

```
super().__init__(...)  
super().method(...)
```

i In Python 3, you usually don't need to pass any arguments: `super()`

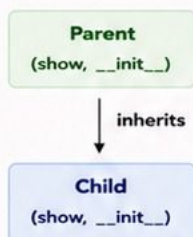
3. Example: Single Inheritance

```
class Parent:  
    def __init__(self, name):  
        self.name = name  
  
    def show(self):  
        print(f"Parent: {self.name}")  
  
class Child(Parent):  
    def __init__(self, name, age):  
        super().__init__(name) # call parent __init__  
        self.age = age  
  
    def show(self):  
        super().show() # call parent show  
        print(f"Child: {self.age}")  
  
c = Child("Alice", 10)  
c.show()
```

Output

```
Parent: Alice  
Child: 10
```

4. How super() Works (Single Inheritance)



- ▶ When `Child().show()` calls `super().show()`, Python looks for the next method in the Method Resolution Order (MRO).
- ▶ It finds `show()` in Parent and calls it.
- ▶ This avoids directly calling `Parent.show(self)`.

6. Why use super()?

- ✓ Makes code support cooperative multiple inheritance.
- ✓ Avoids tight coupling (no hard-coded parent class names).
- ✓ Easier to maintain (works even if the parent class changes).
- ✓ Recommended in almost all inheritance scenarios.

7. super() vs Direct Parent Call

Aspect	Using super()	Using Parent.method(self)
Flexibility	High (works with MRO)	Low (fixed parent)
Multiple Inheritance	Works correctly	May break the chain
Maintenance	Easier	Harder
Best Practice	Recommended	Not recommended

5. Example: Multiple Inheritance (MRO in Action)

```
class A:  
    def show(self):  
        print("A.show()")  
  
class B(A):  
    def show(self):  
        print("B.show()")  
        super().show()  
  
class C(A):  
    def show(self):  
        print("C.show()")  
        super().show()  
  
class D(B, C):  
    def show(self):  
        print("D.show()")  
        super().show()  
  
obj = D()  
obj.show()  
  
print("MRO:", [cls.__name__ for cls in D.__mro__])
```

Output

```
D.show()  
B.show()  
C.show()  
A.show()  
MRO: ['D', 'B', 'C', 'A', 'object']
```

super() follows the MRO (Method Resolution Order) to find the next method.

8. Important Notes

- ★ Use `super()` inside methods of a class.
- ★ Usually called in `__init__()` to initialize parent class.
- ★ In new-style classes (Python 3), always prefer `super()`.
- ★ Must be used in the context of inheritance.
- ★ `super()` without arguments works because Python knows the current class and instance.

Behind the Scenes

`super()` returns a proxy object that looks for methods in the next class in the MRO.



9. Summary

`super()` is a powerful keyword that helps us access parent class methods in a clean, maintainable, and Pythonic way. It plays a key role in inheritance, especially multiple inheritance, by following the MRO.



Pro Tip

Always prefer `super()` over direct parent class calls. It makes your code future-proof and easier to extend.



DECORATORS IN PYTHON

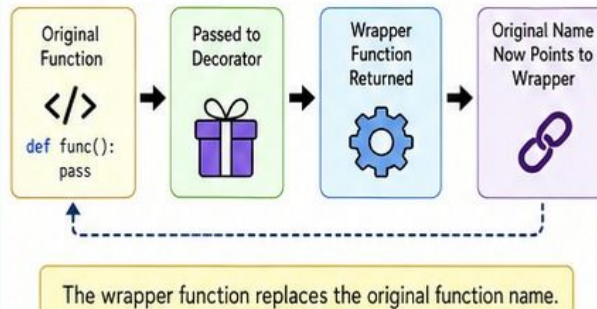


1 WHAT IS A DECORATOR?

- A decorator is a function that takes another function and extends or modifies its behavior without permanently changing the original function.
- Decorators are **higher-order functions**.
- They are applied using the `@decorator_name` syntax.



2 HOW IT WORKS



3 BASIC SYNTAX

Syntax using `@decorator`:

```
@my_decorator
def greet():
    print("Hello")
```

Equivalent form:

```
def greet():
    print("Hello")
greet = my_decorator(greet)
```

4 SIMPLE EXAMPLE

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Output:

```
Before function call
Hello!
After function call
```

5 DECORATOR WITH *ARGS AND **KWARGS

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(2, 3))
```

Output:

```
Calling add
5
```

6 DECORATORS WITH ARGUMENTS

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hi, {name}!")

greet("Bob")
```

Output:

```
Hi, Bob!
Hi, Bob!
Hi, Bob!
```

7 WHY USE DECORATORS?



Reuse common logic



Keep code DRY
(Don't Repeat Yourself)



Add logging, timing, caching,
authentication, validation



Separate core logic from
extra behavior

8 COMMON BUILT-IN / REAL-WORLD USE CASES



`@staticmethod`



`@classmethod`



`@property`



Logging



Timing



Access control /
authentication



Caching (`functools.lru_cache`)

9 PRESERVING METADATA

Use `functools.wraps` to preserve the original function's metadata.

```
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

It preserves `__name__`, `__doc__`,
and other metadata.

10 MULTIPLE DECORATORS

Decorators are applied from bottom to top.

```
@decorator1
@decorator2
def func():
    pass
```

Equivalent to:

```
func = decorator1(decorator2(func))
```

11 KEY POINTS / LIMITATIONS



A decorator usually returns a wrapper function.



Without `@wraps`, metadata may be lost.



Overusing decorators can reduce readability.



Use them for cross-cutting concerns.



Decorators run at function definition time
(not when the function is called).

12 QUICK SUMMARY

Concept	Meaning
Decorator	modifies behavior of a function
Wrapper	inner function adding extra behavior
@syntax	applies decorator
*args, **kwargs	handles any function signature
@wraps	preserves metadata



PRO TIP

Think of a
decorator as a
reusable wrapper
around a function.



DECORATORS MAKE YOUR CODE **CLEANER, SMARTER, AND MORE POWERFUL!**





Python Decorators



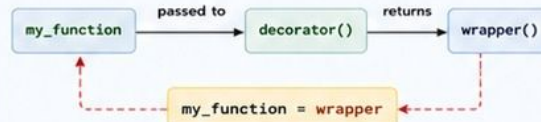
A decorator is a function that takes another function and extends or modifies its behavior without permanently modifying the original function.

1. What is a Decorator?

- ✓ A decorator is a higher-order function.
- ✓ It takes a function as an argument.
- ✓ It returns a new function with added functionality.
- ✓ It is applied using the @decorator_name syntax.

2. How Does it Work?

my_function() is passed to decorator().
Decorator returns a new function (wrapper).
The name my_function now refers to the wrapper.



3. Syntax

```
@decorator_name
def my_function():
    pass
```

is equivalent to:

```
def my_function():
    pass
my_function = decorator_name(my_function)
```

4. Example

A simple decorator that prints messages before and after a function call.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

@my_decorator
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Alice")
```

Output:

```
Before function call
Hello, Alice!
After function call
```

5. Example: With Arguments

The decorator can handle functions with arguments.

```
def repeat(num_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hi, {name}!")

greet("Bob")
```

Output:

```
Hi, Bob!
Hi, Bob!
Hi, Bob!
```

6. Multiple Decorators

Decorators are applied from the bottom up.

```
def decorator1(func):
    def wrapper(*args, **kwargs):
        print("Decorator 1 - Before")
        result = func(*args, **kwargs)
        print("Decorator 1 - After")
        return result
    return wrapper

def decorator2(func):
    def wrapper(*args, **kwargs):
        print("Decorator 2 - Before")
        result = func(*args, **kwargs)
        print("Decorator 2 - After")
        return result
    return wrapper

@decorator1
@decorator2
def say(name):
    print(f"Hello {name}")

say("Python")
```

Output:

```
Decorator 1 - Before
Decorator 2 - Before
Hello Python
Decorator 2 - After
Decorator 1 - After
```

7. Built-in Use Cases

- ✓ Logging
Measure and log function calls.
- ✓ Authentication
Check permissions before execution.
- ✓ Timing
Measure execution time.
- ✓ Caching
Store results to avoid recomputation.
- ✓ Validation
Check inputs before running function.

8. Preserving Function Metadata

Use functools.wraps to preserve the original function's metadata (name, docstring, etc.).

```
from functools import wraps

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        """This is wrapper docstring."""
        return func(*args, **kwargs)
    return wrapper

@decorator
def my_func():
    """This is original docstring."""
    pass

print(my_func.__name__) # my_func
print(my_func.__doc__) # This is original docstring.
```

9. Key Points

- ✓ Decorators provide a clean and readable way to extend functionality.
- ✓ They follow the DRY (Don't Repeat Yourself) principle.
- ✓ Decorators are executed at function definition time.
- ✓ They return a new function object.
- ✓ Use @wraps to keep metadata intact.

10. Summary

Concept	Description
Decorator	A function that modifies another function.
@ Syntax	Applies the decorator to a function.
*args, **kwargs	Allow the wrapper to accept any arguments.
@wraps(func)	Preserves the metadata of the original function.



Pro Tip

Stack decorators for powerful and reusable function enhancements!



Getters and Setters in Python



Getters and setters are methods used to access (get) and update (set) the value of an object's private attributes in a controlled way.

1. Why Use Getters and Setters?

- ✓ **Encapsulation:** Protects data from direct access and unintended modification.
- ✓ **Validation:** Allows you to validate data before updating an attribute.
- ✓ **Flexibility:** You can change the internal implementation later without affecting external code.
- ✓ **Control:** Restrict access (read-only, write-only, or with rules).

2. Basic Example

```
class Person:
    def __init__(self, name, age):
        self._name = name        # protected by convention
        self._age = age          # protected by convention

    # Getter method
    def get_age(self):
        return self._age

    # Setter method
    def set_age(self, age):
        if age < 0:
            raise ValueError("Age cannot be negative")
        self._age = age

# Usage
p = Person("Alice", 25)
print(p.get_age())    # 25
p.set_age(30)
print(p.get_age())    # 30
# p.set_age(-5)       # ValueError
```

3. Using @property Decorator (Pythonic Way)

Python `property()` lets us use getters and setters like attributes.

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def age(self):          # getter
        return self._age

    @age.setter
    def age(self, value):  # setter
        if value < 0:
            raise ValueError("Age cannot be negative")
        self._age = value

# Usage
p = Person("Bob", 20)
print(p.age)              # calls getter
p.age = 35                # calls setter
print(p.age)              # 35
# p.age = -1              # ValueError
```

4. Types of Access Control

Type	How to Implement	Description	Example
Read-Write (getter & setter)	@property @<prop>.setter	Attribute can be read and written with validation.	p.age # read p.age = 30 # write
Read-Only (getter only)	@property	Attribute can be read but not written.	print(p.name) # p.name = "John" -> AttributeError
Write-Only (setter only)	Only @<prop>.setter (no getter)	Attribute can be set but not read directly.	p.password = "1234" # print(p.password) -> AttributeError

5. Read-Only Example

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

p = Person("Charlie")
print(p.name)    # Charlie
# p.name = "Dave" # AttributeError: can't set attribute
```

6. Write-Only Example

```
class Account:
    def __init__(self):
        self._password = None

    @property
    def password(self):
        raise AttributeError("Password is write-only")

    @password.setter
    def password(self, value):
        if len(value) < 6:
            raise ValueError("Password must be at least 6 chars")
        self._password = value

a = Account()
a.password = "secret123"    # OK
# print(a.password)        # AttributeError
```

7. Benefits Summary

- ★ Protects data from direct modification.
- ★ Helps in keeping the class implementation flexible.
- ★ Allows data validation and error checking.
- ★ Provides better control over attribute access.

8. Important Notes

- Python doesn't enforce private truly. It follows the naming convention:
 `_attribute` -> protected (convention)
 `__attribute` -> private (name mangling)
- Getters and setters add logic; use them only when necessary.

9. Private Attributes (Name Mangling)

Double underscore (`__`) makes attribute name-mangled.

```
class Example:
    def __init__(self):
        self.__secret = 100

e = Example()
# print(e.__secret)    # AttributeError
# Accessing (not recommended):
print(e._Example__secret)    # 100
```

Quick Summary

- Getter** -> to get (read) value
- Setter** -> to set (update) value
- @property** -> makes access clean and Pythonic



dir(), __dict__ and help() in Python



💡 These are built-in tools in Python that help us explore objects, their attributes and documentation.

1. dir()

Returns a sorted list of names (attributes and methods) of an object.

- ✔ Useful to know what all attributes/methods an object has.
- ✔ Includes default attributes too.

Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hi, I'm {self.name}"

s = Student("Alice", 20)
print(dir(s))
```

Output (truncated)

```
['__class__', '__delattr__', '__dict__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__',
'__getattr__', ... 'age', 'greet', 'name']
# full list is longer
```

2. __dict__

Returns the dictionary of an object containing its writable attributes (instance variables).

- 🕒 Shows attributes that are stored in the instance namespace.
- ✔ Does NOT include methods or built-in attributes.

Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.course = "Python"

s = Student("Alice", 20)
print(s.__dict__)
```

Output

```
{'name': 'Alice', 'age': 20, 'course': 'Python'}
```

🕒 Note: Shows only instance variables added by us.

3. help()

Displays the documentation for modules, classes, functions, methods or keywords.

- ✔ Provides useful information like purpose, usage, parameters, etc.
- ✔ Very helpful for learning and quick reference.

Example

```
class Student:
    """This is a Student class."""

    def greet(self):
        """Returns a greeting message."""
        return "Hello!"

help(Student)
```

Output (truncated)

```
Help on class Student in module __main__:

class Student(builtins.object)
|   This is a Student class.
|
|   Methods defined here:
|
|       greet(self)
|           Returns a greeting message.
|
|   (press 'q' to quit help in interactive mode)
```

4. Quick Comparison

Feature	dir()	__dict__	help()
Purpose	List all attributes and methods	Show instance variables only	Show documentation and info
Includes Methods?	Yes	No	N/A
Includes Built-in Attributes?	Yes	No	N/A
Returns	List of names	Dictionary	Displays help text
Usage	dir(obj)	obj.__dict__	help(obj)

5. More Examples

dir() on built-in object

```
print(dir("hello")) # string object
# many string methods listed
```

__dict__ on empty object

```
class A: pass
a = A()
print(a.__dict__) # {}
```

help() on function

```
def add(a, b):
    """Returns sum of two numbers."""
    return a + b

help(add)
```

6. Key Takeaways

- ✔ Use **dir()** to explore what an object can do.
- ✔ Use **__dict__** to see an object's data (attributes) that it holds.
- ✔ Use **help()** to read documentation and understand how to use it.

💡 Pro Tips

- ★ **dir()** is great for exploration.
- ★ **__dict__** helps in debugging object state.
- ★ **help()** is your friend for quick docs!



Dunder Methods in Python

Legend	
	Instance method
	Class method
	Called implicitly



Dunder (double underscore) methods are special methods in Python classes. They start and end with double underscores, like `__init__` or `__str__`.

1. Object Creation, Initialization & Destruction

Method	Purpose	Called	Example
<code>__new__(cls, ...)</code>	Creates a new instance of the class	Before <code>__init__</code>	<code>obj = MyClass.__new__(MyClass)</code>
<code>__init__(self, ...)</code>	Initializes the new instance	After <code>__new__</code>	<code>obj = MyClass(...)</code>
<code>__del__(self)</code>	Called when object is about to be destroyed	Garbage collection	<code>del obj</code>
<code>__repr__(self)</code>	Official string representation (unambiguous)	<code>repr(obj)</code> , interpreter	<code>repr(obj)</code>
<code>__str__(self)</code>	Human-readable string representation	<code>print(obj)</code> , <code>str(obj)</code>	<code>print(obj)</code>
<code>__format__(self, format_spec)</code>	Custom behavior for the <code>format()</code> function	<code>format(obj, '...')</code>	<code>format(obj, '.2f')</code>

4. Type Conversion

Method	Purpose	Example
<code>__bool__(self)</code>	Truth value testing (<code>bool()</code>)	<code>bool(obj)</code>
<code>__int__(self)</code>	Convert to int	<code>int(obj)</code>
<code>__float__(self)</code>	Convert to float	<code>float(obj)</code>
<code>__complex__(self)</code>	Convert to complex	<code>complex(obj)</code>
<code>__index__(self)</code>	Used for slicing, <code>hex()</code> , <code>bin()</code>	<code>hex(obj)</code>
<code>__hash__(self)</code>	Hash value (for dict/set)	<code>hash(obj)</code>

5. Collection / Container Protocols

Method	Purpose	Example
<code>__len__(self)</code>	Length of object (<code>len()</code>)	<code>len(obj)</code>
<code>__getitem__(self, key)</code>	Get item by key/index (<code>obj[key]</code>)	<code>obj[key]</code>
<code>__setitem__(self, key, value)</code>	Set item by key/index	<code>obj[key] = value</code>
<code>__delitem__(self, key)</code>	Delete item by key/index	<code>del obj[key]</code>
<code>__iter__(self)</code>	Return iterator (for loops)	<code>for x in obj: ...</code>
<code>__next__(self)</code>	Return next item (iterator)	<code>next(it)</code>
<code>__contains__(self, item)</code>	Membership test (<code>in</code>)	<code>item in obj</code>
<code>__reversed__(self)</code>	Return reverse iterator	<code>reversed(obj)</code>

6. Attribute Access & Management

Method	Purpose	Example
<code>__getattr__(self, name)</code>	Called when attribute not found	<code>obj.missing</code>
<code>__getattribute__(self, name)</code>	Called on every attribute access	<code>obj.attr</code>
<code>__setattr__(self, name, value)</code>	Called on attribute set	<code>obj.attr = value</code>
<code>__delattr__(self, name)</code>	Called on attribute delete	<code>del obj.attr</code>
<code>__dir__(self)</code>	List attributes (<code>dir()</code>)	<code>dir(obj)</code>

2. Comparison & Equality

Method	Purpose	Example
<code>__eq__(self, other)</code>	Equal to (<code>==</code>)	<code>a == b</code>
<code>__ne__(self, other)</code>	Not equal (<code>!=</code>)	<code>a != b</code>
<code>__lt__(self, other)</code>	Less than (<code><</code>)	<code>a < b</code>
<code>__le__(self, other)</code>	Less than or equal (<code><=</code>)	<code>a <= b</code>
<code>__gt__(self, other)</code>	Greater than (<code>></code>)	<code>a > b</code>
<code>__ge__(self, other)</code>	Greater than or equal (<code>>=</code>)	<code>a >= b</code>

3. Arithmetic Operations

Method	Purpose	Example
<code>__add__(self, other)</code>	Addition (<code>+</code>)	<code>a + b</code>
<code>__sub__(self, other)</code>	Subtraction (<code>-</code>)	<code>a - b</code>
<code>__mul__(self, other)</code>	Multiplication (<code>*</code>)	<code>a * b</code>
<code>__truediv__(self, other)</code>	True division (<code>/</code>)	<code>a / b</code>
<code>__floordiv__(self, other)</code>	Floor division (<code>//</code>)	<code>a // b</code>
<code>__mod__(self, other)</code>	Modulo (<code>%</code>)	<code>a % b</code>
<code>__pow__(self, other[, mod])</code>	Power (<code>**</code>)	<code>a ** b</code>
<code>__neg__(self)</code>	Unary negation (<code>-x</code>)	<code>-a</code>
<code>__pos__(self)</code>	Unary plus (<code>+x</code>)	<code>+a</code>
<code>__abs__(self)</code>	Absolute value (<code>abs(x)</code>)	<code>abs(a)</code>
<code>__round__(self, ndigits=None)</code>	Round value	<code>round(a, 2)</code>

7. Callable, Context Manager & Others

Method	Purpose	Example
<code>__call__(self, ...)</code>	Make object callable	<code>obj(...)</code>
<code>__enter__(self)</code>	Enter runtime context	<code>with obj as x: ...</code>
<code>__exit__(self, exc_type, exc, tb)</code>	Exit runtime context	<code>with obj as x: ...</code>
<code>__await__(self)</code>	Make object awaitable	<code>await obj</code>
<code>__aiter__(self)</code>	Async iterator	<code>async for x in obj: ...</code>
<code>__anext__(self)</code>	Async next item	<code>anext(ait)</code>
<code>__sizeof__(self)</code>	Size of object in bytes	<code>obj.__sizeof__()</code>

8. Operator Overloading (Bitwise & Others)

Method	Purpose	Example
<code>__and__(self, other)</code>	Bitwise AND (<code>&</code>)	<code>a & b</code>
<code>__or__(self, other)</code>	Bitwise OR (<code> </code>)	<code>a b</code>
<code>__xor__(self, other)</code>	Bitwise XOR (<code>^</code>)	<code>a ^ b</code>
<code>__invert__(self)</code>	Bitwise NOT (<code>~</code>)	<code>~a</code>
<code>__lshift__(self, other)</code>	Left shift (<code><<</code>)	<code>a << b</code>
<code>__rshift__(self, other)</code>	Right shift (<code>>></code>)	<code>a >> b</code>
<code>__matmul__(self, other)</code>	Matrix multiply (<code>@</code>)	<code>a @ b</code>

Key Points

- ★ Dunder methods enable operator overloading and Pythonic behavior.
- ★ They are called implicitly by Python, not directly in most cases.
- ★ Implement only the ones that make sense for your class.
- ★ Overuse can make code harder to understand.

Quick Tips

- ✓ Use `__str__` for user-friendly output.
- ✓ Use `__repr__` for debugging and official representation.
- ✓ Implement `__eq__` and `__hash__` together for hashable objects.
- ✓ Follow Python data model & keep behavior consistent.

Note

This list covers the most commonly used dunder methods. Python has more (e.g. `__subclasses__()`, `__mro_entries__()`, `__class_getitem__()`, etc.) used in advanced scenarios.



Method Overriding in Python



Method overriding is a feature of OOP in which a subclass provides a specific implementation of a method that is already defined in its superclass. The method in the subclass has the same name, same parameters (or compatible), and same return type (or compatible) as the method in the superclass.

1. Why Method Overriding?

- ✓ Allows a subclass to provide a specific implementation of a method.
- ✓ Used to achieve runtime polymorphism.
- ✓ The method to be overridden is already present in the parent class.
- ✓ The method name, parameters, and return type (or compatible) must be the same.
- ✓ Python always calls the method of the child class (subclass), not the parent class.

2. Basic Syntax

```
class Parent:
    def method(self):
        # some implementation
        pass

class Child(Parent):
    def method(self): # overriding
        # new implementation in child
        pass
```

i The method in Child class overrides the method in Parent class.

3. Example

```
class Animal:
    def speak(self):
        return "Animal makes a sound"

class Dog(Animal):
    def speak(self): # Overriding speak()
        return "Dog barks"

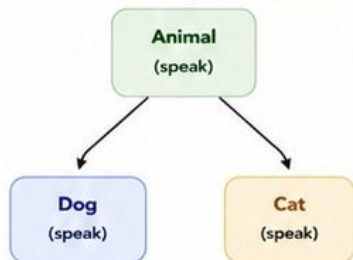
class Cat(Animal):
    def speak(self): # Overriding speak()
        return "Cat meows"

# Using the classes
animals = [Animal(), Dog(), Cat()]
for a in animals:
    print(a.speak())
```

Output

Animal makes a sound
Dog barks
Cat meows

4. How It Works (Visualization)



Both Dog and Cat classes have their own implementation of `speak()`.

When we call `speak()` using an object of Dog or Cat, Python looks at the object's class and calls the version of `speak()` defined in that class.

This is Runtime Polymorphism

5. Method Resolution in Action

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Puppy(Dog):
    pass

p = Puppy()
p.speak() # Output: Dog barks
```

- ▶ Puppy does not have its own `speak()` method.
- ▶ Python looks for `speak()` in Puppy → not found.
- ▶ Then in Dog → found! So Dog's `speak()` is called.
- ▶ Finally, if not found in Dog, it would go to Animal.

i This search order is called **Method Resolution Order (MRO)**.

6. Overriding with `super()`

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        super().speak() # call parent class method
        print("Dog barks") # additional behavior

d = Dog()
d.speak()
```

Output

Animal speaks
Dog barks

`super()` allows the child class to extend the behavior of the parent class instead of completely replacing it.

7. Important Points

- ★ The method in the subclass must have the same name as in the parent class.
- ★ The parameters should be the same or compatible.
- ★ The return type should be the same or compatible.
- ★ Access specifier can be the same or broader (not more restrictive).
- ★ Overriding happens at runtime, not compile time.
- ★ It is a key feature for achieving polymorphism in Python.

8. Common Use Case

Useful when subclasses need to provide specific behavior that is different from the general behavior defined in the parent class.

Example:

- ▶ Different shapes (Circle, Rectangle) overriding `area()` method.
- ▶ Different animals overriding `sound()` method.
- ▶ Different payment types overriding `process_payment()` method.

9. Summary Table

Aspect	Description
Definition	Subclass provides a specific implementation of a method already defined in its superclass.
Purpose	To achieve runtime polymorphism.
How it works	The method in the subclass is called instead of the parent class method.
Key Benefit	Allows flexibility and reusability in code.
Keyword (optional)	<code>super()</code> can be used to call the parent class method.



Operator Overloading in Python



Operator overloading is a feature of OOP that allows operators (+, -, *, ==, etc.) to work with user-defined objects. It is achieved by defining special (dunder) methods in a class.

1. What is Operator Overloading?

- ✓ Operators have a special meaning for built-in types (int, float, str, etc.).
- ✓ We can define the behavior of these operators for objects of our own classes.
- ✓ This makes code more intuitive and natural to read.
- ✓ It does not create new operators, but redefines the existing ones for user-defined types.

2. How Does It Work?

- ▶ Every operator in Python is internally mapped to a special method.
- ▶ When an operator is used with an object, Python calls the corresponding special method.
- ▶ If that method is not found in the class, Python tries the reverse method (if applicable).
- ▶ If still not found, `TypeError` is raised.

3. Basic Example

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other): # overloading + operator
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
v3 = v1 + v2 # Calls v1.__add__(v2)
print(v3) # Output: Vector(6, 8)
```

Output
Vector(6, 8)

4. Operator Overloading: Common Operators and Corresponding Dunder Methods

Operator	Expression	Dunder Method	Meaning	Operator	Expression	Dunder Method	Meaning
+	a + b	<code>__add__(self, other)</code>	Addition	a > b		<code>__gt__(self, other)</code>	Greater Than
-	a - b	<code>__sub__(self, other)</code>	Subtraction	a >= b		<code>__ge__(self, other)</code>	Greater Than or Equal
*	a * b	<code>__mul__(self, other)</code>	Multiplication	a[i]		<code>__getitem__(self, key)</code>	Get Item
/	a / b	<code>__truediv__(self, other)</code>	True Division	a[i] = v		<code>__setitem__(self, key, value)</code>	Set Item
//	a // b	<code>__floordiv__(self, other)</code>	Floor Division	del a[i]		<code>__delitem__(self, key)</code>	Delete Item
%	a % b	<code>__mod__(self, other)</code>	Modulo	len(a)		<code>__len__(self)</code>	Length
**	a ** b	<code>__pow__(self, other)</code>	Power	a in b		<code>__contains__(self, item)</code>	Containment Test
	-a	<code>__neg__(self)</code>	Unary Negative	callable objects	a(...)	<code>__call__(self, ...)</code>	Make Object Callable
	+a	<code>__pos__(self)</code>	Unary Positive	str(a)		<code>__str__(self)</code>	String Representation
	abs(a)	<code>__abs__(self)</code>	Absolute Value	repr(a)		<code>__repr__(self)</code>	Official String Representation
	a == b	<code>__eq__(self, other)</code>	Equal	int(a)		<code>__int__(self)</code>	Integer Conversion
	a != b	<code>__ne__(self, other)</code>	Not Equal	float(a)		<code>__float__(self)</code>	Float Conversion
	a < b	<code>__lt__(self, other)</code>	Less Than	bool(a)		<code>__bool__(self)</code>	Boolean Value
	a <= b	<code>__le__(self, other)</code>	Less Than or Equal				

5. Comparison Example

```
class Number:
    def __init__(self, value):
        self.value = value

    def __eq__(self, other):
        return self.value == other.value

    def __lt__(self, other):
        return self.value < other.value

n1 = Number(10)
n2 = Number(20)
print(n1 == n2) # False
print(n1 < n2) # True
```

6. In-place (Augmented) Assignment Operators

Operator	Expression	Dunder Method
+=	a += b	<code>__iadd__(self, other)</code>
-=	a -= b	<code>__isub__(self, other)</code>
*=	a *= b	<code>__imul__(self, other)</code>
/=	a /= b	<code>__itruediv__(self, other)</code>
%=	a %= b	<code>__imod__(self, other)</code>
**=	a **= b	<code>__ipow__(self, other)</code>
//=	a //= b	<code>__ifloordiv__(self, other)</code>

💡 If in-place method is not defined, Python falls back to the normal operator method.

7. Example: In-place Addition

```
class Counter:
    def __init__(self, value=0):
        self.value = value

    def __iadd__(self, other): # for +=
        self.value += other
        return self

    def __str__(self):
        return str(self.value)

c = Counter(10)
c += 5 # Calls __iadd__
print(c) # Output: 15
```

Output
15

8. Other Useful Dunder Methods

`__index__(self)` : Used for integer index operations.
`__hash__(self)` : Hash value (for hashing collections).
`__enter__(self)` : Context manager entry (with statement).
`__exit__(self, exc_type, exc, tb)` : Context manager exit.
`__iter__(self)` : Return iterator.
`__next__(self)` : Next item from iterator.
`__reversed__(self)` : Return reverse iterator.
`__format__(self, format_spec)` : Format specification.
`__round__(self, n=None)` : Rounding.

9. Important Points

- ✓ Operator overloading improves the readability and usability of classes.
- ✓ Overload operators only when it makes semantic sense.
- ✓ Maintain consistency with the behavior of built-in types.
- ✓ It does not create new operators; it redefines the behavior of existing ones.

10. Benefits

- ★ Makes custom objects behave like built-in types.
- ★ Enhances code readability and expressiveness.
- ★ Helps in writing clean and intuitive code.
- ★ Useful in domains like mathematics, scientific computing, vectors, matrices, etc.



11. Key Takeaway

Operator overloading lets you give meaning to operators for your own classes by defining special methods. It makes your objects intuitive to use, just like Python's built-in types.



Time Module in Python



The `time` module in Python provides various time-related functions. It allows us to work with time in many ways such as representing time in different formats, performing time-based calculations, pausing execution, measuring performance, etc.

1. Importing the Module

```
import time
```

All functions of `time` module are accessed using `time.method()`.

2. Getting Current Time

Function	Description
<code>time.time()</code>	Current time in seconds since the Epoch (January 1, 1970, 00:00:00 UTC). (float)
<code>time.ctime()</code>	Current time as a formatted string (local time).
<code>time.gmtime()</code>	Current time as <code>struct_time</code> in UTC.
<code>time.localtime()</code>	Current time as <code>struct_time</code> in local time.

Example

```
import time
print(time.time())      # 1716285592.123456
print(time.ctime())     # Mon May 20 10:53:12 2024
print(time.gmtime())    # time.struct_time(...)
print(time.localtime()) # time.struct_time(...)
```

`struct_time` is a tuple-like object with attributes like `tm_year`, `tm_mon`, etc.

3. Converting Time

Function	Description	Example
<code>time.asctime([t])</code>	Convert a <code>struct_time</code> or tuple to a readable string.	<pre>t = time.localtime() print(time.asctime(t)) # 'Mon May 20 10:55:10 2024'</pre>
<code>time.gmtime([secs])</code>	Convert seconds since Epoch to <code>struct_time</code> in UTC.	<pre>time.gmtime(0) # time.struct_time(tm_year=1970, # tm_mon=1, tm_mday=1, ...)</pre>
<code>time.localtime([secs])</code>	Convert seconds since Epoch to <code>struct_time</code> in local time.	<pre>time.localtime(0) # time.struct_time(tm_year=1970, # tm_mon=1, tm_mday=1, ...)</pre>
<code>time.mktime(t)</code>	Convert <code>struct_time</code> in local time to seconds since Epoch.	<pre>t = time.localtime() secs = time.mktime(t)</pre>
<code>time.strptime(string, format)</code>	Parse a time string according to the given format and return <code>struct_time</code> .	<pre>t = time.strptime('20 May 2024', '%d %b %Y') # time.struct_time(...)</pre>
<code>time.strftime(format, t)</code>	Convert <code>struct_time</code> (or tuple) to a formatted string.	<pre>t = time.localtime() print(time.strftime('%Y-%m-%d %H:%M:%S', t)) # '2024-05-20 10:58:30'</pre>

Common `strftime()` / `strptime()` Format Codes

Directive	Meaning	Example	Directive	Meaning	Example
<code>%Y</code>	Year with century	2024	<code>%I</code>	Hour (12-hour clock)	02
<code>%y</code>	Year without century	24	<code>%M</code>	Minute (00-59)	58
<code>%m</code>	Month (01-12)	05	<code>%S</code>	Second (00-59)	30
<code>%b</code>	Abbreviated month name	May	<code>%p</code>	AM or PM	PM
<code>%B</code>	Full month name	May	<code>%a</code>	Abbreviated weekday	Mon
<code>%d</code>	Day of month (01-31)	20	<code>%A</code>	Full weekday name	Monday
<code>%H</code>	Hour (24-hour clock)	14	<code>%%</code>	A literal %	%

4. Measuring Time / Performance

Function	Description	Example
<code>time.perf_counter()</code>	High resolution timer for measuring short durations.	<code>start = time.perf_counter()</code>
<code>time.process_time()</code>	CPU time of current process.	<code>cpu_time = time.process_time()</code>
<code>time.monotonic()</code>	Monotonic clock, not affected by system clock changes.	<code>t = time.monotonic()</code>

Example: Measure Execution Time

```
import time
start = time.perf_counter()

# some code
for _ in range(1000000):
    _ = 2 * 2

end = time.perf_counter()
print(f'Execution time: {end - start:.6f} seconds')
# Gives precise elapsed time
```

5. Time Delays

Function	Description	Example
<code>time.sleep(secs)</code>	Pause execution for the given number of seconds (can be float).	<pre>time.sleep(2) # pauses for 2 seconds</pre>
<code>time.sleep_ms(ms)</code> (Python 3.11+)	Pause execution for the given milliseconds.	<pre>time.sleep_ms(500) # pauses for 500 ms</pre>

Example

```
import time
print('Start')
time.sleep(1.5)
print('After 1.5 seconds')
```

6. Working with Time Zones (UTC vs Local)

- `time` module works with both UTC and local time.
- Use `gmtime()` for UTC and `localtime()` for local time.
- Conversions between the two can be done using `mktime()`, `gmtime()`, `localtime()`.

Example

```
import time
local = time.localtime()
utc = time.gmtime()
print('Local Time :', time.asctime(local))
print('UTC Time   :', time.asctime(utc))
```

7. Other Useful Functions

Function	Description	Example
<code>time.time_ns()</code> (3.7+)	Current time in nanoseconds since Epoch.	<code>ns = time.time_ns()</code>
<code>time.clock_gettime(clk_id)</code>	Get time according to the specified clock (Unix only).	<code>time.clock_gettime(time.CLOCK_REALTIME)</code>
<code>time.clock_settime(clk_id, time)</code>	Set time for the specified clock (Unix only, requires permission).	<code># time.clock_settime(time.CLOCK_REALTIME, t)</code>
<code>time.get_clock_info(name)</code>	Get information about a clock.	<code>info = time.get_clock_info('perf_counter')</code>
<code>time.tzset()</code> (Unix only)	Re-read time zone from environment variables.	<code>time.tzset()</code>

8. Key Takeaways

- ✓ Use `time.time()` to get current timestamp.
- ✓ Use `ctime()`, `asctime()` for human-readable time.
- ✓ Use `strftime()` and `strptime()` for custom formatting and parsing.
- ✓ Use `sleep()` to pause execution.
- ✓ Use `perf_counter()` for accurate performance measurement.
- ✓ Use `gmtime()` for UTC and `localtime()` for local time.



Note:

For advanced date and time handling (time zones, durations, complex calculations), prefer the `datetime` module.

Related Module: `datetime`

More powerful and easier for most applications.



The Walrus Operator (:=) in Python



The **walrus operator** (:=) was introduced in Python 3.8 (PEP 572). It allows you to assign a value to a variable as part of an expression.

1. What is the Walrus Operator?

- ✓ It combines assignment and expression.
- ✓ Syntax: **variable** := **expression**
- ✓ Returns the assigned value.
- ✓ Helps reduce code duplication and improve readability.

2. Basic Syntax



3. Basic Example

```
n := 5
print(n)            # Output: 5
print(n * 2)        # Output: 10
```

The value 5 is assigned to **n** and also returned.

4. Common Use Case: In Loops (Avoiding Duplicate Work)

```
data = "apple,banana,cherry,"
while (chunk := data.read(5)):
    print(chunk)
```

Read 5 characters and assign to chunk.

Output:

```
apple
,bana
na_ch
erry,
```

Loop continues as long as chunk is not empty.

Without :=, we would need to call `data.read(5)` twice or use extra lines, making the code less efficient.

5. In Conditional Statements

```
user_input = input("Enter your name: ")
if (n := len(user_input)) > 0:
    print(f"Hello, {user_input}!")
    print(f"Length of your name is {n}.")
else:
    print("You entered an empty string.")
```

How it works:

`len(user_input)` is computed once, stored in `n`, then used in the condition and inside the block.

6. In List Comprehensions

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [x*x for x in numbers if (x % 2) == 0]
print(squares)    # Output: [4, 16, 36]
```

Here, `(x % 2) == 0` is computed and checked in the same expression (assignment is not stored in a variable here, but the idea of using := is more useful when storing the value).

7. Practical Example: File Reading

```
with open('example.txt', 'r') as f:
    while (line := f.readline()):
        print(line.strip())
```

Reads a line, assigns it to `line`, and continues the loop until an empty string (EOF) is returned.

8. Without Walrus Operator (for comparison)

```
user_input = input("Enter your name: ")
n = len(user_input)
if n > 0:
    print(f"Hello, {user_input}!")
    print(f"Length of your name is {n}.")
else:
    print("You entered an empty string.")
```

More lines, and the expression `len(user_input)` is computed even if not needed later.

9. Where Can You Use Walrus Operator?

- In `if`, `elif`, `while` conditions
- In list, dict, set comprehensions
- In generator expressions
- In any expression where a value is allowed (except the left-hand side of an assignment)

10. Where You CANNOT Use It

- ✗ As a statement by itself
- ✗ As the target of an assignment
- ✗ In default parameter values
- ✗ In lambda expressions
- ✗ In some complex unpacking scenarios

11. Key Takeaways

- ✓ Walrus operator saves lines of code.
- ✓ Improves readability in loops and conditions.
- ✓ Avoids repeating expensive function calls.
- ✓ Use it wisely – too much can reduce readability.



Important:

Overusing the walrus operator can make code harder to read for people unfamiliar with it. Use it when it makes your code clearer, not just shorter.





Python shutil Module



The `shutil` module (short for "shell utilities") provides a high-level interface for working with files, collections of files, and folders. It automates common file operations like copying, moving, deleting, archiving, and more.

1. Introduction

- ✓ Part of Python Standard Library.
- ✓ Introduced in Python 2.4.
- ✓ Works across platforms (Windows, Linux, macOS).
- ✓ Built on top of lower-level modules like `os`, `pathlib`, and `filecmp`.
- ✓ Ideal for high-level file and directory operations without writing OS-specific code.

2. Importing shutil

```
import shutil

# import the module
```

3. File Operations

Function	Description	Example
<code>copy(src, dst)</code>	Copy file content and permissions	<code>shutil.copy('a.txt', 'b.txt')</code>
<code>copy2(src, dst)</code>	Copy file content, permissions and metadata (timestamps)	<code>shutil.copy2('a.txt', 'b.txt')</code>
<code>copyfile(src, dst)</code>	Copy file content only	<code>shutil.copyfile('a.txt', 'b.txt')</code>
<code>copymode(src, dst)</code>	Copy permission bits only	<code>shutil.copymode('a.txt', 'b.txt')</code>
<code>copystat(src, dst)</code>	Copy all file metadata (permission, times, flags)	<code>shutil.copystat('a.txt', 'b.txt')</code>

4. Directory Operations

Function	Description	Example
<code>copytree(src, dst)</code>	Copy entire directory tree	<code>shutil.copytree('dir1', 'dir2')</code>
<code>move(src, dst)</code>	Move/rename file or directory	<code>shutil.move('a.txt', 'new/a.txt')</code>
<code>rmtree(path)</code>	Recursively delete a directory tree	<code>shutil.rmtree('dir1')</code>
<code>ignore_patterns(patterns)</code>	Helper for <code>copytree()</code> to ignore certain files/directories	<code>shutil.copytree('dir1', 'dir2', ignore=shutil.ignore_patterns('*.pyc', '__pycache__'))</code>
<code>chown(path, user=None, group=None)</code>	Change owner and group of a file/directory (Unix only)	<code>shutil.chown('file.txt', user='john', group='users')</code>

5. Disk Usage

Function	Description	Example
<code>disk_usage(path)</code>	Return total, used and free space on a disk	<code>shutil.disk_usage('/')</code>
<code>get_archive_size(base_name, format)</code>	Return size of archive that would be created	<code>shutil.get_archive_size('data', 'zip')</code>

6. Archiving Operations

Function	Description	Example
<code>make_archive(base_name, format, root_dir=None, base_dir=None)</code>	Create an archive file (zip, tar, bztar, gztar, xztar)	<code>shutil.make_archive('backup', 'zip', root_dir='my_folder')</code>
<code>get_archive_formats()</code>	List supported archive formats	<code>shutil.get_archive_formats()</code>
<code>unpack_archive(filename, extract_dir=None, format=None)</code>	Extract an archive file	<code>shutil.unpack_archive('backup.zip', extract_dir='out')</code>
<code>register_archive_format(...)</code>	Register a new archive format (advanced)	<code>shutil.register_archive_format(...)</code>
<code>unregister_archive_format(name)</code>	Unregister a custom archive format	<code>shutil.unregister_archive_format('myfmt')</code>

7. Other Useful Functions

Function	Description	Example
<code>which(cmd)</code>	Return path to executable in the system PATH	<code>shutil.which('python')</code>
<code>which(cmd, path=...)</code>	Search executable in specified path	<code>shutil.which('git', path='/usr/bin')</code>
<code>SameFileError</code>	Raised when source and destination are same file	Used internally by copy functions
<code>Error</code>	General exception for shutil operations	Handle <code>shutil.Error</code> in try-except

8. Common Usage Examples

```
# 1. Copy a file
shutil.copy('source.txt', 'backup.txt')

# 2. Copy a file with metadata (timestamps, etc.)
shutil.copy2('source.txt', 'backup.txt')

# 3. Copy an entire directory tree
shutil.copytree('project/', 'project_backup/')

# 4. Copy directory but ignore some files
shutil.copytree('project/', 'project_backup/',
    ignore=shutil.ignore_patterns('*.pyc', '*.log'))

# 5. Move (or rename) a file
shutil.move('old.txt', 'new_folder/old.txt')

# 6. Delete a directory and all its contents
shutil.rmtree('temp_folder')

# 7. Create a ZIP archive of a folder
shutil.make_archive('my_archive', 'zip', 'project/')

# 8. Extract an archive
shutil.unpack_archive('my_archive.zip', 'extracted/')

# 9. Get disk usage information
usage = shutil.disk_usage('/')
print(f'Total: {usage.total}, Used: {usage.used}, Free: {usage.free}')

# 10. Find path to an executable
python_path = shutil.which('python')
print(python_path)
```

9. Supported Archive Formats

Format Name	Description	Extensions
zip	ZIP archive	.zip
tar	Uncompressed TAR	.tar
gztar	Gzipped TAR	.tar.gz, .tgz
bztar	Bzip2 compressed TAR	.tar.bz2, .tbz
xztar	LZMA compressed TAR	.tar.xz, .txz

💡 Use `shutil.get_archive_formats()` to see all supported formats on your system.

10. ignore_patterns() Examples

```
# Ignore .pyc and __pycache__ folders
ignore = shutil.ignore_patterns('*.pyc', '__pycache__')
shutil.copytree('src/', 'dst/', ignore=ignore)

# Ignore multiple patterns
ignore = shutil.ignore_patterns('*.tmp', '*.log',
    'build/', 'dist/')
```

13. Practical Example: Backup a Folder

This script creates a timestamped backup (ZIP) of a folder.

```
import shutil
import datetime

# Folder to backup
source_folder = 'my_project'

# Create a timestamped archive name
timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
archive_name = f'backup_{timestamp}'

# Create the archive
shutil.make_archive(archive_name, 'zip', source_folder)
print(f'Backup created: {archive_name}.zip')

# Get size of the archive
size = shutil.get_archive_size(archive_name, 'zip')
print(f'Archive size: {size} bytes')
```

12. Exceptions in shutil

Exception	When it Occurs
<code>shutil.Error</code>	General error (e.g., problems during <code>copytree</code> or <code>rmtree</code>)
<code>SameFileError</code>	Source and destination resolve to the same file
<code>SpecialFileError</code>	Operation not supported on special files
<code>ExecError</code>	Error executing external archiving programs
<code>RegistryError</code>	Error in registering/unregistering archive formats



Key Takeaways

- ✓ `shutil` provides high-level operations for files, directories, and archives.
- ✓ It simplifies complex tasks like copying directory trees, creating archives, and deleting folders.
- ✓ Use it for safe, cross-platform file management with minimal code.



Python Requests Module



The **requests** module allows you to send HTTP requests easily and elegantly. It is one of the most popular third-party libraries in Python for making web requests and handling responses.

Install

```
pip install requests
```

1. Importing Requests

```
import requests
```

Always import the module before using it.

2. HTTP Methods

Requests supports all major HTTP methods.

Method	Description
GET	Retrieve data from the server
POST	Send data to the server to create/update
PUT	Update data on the server
PATCH	Partially update data on the server
DELETE	Delete data from the server
HEAD	Same as GET but only returns headers
OPTIONS	Get supported HTTP methods

3. Basic GET Request

```
import requests
```

```
url = 'https://jsonplaceholder.typicode.com/posts/1'
response = requests.get(url)
```

```
print(response.status_code) # 200
print(response.text)        # Raw response (string)
print(response.json())      # Response as JSON (dict)
```

Example Output

```
200
{'userId': 1, 'id': 1, 'title': '...', 'body': '...'}
```

Response Object

The request methods return a Response object with useful attributes and methods.

- `status_code` – HTTP status code
- `text` – Response content as string
- `content` – Response content in bytes
- `json()` – Parse JSON response
- `headers` – Response headers
- `cookies` – Response cookies
- `ok` – True if `status_code < 400`
- `raise_for_status()` – Raise `HTTPError` for bad responses (4xx or 5xx)

4. Query Parameters

Pass parameters in the URL.

```
url = 'https://httpbin.org/get'
params = {
    'name': 'John',
    'age': 25
}
r = requests.get(url, params=params)
print(r.url) # Final URL with parameters
print(r.json()['args'])
```

Output

```
https://httpbin.org/get?name=John&age=25
{'name': 'John', 'age': '25'}
```

5. POST Request (Form Data)

Send form-encoded data.

```
url = 'https://httpbin.org/post'
data = {
    'username': 'john_doe',
    'password': 'secret'
}
r = requests.post(url, data=data)
print(r.status_code) # 200
print(r.json()['form'])
```

Output

```
{'username': 'john_doe', 'password': 'secret'}
```

6. POST Request (JSON Data)

Send JSON data in the request body.

```
url = 'https://httpbin.org/post'
json_data = {
    'username': 'john_doe',
    'password': 'secret'
}
r = requests.post(url, json=json_data)
print(r.status_code) # 200
print(r.json()['json'])
```

Output

```
{'username': 'john_doe', 'password': 'secret'}
```

7. Custom Headers

Send custom HTTP headers.

```
url = 'https://httpbin.org/headers'
headers = {
    'User-Agent': 'MyApp/1.0',
    'Accept': 'application/json'
}
r = requests.get(url, headers=headers)
print(r.json()['headers'])
```

Output (truncated)

```
{'User-Agent': 'MyApp/1.0', 'Accept': 'application/json', ...}
```

8. Path Parameters

Include values in the URL path.

```
user_id = 1
url = f'https://jsonplaceholder.typicode.com/users/{user_id}'
r = requests.get(url)

print(r.status_code) # 200
print(r.json()['name']) # Leanne Graham
```

Output

```
200
Leanne Graham
```

9. Other HTTP Methods

PUT (update), PATCH (partial update), DELETE

```
url = 'https://httpbin.org/put'
r_put = requests.put(url, data={'key': 'value'})

url = 'https://httpbin.org/patch'
r_patch = requests.patch(url, data={'key': 'value'})

url = 'https://httpbin.org/delete'
r_delete = requests.delete(url)
print(r_delete.status_code) # 200
```

10. Authentication

Basic Authentication

```
from requests.auth import HTTPBasicAuth

url = 'https://httpbin.org/basic-auth/user/passwd'
response = requests.get(url, auth=HTTPBasicAuth('user', 'passwd'))
print(response.status_code) # 200
```

Token / Bearer Authentication

```
url = 'https://api.example.com/protected'
headers = {
    'Authorization': 'Bearer YOUR_ACCESS_TOKEN'
}
r = requests.get(url, headers=headers)
```

11. Cookies

Send cookies with the request.

```
url = 'https://httpbin.org/cookies'
cookies = {
    'sessionid': 'abc123',
    'theme': 'dark'
}
r = requests.get(url, cookies=cookies)
print(r.json()['cookies'])
```

Output

```
{'sessionid': 'abc123', 'theme': 'dark'}
```

12. Timeouts

Set a timeout to avoid waiting forever.

```
try:
    r = requests.get('https://httpbin.org/delay/5', timeout=2)
except requests.exceptions.Timeout:
    print('Request timed out!')
```

13. Error Handling

Handle HTTP errors gracefully.

```
url = 'https://httpbin.org/status/404'
r = requests.get(url)
try:
    r.raise_for_status() # Raises HTTPError for 4xx/5xx
except requests.exceptions.HTTPError as err:
    print(f'HTTP Error: {err}')
```

Output

```
HTTP Error: 404 Client Error: NOT FOUND for url: https://httpbin.org/status/404
```

14. Sessions (Connection Pooling)

Use Session for better performance with multiple requests.

```
session = requests.Session()

session.headers.update({'User-Agent': 'MyApp/1.0'})

r1 = session.get('https://httpbin.org/get', params={'a': 1})
r2 = session.post('https://httpbin.org/post', data={'key': 'value'})

print(r1.status_code, r2.status_code)
session.close()
```

15. Streaming a Response

Download large content in chunks.

```
url = 'https://speed.hetzner.de/100MB.bin'

with requests.get(url, stream=True) as r:
    with open('file.bin', 'wb') as f:
        for chunk in r.iter_content(chunk_size=8192):
            if chunk:
                f.write(chunk)
print('Download complete!')
```

16. Common Exceptions

Exception	Description
<code>requests.exceptions.RequestException</code>	Base class for all exceptions
<code>requests.exceptions.ConnectionError</code>	Connection failed
<code>requests.exceptions.Timeout</code>	The request timed out
<code>requests.exceptions.HTTPError</code>	HTTP error response
<code>requests.exceptions.TooManyRedirects</code>	Exceeded max redirects
<code>requests.exceptions.InvalidURL</code>	Invalid URL
<code>requests.exceptions.MissingSchema</code>	Invalid URL schema

17. Useful Response Attributes

Attribute	Example	Description
<code>status_code</code>	200	HTTP status code
<code>headers</code>	{...}	Response headers (dict-like)
<code>text</code>	'<html>...</html>'	Response body as string
<code>content</code>	b'...bytes...'	Response body in bytes
<code>json()</code>	{...}	Parse JSON response
<code>cookies</code>	<RequestsCookieJar>	Cookies sent by the server
<code>url</code>	'https://...'	Final URL (after redirects)
<code>history</code>	[<Response>, ...]	Redirect history

18. Key Takeaways

- ✓ The `requests` module makes HTTP requests simple and intuitive.
- ✓ Use appropriate HTTP methods for your use case.
- ✓ Leverage headers, params, cookies, and authentication.
- ✓ Always handle exceptions and set timeouts for reliability.
- ✓ Use Session for multiple requests to improve performance.



Note: The `requests` module is external. Make sure to install it using: `pip install requests`



python

GENERATORS IN PYTHON



python



- ★ A generator is a special kind of iterator that produces values **one at a time** instead of storing them all at once.
- ★ Generators are **memory-efficient** and useful for large data or **lazy evaluation**.



1

WHAT IS A GENERATOR?



- A generator function uses the keyword **yield**.
- Each time it pauses, it remembers its state and continues from there on the next call.

MEMORY TRICK

Generator =
vending machine / water tap /
one item at a time.

2

REGULAR FUNCTION vs GENERATOR FUNCTION

Regular Function

- Uses **return**
- Sends everything back at once
- Then ends

```
def get_nums():  
    return [1, 2, 3]
```

VS

Generator Function

- Uses **yield**
- Gives one value at a time
- Pauses and resumes

```
def get_nums():  
    yield 1  
    yield 2  
    yield 3
```

3

BASIC SYNTAX

Generator Function

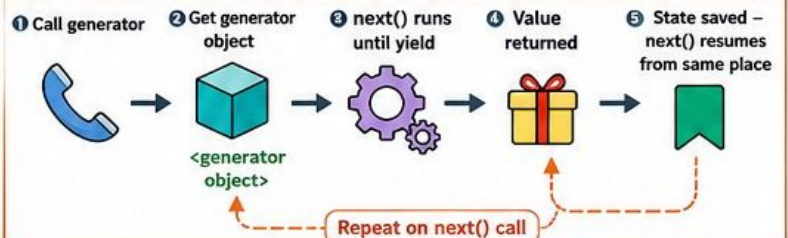
```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1
```

Usage

```
gen = count_up_to(3)  
print(next(gen)) # 1  
print(next(gen)) # 2  
print(next(gen)) # 3
```

4

HOW yield WORKS



5

next() AND StopIteration

- next() gets the next value.
- When values are finished, Python raises **StopIteration**.



StopIteration is raised automatically when the generator is exhausted.

```
gen = count_up_to(2)  
try:  
    while True:  
        print(next(gen))  
except StopIteration:  
    print("No more values!")
```

6

USING GENERATORS IN FOR LOOPS



For loops automatically handle **StopIteration**.

```
for num in count_up_to(5):  
    print(num)  
  
# Output:  
1  
2  
3  
4  
5
```

7

GENERATOR EXPRESSIONS

- Like list comprehensions but use parentheses ().

```
squares = (x*x for x in range(5))  
print(list(squares)) # [0, 1, 4, 9, 16]
```

COMPARISON

<pre>[x*x for x in range(5)]</pre> → list (stored in memory)	<pre>(x*x for x in range(5))</pre> → generator (lazy)
---	--

8

BENEFITS



Memory efficient – does not store all values.



Lazy evaluation – computes values only when needed.



Faster for large streams of data.



Great for reading files, processing logs, pipelines, infinite sequences.



SMART & FAST!

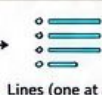
9

COMMON USE CASES

- ✓ Reading large files line by line
- ✓ Streaming data
- ✓ Fibonacci sequence
- ✓ Data pipelines
- ✓ Infinite counters

Example: Read file line by line

```
def read_lines(file):  
    for line in file:  
        yield line.strip()
```



File → Generator → Lines (one at a time)

10

ADVANCED GENERATOR METHODS



send(value)
Send a value into the generator at the yield point.

```
val = gen.send(10)  
# Sends 10 into generator
```



throw(exc)
Raise an exception inside the generator.

```
gen.throw(ValueError("Oops"))  
# Exception raised inside
```



close()
Stop the generator.

```
gen.close()  
# Terminates the generator
```

11

COMMON MISTAKES

- ✗ Forgetting that generators can be exhausted after one full iteration.
- ✗ Confusing generator object with actual values.
- ✗ Trying to index a generator like a list.



NOTE

Convert to list if you need all values at once.

```
list(gen)
```

12

QUICK SUMMARY / TABLE

CONCEPT	DESCRIPTION
Generator function	Uses yield to produce values.
Generator object	Iterable object, produces values lazily.
next()	Get the next item from the generator.
StopIteration	Raised when no more values are available.
Generator expression	Compact syntax using parentheses ().

13

PRO TIP



Use generators when you want **one value at a time**, not all values in memory at once.

MEMORY LINE

List = full lunchbox,
Generator = lunch dispenser.



THINK LAZY, SAVE MEMORY, WRITE SMARTER CODE!





PYTHON MINI PROJECTS

Small Projects. Big Learning. Real Impact! 🌟

```
print("Build Projects.")
print("Gain Skills.")
print("Create Impact!")
print("Keep Coding! 🚀")
```



1. TO-DO LIST

A simple command-line to-do list where you can add, view, complete and delete tasks.

Features:

- Add new tasks
- View all tasks
- Mark tasks as complete
- Delete tasks
- Data saved in a file



Example (add task):

```
task = input("Enter task: ")
with open("tasks.txt", "a") as f:
    f.write(task + "\n")
```

Key Concepts: File Handling, Lists, Loops

2. CALCULATOR

A simple calculator that performs basic arithmetic operations.

Features:

- Addition, Subtraction
- Multiplication, Division
- User-friendly menu
- Handle invalid input



Example (add):

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Sum =", a + b)
```

Key Concepts: Functions, Conditionals

3. PASSWORD GENERATOR

Generates strong and random passwords based on user preferences.

Features:

- Choose length of password
- Include letters, numbers and symbols
- Random and secure



Example (generate):

```
import random, string
chars = string.ascii_letters +
        string.digits + string.punctuation
password = "".join(random.choice(chars)
                    for _ in range(12))
```

Key Concepts: Random Module, Strings

4. NUMBER GUESSING GAME

The computer thinks of a number and you have to guess it.

Features:

- Random number generation
- Hints (too high / too low)
- Count attempts
- Play again option



Example (check guess):

```
if guess < number:
    print("Too low!")
elif guess > number:
    print("Too high!")
else:
    print(f"Correct! You guessed in {attempts} tries.")
```

Key Concepts: Loops, Conditionals, Random

5. STUDENT GRADE BOOK

Store student names and their marks and display grades.

Features:

- Add student and marks
- View all records
- Calculate average
- Assign grades



Example (average):

```
avg = sum(marks) / len(marks)
if avg >= 90: grade = "A"
elif avg >= 75: grade = "B"
elif avg >= 50: grade = "C"
else: grade = "D"
```

Key Concepts: Lists, Dictionaries, Functions

6. EXPENSE TRACKER

Track your daily expenses and view summaries.

Features:

- Add, view and delete expenses
- Category-wise tracking
- Total and summary
- Data saved in file



Example (add expense):

```
date = input("Date (YYYY-MM-DD): ")
amount = float(input("Amount: "))
category = input("Category: ")
with open("expenses.csv", "a") as f:
    f.write(f"{date},{category},{amount}\n")
```

Key Concepts: File Handling, CSV, Functions

7. WEATHER APP (CLI)

Get current weather info of any city using an API.

Features:

- Get temperature, humidity and weather description
- Search by city name
- Uses public API



Example (using OpenWeatherMap API):

```
import requests
city = input("City: ")
api_key = "YOUR_API_KEY"
url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
data = requests.get(url).json()
print(data["weather"][0]["description"])
```

Key Concepts: APIs, Requests, JSON

8. QUIZ GAME

A fun quiz game with multiple choice questions.

Features:

- Multiple questions
- Score calculation
- Final result
- Play again option



Example (check answer):

```
if answer.lower() == correct_answer:
    score += 1
    print("Correct!")
else:
    print("Wrong!")
```

Key Concepts: Lists, Conditionals, Functions

9. CONTACT BOOK

Store and manage contacts.

Features:

- Add, view, search, update, delete
- Store name, phone, email
- Data persistence



Example (add contact):

```
name = input("Name: ")
phone = input("Phone: ")
email = input("Email: ")
contacts[name] = {"phone": phone, "email": email}
save_contacts(contacts)
```

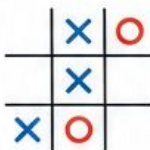
Key Concepts: Dictionaries, File Handling

10. TIC TAC TOE

Play Tic Tac Toe against another player in the console.

Features:

- 2 Player game
- Display board
- Win/Draw detection
- Play again option



Example (check win):

```
def check_winner(board):
    # check rows, columns and diagonals
    pass
```

Key Concepts: Lists, Functions, Logic Building

11. TEXT ANALYZER

Analyze any text or file.

Features:

- Word count
- Character count
- Line count
- Most common words



Example (word count):

```
text = open("sample.txt").read()
words = text.split()
print("Total words:", len(words))
```

Key Concepts: File Handling, Strings, Collections

12. LIBRARY MANAGEMENT SYSTEM

Manage books and issued records.

Features:

- Add books
- Issue/Return books
- View available books
- Store data in files



Example (add book):

```
title = input("Book title: ")
author = input("Author: ")
with open("books.txt", "a") as f:
    f.write(f"{title}|{author}\n")
```

Key Concepts: OOP (Basic), File Handling



The best way to learn Python is to **BUILD!**
Start small. Stay consistent. Build amazing things!

